EHMbrAIn

AI-based versus traditional Engine Health Monitoring:
a gas path analysis testbed on the CFM56-7B with synthetic ground truth

— (anonymous in the public repository) —

July 6, 2026

Living document: updated at every project milestone.

Contents

Notation and acronyms

Acronyms

BPR	bypass ratio	LOO	leave-one-out (cross-validation)
CEA	NASA Chemical Equilibrium with Applications	LPC	low-pressure compressor (booster)
CI	continuous integration	LPT	low-pressure turbine
C-MAPSS	NASA turbofan simulation benchmark	MAE	mean absolute error
EEDB	ICAO Engine Emissions Databank	OPR	overall pressure ratio
EGT(M)	exhaust gas temperature (margin)	PC	power code: fraction of reference thrust
EHM	engine health monitoring	PCS	Physics-Consistency Score (ours, C5)
FADEC	full-authority digital engine control	PHM	prognostics and health management
FAR	fuel–air ratio	RMSE	root-mean-square error
FOD	foreign-object damage	RUL	remaining useful life
FPR	false-positive rate	SAC	single annular combustor
GPA	gas path analysis	SHAP	Shapley additive explanations
HPC	high-pressure compressor	SLS	sea-level static
HPT	high-pressure turbine	SVD	singular value decomposition
ICM	influence coefficient matrix	TCDS	type-certificate data sheet
ISA	international standard atmosphere	TSFC	thrust-specific fuel consumption
N1, N2	low-/high-pressure spool speeds	WLS	weighted least squares

Symbols

$\theta,\ \delta$	inlet total temperature / pressure ratios to ISA sea level	$\mathbf{H}$	influence coefficient matrix
$\Delta\eta$	component efficiency deviation [%]	$\mathbf{Q},\ \mathbf{R}$	process / measurement noise covariance
$\Delta\Gamma$	component flow-capacity deviation [%]	$\mathbf{P}_0,\ \lambda$	prior shape / strength of the WLS regularization
$\mathbf{x}$	health-parameter vector (10 components)	$N1_c$	corrected fan speed $N1/\sqrt{\theta}$
$\Delta\mathbf{z}$	measurement deviations from baseline	T_{t4}	combustor-exit total temperature
$W_f,\ F_n$	fuel flow, net thrust	ΔT_{ISA} (dTs)	ambient offset from the ISA day

Reading order. Every technical term is defined at first use; Chapter 2 builds the engine and GPA foundations, Section 3.3 the machine-learning toolbox. The hypotheses table (Table 1.1) can be read superficially on a first pass and revisited after those two stops.

Chapter 1

Introduction

This chapter in brief: jet engines degrade invisibly, airlines monitor them through a handful of measurements, and two schools of methods compete to interpret those measurements. This project builds a level playing field to compare them. If you are new to the field, skim the hypotheses table on first read and return to it after Chapter 2.

Key idea. The field compares AI against weakly-implemented traditional baselines. The only honest comparison gives both families *identical* data, metrics, splits and tuning budget; everything in this document follows from insisting on that symmetry.

1.1 Motivation

A commercial jet engine degrades from its very first flight. Dust and pollution stick to compressor blades, hot-section parts creep and oxidize, tip clearances open up, seals wear. The airline cannot see any of this directly —the engine stays on the wing for years— so it watches the engine's *symptoms* instead: the temperatures, pressures, shaft speeds and fuel flow recorded on every flight. The discipline that turns those symptoms into maintenance decisions is called *Engine Health Monitoring* (EHM). The stakes are large: a single shop visit for an engine overhaul costs millions of dollars, and an unscheduled engine removal disrupts an entire fleet schedule.

The classical analytical core of EHM, in use since the 1970s, is *Gas Path Analysis* (GPA) [1]. The idea: each physical fault (a dirty compressor, an eroded turbine) leaves a characteristic fingerprint on the measured parameters; by inverting a thermodynamic model of the engine, one can estimate *which* component has deteriorated and *by how much* from the measurements alone. Around GPA the industry built a mature toolbox —baseline models, exponential trend smoothing, Kalman filters, expert fault-isolation rules [2]— which this project collectively calls *traditional EHM*.

Over the last decade, machine learning (ML) has produced striking results in the adjacent research field of prognostics: predicting the *Remaining Useful Life* (RUL) of an engine, i.e. how many flights remain before it must come off the wing, mostly on synthetic benchmark datasets such as NASA's C-MAPSS [3] and N-CMAPSS [4]. Yet the literature has a structural weakness: **AI methods are almost never compared against a traditional pipeline implemented with equal care**, on the same data, with the same metrics and the same tuning effort. The result is a stream of comparisons against straw men, which makes it impossible to know what AI actually contributes, under which conditions, and at what cost.

1.2 Objective

This project builds a reproducible testbed in which both EHM families —traditional and AI-based— observe *exactly the same data* from a synthetic fleet of CFM56-7B26 turbofans and are scored with *exactly the same metrics*. The methodological key is that the synthetic data carries **complete ground truth**: the true health state of every engine component (its efficiency and flow-capacity deviations, defined in Section 2.4) at every flight. With real airline data this is impossible —nobody knows the true internal state of an engine on the wing— but with synthetic data one can measure not only whether a method detects or predicts well, but whether it *attributes the damage to the correct component*. That attribution question is precisely where traditional GPA is weakest (the *smearing* phenomenon, Section 2.4) and where AI is least transparent (the black-box problem).

Audience and register. This document is a research work aimed at two readers at once: the engineering organization deciding whether and how to integrate AI into its engine health monitoring practice, and the researcher looking for a rigorous, reproducible benchmark at the knowledge frontier. That dual audience sets the register: every method is pushed to publishable rigor (pre-registration, statistical tests, audited datasets), and every concept is introduced without assuming prior specialization — chapter briefs, worked numeric examples and a notation front-matter keep the document self-contained. Advancing the frontier and being readable are treated as the same job, not competing ones.

Since neither a performance model of the engine nor degradation data were available to the project, both are built from scratch: a thermodynamic "digital twin" of the CFM56-7B26 in pyCycle [5], calibrated against certified public data (Chapter 4), and a fleet degradation generator with physically derived fault signatures (Chapter 6).

1.3 Hypotheses

Each hypothesis is formalized with quantitative thresholds *before* any test-set result is seen (the pre-registration protocol of Section 3.6). All are tested with a Wilcoxon signed-rank test paired by engine, Holm correction for multiple comparisons ($\alpha = 0.05$), and BCa bootstrap confidence intervals. In plain words: for every comparison we check that the improvement is statistically significant, not a fluke of one lucky engine. The table below necessarily uses terms defined later —the GPA quantities in Chapter 2, the machine-learning methods in Section 3.3, every metric in the metrics section of Chapter 3, and all acronyms in the notation chapter— so it is meant to be skimmed now and revisited after those stops.

Table 1.1: Project hypotheses and their confirmation criteria.

ID	Hypothesis	Confirmation criterion
H1	AI detects faults earlier and with fewer false alarms than trend monitoring	median lead time $\geq +20\,\%$ and false-positive rate $\leq 0.5\times$ versus trending, at a fixed recall of 0.9; $p < 0.05$
H2	AI isolates faults with overlapping signatures better	+10 percentage points of isolation accuracy on the "confusable" subset (fault signatures less than 15° apart in the influence-matrix space) versus weighted-least-squares GPA; $p < 0.05$
H3	AI predicts RUL better	simultaneous improvement in RMSE and in the asymmetric NASA PHM08 score; $p < 0.05$
H4	The physics-informed hybrid dominates when data is scarce	RUL RMSE $\leq$ pure AI with $100\,\%$ of the training data and strictly better with $10\,\%$ and $25\,\%$; $p < 0.05$
H5	Conformal intervals are better calibrated than the Kalman covariance	smaller $\lvert$empirical coverage $- 0.9\rvert$ with mean interval width no worse than $+20\,\%$

Refuting any hypothesis is itself a publishable result: pre-registration is what makes the refutation credible. Reformulating hypotheses after the repository tag `prereg-v1` is forbidden. One operationalization was refined *at* the freeze, before any test-set run and disclosed in the frozen document: H1's "fixed recall 0.9" proved unreachable at the v1.1 fault magnitudes, so its operating point was redefined per family under a false-alarm budget (Section 9.2). Changing an operating definition at freeze time, on the record, is legitimate; changing a threshold after seeing the answer is what the tag forbids.

1.4 Contributions

In plain terms first, before the technical names: the project builds a realistic test dataset with hidden answers no real fleet can provide (C1), runs a scrupulously fair contest between the two method families (C2), and from it delivers a physics-informed hybrid (C3), calibrated uncertainty (C4), a physics-anchored explainability score (C5), a sensor-and-data trade map (C6), an external reality check (C7), and — the furthest reach — two per-engine certificates, proven against the truth, of what a diagnosis can honestly know (C8) and how much of an engine's future its present can reveal (C9). Each item below names the technique; every term is defined where it is first used, so this list is a preview, not a prerequisite.

C1. **SynCFM56**: the first public synthetic GPA dataset for a specific commercial engine (CFM56-7B26), with per-flight health-parameter ground truth and an ACARS-style snapshot format (one takeoff report and one cruise report per flight, the way real airline EHM data actually arrives), more realistic than the continuous time series of C-MAPSS.

C2. A head-to-head comparison under a **pre-registered protocol** with a symmetric tuning budget (the same number of Optuna trials) for both families.

C3. A physics-informed hybrid with three separately ablated mechanisms: residual features against the digital twin, physically constrained loss functions, and GPA→ML stacking.

C4. Compared uncertainty quantification: conformal-prediction RUL intervals versus the Kalman filter covariance.

C5. The **Physics-Consistency Score (PCS)**: a new explainability metric that projects SHAP attributions into the physical health-parameter space through the influence coefficient matrix.

C6. A determinability map: a sensors × noise × data-size ablation showing *when* each approach wins.

C7. Sim-to-real validation: the full methodology is re-run on N-CMAPSS to check that the method ranking is not an artifact of our own simulator.

C8. **The earned-confidence certificate** (Chapter 13): a per-engine, history-resolved identifiability guarantee for gas-path diagnosis — and, uniquely, a *proof against ground truth* that it is honest ($\rho = 0.70$ between certified precision and actual error). It states exactly which health directions an engine's data lets one conclude, and doubles as a sensor-acquisition instrument (predicting that station probes rescue HPC efficiency 45×). The validation is one only a synthetic-truth benchmark can perform.

C9. **The prognostic floor** (Section 13.5): the predictive counterpart of C8. By conditioning on each engine's *true* current health state it measures, against ground truth, the aleatoric floor on remaining-life error — the RMSE no method can beat. It shows 87 % of mid-life RUL uncertainty is irreducible, yet a 4× reducible gap opens late in life, telling an operator *where* a better prognostic is worth building and where the future is simply unknowable.

1.5 Document structure

The document follows the work in order, and a reader can stop wherever their interest ends. Chapter 2 builds the foundations from zero: how a turbofan works, what is measured on the wing, and the mathematics of gas-path diagnosis — no prior knowledge assumed. Chapter 3 lays out the testbed and the fairness safeguards that make the comparison trustworthy. The next three chapters build the ingredients: the thermodynamic twin of the engine (Chapter 4), what the data *is* and where it comes from (Chapter 5), and the synthetic fleet generated from it (Chapter 6). The two EHM families are then implemented and measured — traditional (Chapter 7) and AI (Chapter 8) — and adjudicated head to head under the pre-registered protocol (Chapter 9), with five real engines telling the story in operations language (Chapter 10).

The remaining chapters push past the main study. Chapter 11 asks whether more observability can be bought without new hardware; Chapter 12 turns each declared limitation into its own experiment; Chapter 13 is the project's furthest reach, the ground-truth-validated certificate; and Chapter 14 asks, cautiously, what any of it is worth. Chapter 15 closes. The appendices hold the decision register (every choice and its justification), the dated milestone log, and a plain-language deep dive on how each machine-learning method works (Chapter C).

Chapter 2

Background

This chapter in brief: how a turbofan works and what its two spools are; which handful of numbers an airline actually sees; why raw readings must be corrected before comparison (with a worked example); how gas path analysis turns corrected deviations into a health diagnosis; a two-line toy problem showing exactly how that diagnosis fails (smearing); and how real engines physically degrade. Everything later builds on these six ideas.

Key idea. Each fault leaves a signature (a column of the influence matrix). When two signatures are nearly parallel, a single noise-sized measurement error flips the diagnosis to the wrong component — *smearing* — and it is the phenomenon this whole project revolves around.

This chapter assumes no prior knowledge of gas turbines. It introduces the engine, the measurements available in service, the mathematics of gas path diagnosis, and the physics of degradation —everything needed to follow the rest of the document.

2.1 How a two-spool turbofan works

A turbofan produces thrust by accelerating air backwards. Air enters through the *fan* (the large visible rotor); most of it —the *bypass* stream— goes around the core and exits through the bypass nozzle, producing the majority of the thrust. The rest enters the *core*: it is compressed by a low-pressure compressor (the *booster*) and a high-pressure compressor (HPC), heated by burning fuel in the *combustor*, and expanded through a high-pressure turbine (HPT) and a low-pressure turbine (LPT) that extract the work needed to drive the compressors, before leaving through the core nozzle.

The machine is arranged in two mechanically independent *spools* (concentric shafts):

- the **low-pressure (LP) spool**: fan + booster driven by the LPT, rotating at speed **N1**;
- the **high-pressure (HP) spool**: HPC driven by the HPT, rotating at speed **N2**.

The ratio of bypass to core mass flow is the *bypass ratio* (BPR); the ratio of compressor exit pressure to inlet pressure is the *overall pressure ratio* (OPR). Higher BPR and OPR generally mean better fuel efficiency, measured as *thrust-specific fuel consumption* (TSFC): fuel flow per unit of thrust. Figure 2.1 shows the layout and the station numbers used throughout this document.

2.2 The subject engine: CFM56-7B26

The CFM56-7B (CFM International, a Safran–GE joint venture) powers the Boeing 737NG family and is one of the most widely operated commercial engines in history, which guarantees

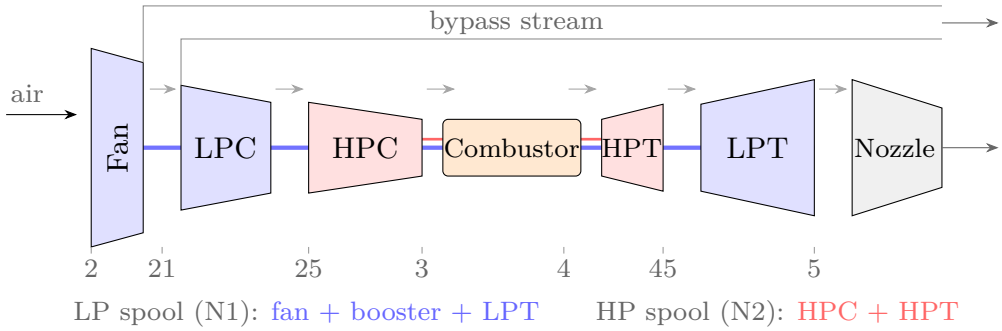

Figure 2.1: Two-spool separate-flow turbofan layout with the SAE station numbers used in this document. The EGT probe of the real CFM56-7B sits inside the LPT (station 49.5); this model uses station 45 (HPT exit) as its consistent EGT proxy.

abundant public type data. The **-7B26** variant (117 kN takeoff thrust, Boeing 737-800) is chosen as the most common one. Its type-certificate data sheet (TCDS) — the airworthiness authority's official record of certified performance, here from the European regulator EASA, document E.004, issue 07 [6]) describes: a single-stage fan, 3-stage booster, 9-stage HPC, annular combustor, single-stage HPT and 4-stage LPT, governed by a dual-channel FADEC (a full-authority digital engine control —the computer that meters fuel). The pilot-facing thrust-setting parameter is N1, not engine pressure ratio.

Table 2.1: Certified CFM56-7B26 data used as the calibration contract. Sources: TCDS EASA E.004 issue 07 [6] and the ICAO Engine Emissions Databank, edition 03/2026, UID 3CM033 [7].

Quantity	Value	Source	Status
Takeoff thrust	11 699 daN (= 116.99 kN)	TCDS	verified
Maximum continuous thrust	11 521 daN	TCDS	verified
N1 redline (104 %)	5382 rpm ⇒ 100 % = 5175 rpm	TCDS	verified
N2 redline (105 %)	15 183 rpm ⇒ 100 % = 14 460 rpm	TCDS	verified
Max. EGT, takeoff / continuous	950 °C / 925 °C (displayed)	TCDS	verified
Dry weight	2386 kg	TCDS	verified
Bypass ratio	5.1	EEDB	verified
Pressure ratio (sea-level static, rated)	27.61	EEDB	verified
Fuel flow at 100/85/30/7 % F_{00}	1.221 / 0.999 / 0.338 / 0.113 kg s^{-1}	EEDB	verified

Two subtleties from the TCDS matter for modeling and are frequently overlooked:

- The certified *exhaust gas temperature* (EGT) —the single most important health indicator, because it rises as the engine deteriorates— is measured at station **T49.5** (the stage-2 LPT nozzle), *not* at the HPT exit. Moreover, the *displayed* cockpit value adds a "shunt" function of +30 °C (for single-annular-combustor engines, active above 8500 rpm N2) plus a model-dependent "trim" (zero for the -7B26).
- Air bleed limits (air extracted from the compressor for the aircraft's cabin and anti-ice systems) are tabulated per extraction stage and shaft speed; at rated speed the 9th-stage bleed is capped at 7 % of core flow.

2.3 What is measured, and why parameters must be corrected

Engine locations are numbered per the SAE ARP755A standard [8]: station 2 is the fan inlet, 21 the fan exit, 25 the HPC inlet, 3 the HPC exit, 4 the combustor exit, 45 the HPT exit, and

5 the LPT exit. The measurements available in airline service (the "cockpit set") are N1, N2, EGT and fuel flow W_f; some installations add pressures and temperatures at stations 25 and 3.

A raw measurement is meaningless on its own because it depends on the day: the same healthy engine reads a higher EGT on a hot day at a high-altitude airport than on a cold day at sea level. To compare flights, measurements are *corrected* to standard sea-level conditions using $\theta = T_{t2}/288.15\,\mathrm{K}$ and $\delta = P_{t2}/101.325\,\mathrm{kPa}$ (the ratios of inlet total temperature and pressure to their standard-day values):

$$N_{\mathrm{corr}} = \frac{N}{\sqrt{\theta}}, \qquad W_{f,\mathrm{corr}} = \frac{W_f}{\delta\,\theta^{a}}, \qquad \mathrm{EGT}_{\mathrm{corr}} = \frac{\mathrm{EGT}}{\theta}. \tag{2.1}$$

The exponent a (typically 0.5–0.7) is often copied from generic tables; here it will instead be fitted by regression on sweeps of our own thermodynamic model, removing a classical source of systematic baseline error.

Worked example. A healthy engine reads 880 K of EGT on a standard day ($\theta = 1$). On an ISA+30 day the inlet runs at 318 K, so $\theta = 318/288.15 = 1.104$, and the same healthy engine reads 966 K. A naive comparison would report an "86 K deterioration" where none exists. Corrected with the fitted exponent $a = 0.95$: $966/1.104^{0.95} = 879\,\mathrm{K}$ — the hot-day reading collapses onto the standard-day one, and only *genuine* health changes remain visible. This is why every deviation in this project is computed in corrected space.

2.4 Gas Path Analysis

We now have the pieces — an engine, a handful of corrected measurements — and can state the central problem: recovering the engine's hidden internal condition from those few numbers. *Gas Path Analysis* (GPA) is the classical way to do it, and the rest of this section builds it up: what "condition" means as a vector, how it maps to the measurements, why inverting that map is hard, and the failure mode (smearing) that the whole project circles.

2.4.1 Health parameters

GPA describes the internal state of the engine with two numbers per turbomachine: the deviation of its *isentropic efficiency* $\Delta\eta$ (how much of the ideal work it actually achieves —dirt and wear reduce it) and the deviation of its *flow capacity* $\Delta\Gamma$ (how much corrected flow it swallows at a given speed —fouling reduces it in compressors; turbine erosion, which opens the effective nozzle area, increases it). With five turbomachines (fan, booster, HPC, HPT, LPT) the health state is the 10-component vector

$$\mathbf{x} = [\Delta\eta_{\mathrm{fan}}, \Delta\Gamma_{\mathrm{fan}}, \Delta\eta_{\mathrm{LPC}}, \Delta\Gamma_{\mathrm{LPC}}, \Delta\eta_{\mathrm{HPC}}, \Delta\Gamma_{\mathrm{HPC}}, \Delta\eta_{\mathrm{HPT}}, \Delta\Gamma_{\mathrm{HPT}}, \Delta\eta_{\mathrm{LPT}}, \Delta\Gamma_{\mathrm{LPT}}]^{\mathsf{T}}, \tag{2.2}$$

expressed in percent relative to a new engine. $\mathbf{x}$ is what maintenance actually wants to know, and it is not directly measurable on the wing.

2.4.2 The linear model

What *is* measurable are deviations $\Delta\mathbf{z}$ of the corrected parameters from a *baseline* (the value a new engine would show at the same operating condition $\mathbf{u}$: altitude, Mach number, N1 setting, ambient temperature deviation, bleed status). For the small deviations typical of in-service deterioration, measurement and health deviations are related linearly:

$$\Delta\mathbf{z} = \mathbf{H}(\mathbf{u})\,\mathbf{x} + \mathbf{S}\,\mathbf{b} + \mathbf{v}, \qquad \mathbf{v} \sim \mathcal{N}(\mathbf{0}, \mathbf{R}), \tag{2.3}$$

where $\mathbf{H}$ is the *influence coefficient matrix* (ICM) —the Jacobian $\partial\mathbf{z}/\partial\mathbf{x}$, i.e. a table of "if the HPT loses 1 % efficiency, EGT rises by this much and fuel flow by that much"— computed by finite differences on the thermodynamic model at the operating condition $\mathbf{u}$; $\mathbf{b}$ collects sensor biases and $\mathbf{S}$ is the *selection matrix* that routes each bias to its own measurement row — one column per biased sensor, all zeros except a single one, so that a bias on the EGT probe shifts only the EGT equation. Estimating $\mathbf{b}$ alongside $\mathbf{x}$ is optional and costs observability; an undetected drift therefore fools the estimator, which is exactly the sensor fault Chapter 6 injects. Finally $\mathbf{v}$ is measurement noise with covariance $\mathbf{R}$.

2.4.3 Estimation, and why it is hard

Inverting Eq. (2.3) is an ill-posed problem. With 4 cockpit measurements and 10 unknowns the system is underdetermined ($\text{rank}(\mathbf{H}) \leq 4$): infinitely many health states explain the same measurements. The weighted-least-squares (WLS) answer regularizes the inversion with prior knowledge of plausible deterioration:

$$\hat{\mathbf{x}} = \left(\mathbf{H}^{\mathsf{T}}\mathbf{R}^{-1}\mathbf{H} + \lambda\,\mathbf{P}_0^{-1}\right)^{-1}\mathbf{H}^{\mathsf{T}}\mathbf{R}^{-1}\Delta\mathbf{z}, \tag{2.4}$$

where the two regularization pieces play distinct roles: $\mathbf{P}_0$ (a diagonal matrix of expected deterioration magnitudes squared) sets the *shape* of the prior — which parameters are allowed to move more — while the scalar λ sets its *strength*, trading fidelity to the measurements against fidelity to the prior. λ is a tuning knob of the traditional pipeline and receives the same optimization budget as the AI's hyperparameters (Section 3.6).

Because deterioration evolves slowly over thousands of flights, tracking it as a *random walk* with a Kalman filter improves on flight-by-flight estimation. The Kalman filter is a recursive estimator that blends the previous health estimate with each new measurement, weighting each by its uncertainty:

$$\mathbf{x}_k = \mathbf{x}_{k-1} + \mathbf{w}_k, \quad \mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{Q}); \qquad \mathbf{z}_k = \mathbf{H}(\mathbf{u}_k)\,\mathbf{x}_k + \mathbf{v}_k, \tag{2.5}$$

with process noise $\mathbf{Q}$ calibrated to realistic degradation rates and the state optionally augmented with sensor biases [2]. At recorded compressor-wash events the recoverable part of the state is reset.

2.4.4 Smearing: a two-parameter toy example

Before the formal statement, a miniature GPA problem shows the failure mode this whole project revolves around. Suppose only two health parameters (HPC and HPT efficiency) and two measurements (EGT and W_f deviations, in %), with influence matrix

$$\mathbf{H} = \begin{pmatrix} 1.0 & 0.9 \\ 0.5 & 0.5 \end{pmatrix},$$

whose columns —the two fault signatures— point in nearly the same direction.

What "the same direction" means (and why we measure it in degrees). Each fault produces its own *pattern* of meter movements: an HPC fault might push EGT up a lot and fuel flow up a little, an HPT fault might push both up in slightly different proportions. Collect those movements into a list and you can picture each fault as an *arrow* in the space of measurements — the arrow's direction is the fault's fingerprint, the proportion in which it moves the meters. Diagnosis is then just asking: which fault's arrow does the observed reading point along? The catch is the *angle between two faults' arrows*. If two faults push the meters in almost the same proportions, their arrows are almost parallel — a small angle — and their readings look nearly

identical, so noise easily makes one masquerade as the other. If the arrows are far apart — a large angle — the two faults look different and are easy to tell apart. The angle is the single number that says how separable two faults are: 90° means completely independent fingerprints (trivial to distinguish), 0° means literally identical (impossible), and somewhere below about 15° ordinary sensor noise blurs them into one. In the matrix above the two columns sit only a couple of degrees apart — which is exactly why the next lines go wrong.

Let the truth be an HPC fault only: $\mathbf{x} = (-1, 0)^\mathsf{T}$, giving exact measurements $\Delta\mathbf{z} = (-1.0, -0.5)^\mathsf{T}$, and indeed $\mathbf{H}^{-1}\Delta\mathbf{z} = (-1, 0)^\mathsf{T}$ recovers it. Now perturb the first measurement by just +0.1 (a 10 % error, comparable to one noise standard deviation):

$$\mathbf{H}^{-1}\begin{pmatrix}-0.9\\-0.5\end{pmatrix} = \begin{pmatrix}10 & -18\\-10 & 20\end{pmatrix}\begin{pmatrix}-0.9\\-0.5\end{pmatrix} = \begin{pmatrix}0\\-1\end{pmatrix}.$$

The estimate now says: HPC healthy, *HPT* degraded by 1 % — a single noise-sized error flipped the diagnosis to the wrong component entirely. Nothing is wrong with the algebra; the information simply is not in the measurements when signatures nearly coincide.

2.4.5 Smearing in general

That is *smearing*: when $\mathbf{H}$ is ill-conditioned —fault signatures nearly parallel— noise makes the estimator distribute a fault that is physically concentrated in one component across several, or attribute it to the wrong one; it is the classical weakness of linear GPA [9]. A singular-value decomposition (SVD) of the ICM quantifies this weakness (rank, condition number, and the signature angles introduced above) and defines the "confusable" subset used by hypothesis H2: fault pairs whose signature arrows are less than 15° apart — the threshold below which, on our sensor noise, two faults are no longer reliably separable. Figure 2.2 shows this for the real engine at cruise: with the cockpit set several pairs fall below the 15° line — the HPC-versus-HPT efficiency pair sits at barely 1.3° (near-identical fingerprints, all but indistinguishable) — while the extended gas-path sensors, by adding meters that the two faults move differently, swing the arrows apart and lift every pair well clear of the line. The angle is geometry, not yet a result; whether it translates into isolation accuracy is what hypothesis H2 tests (Section 9.3) and what the sensor analysis of Chapter 13 returns to.

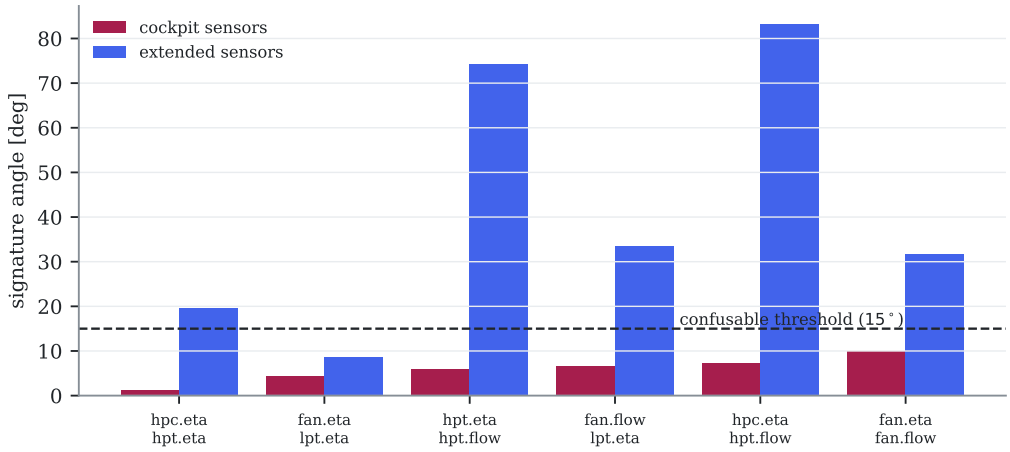

Figure 2.2: Signature angles of the cockpit-confusable fault pairs at cruise. Below the dashed 15° line the pair is confusable; the cockpit set (red) leaves several pairs there, the extended set (blue) rescues them. Auto-generated.

2.5 The physics of degradation

The health vector says *how much* each component has drifted; this section says *why* — the physical mechanisms that move those numbers, and the characteristic shape each leaves over an engine's life. These are what the synthetic fleet of Chapter 6 must reproduce, so they are worth meeting first. Table 2.2 summarizes them.

Table 2.2: Degradation mechanisms modeled in this project and their typical effects, per the literature [10, 11].

Mechanism	Component	Typical effect	Time profile	Recoverable
Fouling (dirt buildup)	fan, LPC, HPC	$\Delta\Gamma$ $-1\ldots-4\,\%$; $\Delta\eta$ $-0.5\ldots-2.5\,\%$	saturating exponential	wash recovers 30–70 %
Erosion	compressors	$\Delta\eta$ $-0.5\ldots-1.5\,\%$	slow linear	no
Tip-clearance opening	HPC/HPT	$\Delta\eta$ $-0.5\ldots-1.5\,\%$	fast initially, then slow	no
Hot-section deterioration	HPT	$\Delta\eta$ $-1\ldots-3\,\%$; $\Delta\Gamma$ $+1\ldots+2\,\%$	linear, accelerating	no
Foreign-object damage (FOD)	fan/HPC	step $\Delta\eta$ $-0.5\ldots-2\,\%$	step (Poisson arrivals)	no
Sensor drift	any probe	e.g. EGT $+1\ldots+5$ °C per 1000 cycles	ramp	recalibration

The aggregate observable effect is the erosion of the *EGT margin*: the distance between the EGT reached on a hot-day takeoff and its certified limit. A new engine has 70–100 °C of margin; when the margin reaches zero the engine can no longer legally produce rated takeoff thrust on a hot day and must come off the wing. Its characteristic pattern over an engine's life —fast early loss, slower steady erosion, sawtooth recoveries at each compressor wash— is what the fleet generator of phase F2 must reproduce. Table 2.2 summarizes the mechanisms, their signatures and their time profiles; its magnitudes parameterize that generator.

Chapter 3

Methodology and testbed architecture

This chapter in brief: the project is six chained phases, each ending in a pass/fail gate (Fig. 3.1). Four safeguards keep the AI-versus-traditional comparison honest; one section explains every machine-learning tool by name before it is used; and the software inventory maps each program to its role. Read Section 3.3 even if you skip the rest.

Key idea. Credibility comes from pre-registration: hypotheses, thresholds and tests are frozen before the test data is seen, and a failed gate triggers redesign or an honest refutation, never a quiet threshold move. A project that cannot fail its own gates is advertising, not research.

3.1 Overall architecture

Before the details, the shape of the whole project. It is a chain of phases, each producing a versioned artifact the next consumes and each guarded by a gate it must pass before the next begins (Fig. 3.1); the rest of this chapter, and the rest of the document, walk that chain. The gates are the spine: they are what turn a sequence of scripts into an argument that can be checked.

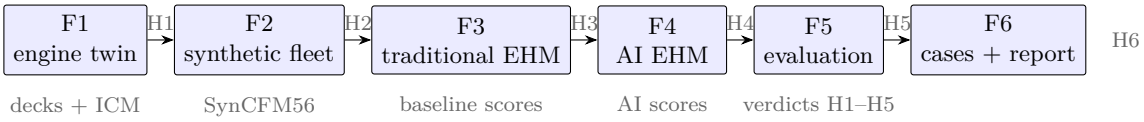

Figure 3.1: The six project phases, their gates (H1–H6) and their key artifacts. Everything to the left of an arrow must pass its gate before the next phase starts.

The testbed is organized in six chained phases, each closed by a milestone (*gate*) with a measurable acceptance criterion:

F1. Thermodynamic twin of the CFM56-7B26 in pyCycle: design point, off-design behavior, baseline decks over the flight envelope (pre-computed tables of what a healthy engine reads at every operating condition) and the influence coefficient matrix. Gate H1: error $\leq 3\,\%$ against certified data at the anchor points.

F2. Synthetic fleet generator (SynCFM56): per-mechanism health trajectories, a hierarchical fleet of $\sim$100 engines run to failure, a sensor model (noise, bias, quantization, missing data) and ACARS-style snapshots with ground truth. Gate H2: dataset audited for realism and difficulty, no leakage between data splits, and a written datasheet.

F3. Traditional EHM reference: baselines and smoothed deviations, trend monitoring with persistence rules, WLS GPA (Eq. (2.4)), Kalman GPA (Eq. (2.5)), expert-rule fault isolation, and RUL by robust extrapolation of the EGT margin. Gate H3: full metrics on the test set.

F4. AI-based EHM: anomaly detection (autoencoders, Isolation Forest), multi-class fault diagnosis (gradient boosting, neural networks), RUL prognosis (LSTM/TCN/Transformer sequence models), the physics-informed hybrid, conformal uncertainty and physical explainability (PCS). Gate H4.

F5. Pre-registered comparative evaluation: common metrics, statistical tests, ablations, and sim-to-real validation on N-CMAPSS. Gate H5.

F6. Case studies, interactive dashboard, and final writing. Gate H6.

3.2 Machine learning without prior background

(Readers who want the full mechanics — how a network is trained, how each method works — will find them in Chapter C; this section gives only the minimum to read on.)

Nothing in this document assumes machine-learning training. Six ideas carry everything:

Learning from data. A machine-learning model is a mathematical function with many adjustable numbers inside (its *parameters*). "Training" means showing it examples (here: deviation histories with known outcomes, because the fleet is synthetic) and letting an algorithm nudge those numbers until the function's outputs match the known answers. Nobody programs the rules; the rules emerge from the examples.

Train / validation / test. Three disjoint groups of engines with three jobs. *Training* engines are the textbook the model studies. *Validation* engines are practice exams: used to choose between model variants and settings, never to adjust parameters directly. *Test* engines are the final exam, sat exactly once — any peeking turns the grade into fiction. The same discipline is applied to the traditional pipeline's thresholds.

Overfitting. A model with enough capacity can memorize its textbook — perfect on training engines, useless on new ones. That is why every number that matters in this document comes from engines the models never saw, and why splitting by *engine* (not by flight) is non-negotiable: flights of the same engine are near-duplicates.

Neural networks, in one paragraph. A neural network is a chain of simple layers: each layer multiplies its inputs by adjustable weights, sums them, and passes the result through a mild nonlinearity. Stacked layers can approximate very general input–output relationships. Training adjusts the weights by *gradient descent*: compute how wrong the output is, compute in which direction each weight should move to reduce that error, take a small step, repeat over the dataset for a number of *epochs* (passes). A *recurrent* network (like the GRU used here) additionally carries a small memory from one time step to the next, so it can read a sequence — an engine's deviation history — the way a reader accumulates context along a sentence.

Hyperparameters and tuning. Settings the training process itself does not adjust — network size, learning-rate, window lengths, alert thresholds — are *hyperparameters*. Choosing them by trial and error on validation data is *tuning*; automating that search is what Optuna does. Both EHM families received exactly 50 tuning trials (Section 3.6): the comparison is between tuned practitioners, not between a tuned one and a default one.

Seeds and variance. Training involves randomness (initial weights, sampling order). A *seed* fixes that randomness so runs are repeatable; training with several seeds and reporting the spread separates a real effect from a lucky draw — which is exactly how the hybrid experiment's negative result was established.

3.3 The machine-learning toolbox, briefly

With those six ideas in hand, the specific techniques, one paragraph each:

Autoencoder (anomaly detection). A neural network trained to compress and then reconstruct *healthy* data. Fed a degraded snapshot, it reconstructs it poorly; the reconstruction error is the anomaly score. No fault labels needed.

Isolation Forest (anomaly detection). An ensemble of random trees; anomalous points are isolated in fewer random splits than normal ones. A classical non-neural baseline.

Gradient boosting / XGBoost (diagnosis). An ensemble of small decision trees, each correcting the errors of the previous ones. State of the art on tabular data; predicts the fault class of a snapshot window.

LSTM, TCN, Transformer (RUL prognosis). Three sequence-model families —recurrent, convolutional and attention-based— that consume a sliding window of past snapshots and regress the remaining useful life. The v0 suite implements a GRU (recurrent); the cross-family comparison belongs to the F5 tuned round.

Conformal prediction (uncertainty). A distribution-free wrapper: from the model's errors on a calibration set it builds prediction intervals with a mathematically guaranteed coverage rate (e.g. 90 %), regardless of the underlying model.

SHAP (explainability). Assigns each input feature its contribution to one specific prediction (a game-theoretic attribution). The PCS (contribution C5) projects these attributions through the ICM into health-parameter space to test their physical coherence.

Optuna (tuning). An automated hyperparameter search library; a "trial" is one training run with a candidate configuration. Both EHM families get the same trial budget.

Supporting infrastructure named later: *DVC* versions datasets the way git versions code; *MLflow* records every training run (configuration, metrics, artifacts) so experiments are traceable and repeatable.

3.4 Designing a fair comparison, as method

The comparison itself is an instrument, and it was designed before any contestant ran. Three principles, stated as axioms because every later choice traces to one of them:

Symmetry. Both families see *identical* inputs (same snapshots, same deviations against the same published baseline), are scored by *identical* metrics on *identical* splits, and receive *identical* optimization budgets (50 tuning trials each, Section 3.5). Any asymmetry — richer features for one side, hand-tuning for the other — silently converts a method comparison into an effort comparison. The literature's chronic disease (Chapter 1) is exactly that conversion.

Evidence hierarchy. Numbers exist in two castes, never mixed: *exploratory* (produced while designing, debugging, iterating — chapters 6–7) and *confirmatory* (produced once, after a frozen pre-registration, on data touched exactly once — chapters 8 and 10). Only the second caste supports verdicts. The discipline is borrowed from clinical trials, where the cost of self-deception is measured in lives; here it is merely measured in wrong fleet decisions.

Gates over milestones. Progress is declared by *passing pre-stated acceptance criteria*, not by finishing work. Gates are allowed to fail (the difficulty audit failed twice, hypothesis H7.3 died on its frozen thresholds), and a failed gate triggers redesign or an honest refutation — never a quiet threshold adjustment. A project that cannot fail its own gates is advertising, not research.

3.5 The statistical machinery, in full

Every verdict in this document rests on the following tests. Each is stated with its mathematics and the reason it was chosen; nothing later in the document uses a statistic not defined here (norm N6).

Wilcoxon signed-rank test (paired continuous outcomes). For paired errors (a_i, b_i) on the same n engines, compute differences $\delta_i = a_i - b_i$, discard zeros, rank $|\delta_i|$ ascending, and sum the ranks of the positive differences:

$$W \;=\; \sum_{i:\,\delta_i>0} \operatorname{rank}(|\delta_i|). \tag{3.1}$$

Under the null (differences symmetric about zero) every rank is equally likely to carry either sign, giving an exact, distribution-free reference: no normality assumption, which matters because RUL errors are heavy-tailed. Pairing by engine removes engine-to-engine difficulty variation from the comparison — the same reason a crossover trial beats two separate cohorts. *Worked example*: $n = 6$ engines with differences $\{+3, +8, -2, +5, +9, +1\}$: ranks of magnitudes $\{3, 7 \ldots\}$ — the two negative values carry little rank mass, W lands near its maximum, and the one-sided exact $p \approx 0.047$.

McNemar's exact test (paired binary outcomes). For paired correct/incorrect outcomes (e.g. both families isolating the same episodes), only the discordant pairs are informative: $b =$ cases only method A got right, $c =$ only method B. Under the null of equal skill each discordant pair is a fair coin, so

$$p \;=\; \Pr\big[X \ge b\big], \qquad X \sim \text{Binomial}(b + c,\ \tfrac{1}{2}), \tag{3.2}$$

one-sided when the hypothesis is directional. The concordant cases cancel — episodes both get right (or both miss) say nothing about the *difference*. This is why Chapter 11 could bound its verdict without replaying the test pass: with 15/23 vs 4/23 correct, the accuracy gap forces $b - c = 11$, and p is maximized at $b = 15, c = 4$, still 0.0096.

Holm correction (multiple comparisons). Testing m hypotheses at $\alpha = 0.05$ each inflates the chance of at least one false confirmation. Holm's step-down procedure sorts the p-values $p_{(1)} \le \cdots \le p_{(m)}$ and compares $p_{(k)}$ against $\alpha/(m - k + 1)$, stopping at the first failure:

$$p^{\text{Holm}}_{(k)} \;=\; \min\Big(1,\ \max_{j \le k}\ (m - j + 1)\, p_{(j)}\Big). \tag{3.3}$$

It controls the family-wise error rate at α with no independence assumption, and is uniformly more powerful than Bonferroni (which would multiply every p by m).

BCa bootstrap confidence intervals. Resample engines (the exchangeable unit) with replacement $B = 10^4$ times, recompute the statistic $\hat{\theta}_b^*$, and read the interval from the bootstrap distribution — but with two corrections: the *bias* term $z_0 = \Phi^{-1}(\#\{\hat{\theta}_b^* < \hat{\theta}\}/B)$ (is the bootstrap distribution off-center?) and the *acceleration* a estimated by jackknife (does the statistic's variance depend on its value?). The adjusted percentiles

$$\alpha_{1,2} \;=\; \Phi\Big(z_0 + \tfrac{z_0 + z_{\alpha_{1,2}}}{1 - a\,(z_0 + z_{\alpha_{1,2}})}\Big) \tag{3.4}$$

replace the naive 2.5/97.5 cuts. For small n (20 test engines) and skewed statistics (RMSE), the corrections matter — the naive percentile interval can miss its nominal coverage badly.

Cliff's delta (effect size). Significance says an effect exists; δ says how big:

$$\delta \;=\; \frac{\#\{(i,j) : a_i > b_j\} - \#\{(i,j) : a_i < b_j\}}{n_a\, n_b} \;\in\; [-1, 1], \tag{3.5}$$

the probability a random error from method A exceeds a random one from B, minus the reverse. $\delta = 0.89$ (the H3 verdict) means: pick any traditional engine-error and any AI engine-error at random, and the traditional one is larger about 94 % of the time. Rank-based, hence robust to the same heavy tails that motivated Wilcoxon.

Split conformal prediction (distribution-free intervals). Fit any predictor on the training set; on a held-out calibration set of size n_c, collect the absolute residuals $r_i = |y_i - \hat{y}_i|$ and take

$$\hat{q} \;=\; \text{the } \lceil (1-\alpha)(n_c+1) \rceil\text{-th smallest } r_i; \qquad C(x) \;=\; [\hat{y}(x) - \hat{q},\; \hat{y}(x) + \hat{q}]. \tag{3.6}$$

Exchangeability alone guarantees $\Pr[y \in C(x)] \geq 1 - \alpha$ — no distributional assumption, any model, finite samples. The $(n_c + 1)$ (not n_c) accounts for the new point joining the ranking: the guarantee is exact, not asymptotic. The classification variant used in Chapter 11 (adaptive prediction sets) sorts the predicted class probabilities, accumulates until the calibration quantile of "mass needed to reach the true class" is covered, and returns that *set* of classes — ambiguity becomes set size instead of silent error.

Tree-structured Parzen estimator (the tuner). Optuna's TPE turns tuning into inference: after some trials, split configurations into the best fraction ($\ell(\theta)$, a density fitted over good configs) and the rest ($g(\theta)$), then propose the θ maximizing $\ell(\theta)/g(\theta)$ — sampling where good configurations concentrate, exploring where g is thin. Fifty trials of TPE typically cover a 14–16-dimensional mixed space far better than grid search's curse of dimensionality allows.

3.6 Methodological safeguards

The credibility of an AI-versus-traditional comparison depends on eliminating the biases that plague the literature. Four safeguards are adopted:

Pre-registration. Hypotheses (Table 1.1), metrics, data splits (with file hashes), statistical tests and the stopping rule are frozen in the repository under a git tag (`prereg-v1`, document `docs/prereg-v1.md`) *before* the confirmatory evaluation. This borrows a standard practice from medicine: you state what would convince you before looking at the answer, so you cannot move the goalposts afterwards. The frozen document also discloses every exploratory test-set exposure that preceded it.

Symmetric tuning budget. Every method has knobs. The traditional pipeline's (alert thresholds, regularization λ, Kalman $\mathbf{Q}$, smoothing windows) and each AI family's hyperparameters are tuned with the *same number* of automated search trials (Optuna), using only the training and validation engines.

Split by engine. The fleet is divided 70/10/20 engines into training, validation and test sets, and no engine ever appears in two sets. Splitting by flight instead of by engine —a common mistake— would let a model memorize an engine's individual quirks and score unrealistically well. An automated leakage check runs in continuous integration.

A strong baseline. The traditional pipeline is implemented with the same care as the AI, every expert rule documented with its physical justification; smearing is *measured* against ground truth, not assumed.

3.7 Reproducibility and infrastructure

The entire project is a public repository[1] with a reproducible environment: Python 3.11 managed with `uv` and a fully pinned dependency lockfile; pyCycle 4.4 on OpenMDAO [12]; declarative YAML configuration; and automated tests (pytest) executed by continuous integration —a service that re-runs the whole test suite on every change, so a regression cannot land silently. Data and experiments of later phases will be versioned with DVC and MLflow respectively.

Three concrete decisions illustrate the intended standard of rigor:

- **A versioned calibration contract.** The model's numerical targets (Table 2.1) live in a single file (`conf/cfm56_7b_targets.yaml`) carrying value, tolerance, source and status (`VERIFIED`/`TO_VERIFY`) per entry. Both the model runners and the tests read that file: changing a target is a visible version-control event, never a silent edit.
- **Dependency canary tests.** During environment setup, an incompatibility between pyCycle 4.4 and numpy ≥ 2 was discovered (a failure inside the tabular thermodynamics package). The fix pins `numpy<2`, and a smoke test that runs a complete engine cycle executes on every CI run, so any future dependency upgrade that breaks the solver fails loudly and immediately.
- **Auto-generated evidence.** Every result figure and table in this document is produced by a script (`scripts/make_report_assets.py`) that solves the model and writes the PDFs, the LaTeX table files (`paper/report/tables/*.tex`) and a provenance file (`assets.json`). No result number is ever copied by hand.
- **Compute-efficiency norms.** The repository carries binding engineering norms (`docs/engineering-norms.md`): long decomposable computations run in parallel across CPU cores (thermodynamic sweeps and ICM columns use one worker process per core — these are scalar Newton solves that gain nothing from a GPU), while the tensor workloads of phase F4 (network training, SHAP) run on the GPU (Apple-silicon MPS backend on the development machine).

3.8 Software architecture and computational pipeline

A reproducible study owes the reader not just its results but its machinery. This section is the complete inventory of the project's code — what each program does, how it does it, and what flows between them — so that any claim can be traced to the exact lines that produced it. It is updated whenever code lands.

[1] `https://github.com/cedarcreeks/EHMbrAIn`

3.8.1 Library modules (`src/ehmbrain/`)

`perf/cycle.py` — the engine model. Defines four model classes and their builders.

- `CFM56(pyc.Cycle)`: one thermodynamic point. Builds the component graph (inlet, fan, splitter, booster, HPC with three bleed ports, post-compressor bleed, combustor, cooled HPT/LPT, ducts, two convergent nozzles, two shafts, performance block) and its balance system. In *design* mode the four balances solve mass flow for thrust, fuel–air ratio for T_{t4}, and both turbine expansion ratios for zero net shaft power. In *off-design* mode they solve mass flow and bypass ratio to hold the design nozzle throat areas, both spool speeds for shaft power balance, and fuel–air ratio according to the throttle mode: `T4` (rating temperature), `percent_thrust` (a fraction of a reference thrust) or `N1` (hold fan speed, the CFM56 rating parameter, implemented through a passthrough component because the speed is itself another balance's output). Each point carries its own Newton solver (tolerance 10^{-8}, Armijo–Goldstein line search, subsystem solves enabled).
- `MPCFM56(pyc.MPCycle)`: DESIGN + the four SLS anchors A2–A5 as thrust-matched `percent_thrust` points. Used for calibration (Section 4.4).
- `MPCFM56Study`: DESIGN + one off-design point whose five turbomachine map scalars (efficiency and flow) pass through multiplier components (`health_<comp>.eta_fac/flow_fac`); setting a factor to $1 + \Delta$ imposes a health deviation. Used for the ICM and, in F2, for degraded-fleet snapshots.
- `MPCFM56Deck`: DESIGN + `OD_max` (maximum thrust at the rating T_4) + `OD_pt` (a commanded fraction of that maximum). Used for the baseline decks.
- Builders (`build_problem`, `build_study_problem`, `build_deck_problem`) return solver-ready problems with all design inputs and balance initial guesses set; `solve_anchors()` encapsulates the idle power continuation ($0.30 \to 0.07$); `snapshot()` reads the full sensor set at a point; `set_health()` imposes a health-deviation dictionary.

`perf/icm.py` — GPA sensitivity analysis. `compute_icm()` builds a Study problem per health parameter (in parallel worker processes, norm N1), perturbs it $\pm 0.5\,\%$ at constant N1 and forms the central-difference column in %-per-%; `svd_report()` returns rank, singular values and condition number; `signature_angles()` the pairwise angles between fault signatures; and `confusable_pairs()` the sub-15° subset that instantiates hypothesis H2.

`perf/surrogate.py` — differentiable neural twin (F8/L1). The residual-learning surrogate architecture and the `SurrogateEmitter` that runs it CPU-only inside the fleet workers, so the nonlinear v2 fleet (Section 12.3) generates at linear-generator cost.

`trad/identifiability.py` — the certificate (F10). Accumulated Fisher information over an operating-condition history → Cramér–Rao posterior, per-direction identifiability tags and the coverage ellipsoid (Chapter 13).

`common/corrections.py` — corrected parameters. ISA atmosphere (`isa_static`), ram-corrected θ/δ at the fan face (`theta_delta`), the exponent regression `fit_correction_exponents()` ($\log Y$ against a cubic polynomial in log corrected speed plus $a \log\theta + b \log\delta$, solved by least squares) and `correct()` to apply the fitted law.

3.8.2 Executable scripts (`scripts/`)

Each script is a pipeline stage writing versioned artifacts under `data/processed/`:

Script	Function and outputs
`run_design_point.py`	Solves the design point, checks the five WP1.1 acceptance windows against the contract; nonzero exit on failure.
`run_anchors.py`	Solves DESIGN + A2–A5 with the idle continuation and compares fuel-flow/OPR predictions to the EEDB (gated tolerances); prints the verdict table.
`make_decks.py`	Envelope sweep for the baseline decks: independent (altitude, Mach) blocks in parallel processes, warm continuation inside a block, cold-rebuild recovery (norm N2), then a per-channel parallel leave-one-out audit. → `baseline_deck.parquet`, `deck_report.json`.
`make_corrected_baseline.py`	Fits the correction exponents on the deck and re-runs the leave-one-out audit in corrected space. → `corrected_baseline.json`.
`make_icm.py`	ICM at the six-point operating grid (columns in parallel), SVD and confusability analysis. → `icm_<point>.npz`, `icm_report.json`.
`cea_crosscheck.py`	Re-solves the design point with CEA thermodynamics and reports deltas against tabular. → `cea_crosscheck.json`.
`make_report_assets.py`	Regenerates every figure and table of this document from the model and the artifacts above (norm N4). → `paper/report/figures/*.pdf`, `paper/report/tables/*.tex`, `assets.json`.

Later phases added, under the same contract-artifact-evidence pattern:

`datagen/` `trajectories.py` (per-mechanism health profiles, exact wash sawtooth, multi-episode acute faults), `fleet.py` (engine sampling, EGT-margin end of life, leakage-free splits), `sensors.py` (noise/quantization/drift/dropout), `snapshots.py` (ACARS-style two-report generator via the F1 linearization). Driver: `make_fleet.py`; audits: `audit_nonlinearity.py`, `audit_dataset.py`.

`trad/` `pipeline.py`: baseline deviations, Holt smoothing with innovations, cumulative-sum (CUSUM) and dual exponentially-weighted-moving-average (EWMA) gap detectors, WLS and wash-aware Kalman GPA, nearest-signature isolation, Theil–Sen RUL. Driver: `run_trad.py`.

`ai/` `data.py` (shared feature builder — same deviations as `trad/`), `models.py` (autoencoder, GRU RUL net, MPS-aware training loops). Drivers: `run_ai.py` (detection/diagnosis/RUL + conformal), `run_hybrid.py` (stacking ablation), `run_pcs.py` (Physics-Consistency Score), `tune_detection.py` (validation sweep).

The data flow is linear: `conf/*.yaml` (the contracts) feed the model and generator; runners and `make_*` scripts write `data/processed/*` artifacts; `make_report_assets.py` turns artifacts into report evidence; the tests gate every stage.

3.8.3 Test suite (`tests/`)

Thirty-seven tests, all run in CI on every change: environment smoke (the vendored pyCycle example end-to-end); design-point convergence and acceptance windows; anchor convergence, fuel-flow and OPR tolerances, and N1/N2 redlines; the Study self-consistency property; ICM physical signs and rank-3 cockpit underdetermination; the corrections module (ISA, θ/δ, exponent-fit recovery on synthetic truth); trajectory physics (wash sawtooth ratios and regrowth, FOD steps, reproducibility, split integrity); the sensor and snapshot models (noise statistics, quantization, drift ramps, dropout fraction, end-to-end consistency); traditional-pipeline primitives (Holt trend and innovations, CUSUM step-not-trend, gap-detector ramp capture, WLS recovery, Kalman observation-space convergence, Theil–Sen); and AI primitives (feature fill, window alignment, device selection, autoencoder separation, GRU countdown learning).

3.9 Replication guide

This document and the repository are designed to be used together: every result chapter names the script that produced its evidence, and this section gives the complete replication path. **Reference machine for every wall-clock figure in this document**: an Apple M5 desktop chip (10 cores, 16 GB unified memory), with tensor training on its GPU through the torch MPS backend — consumer hardware, deliberately. Total cost: about **11 minutes** end to end (Table 3.1; per-stage breakdown measured by `scripts/benchmark_pipeline.py`, norm N5).

Setup (once).

1. Clone `github.com/cedarcreeks/EHMbrAIn` and install `uv` (the Python package manager).
2. `uvsync` — reproduces the exact pinned environment (Python 3.11, pyCycle 4.4, torch with MPS).
3. `uvrunpytest` — 37 tests must pass before anything else; they gate every claim in this document.

Pipeline (in order). `makeall` runs everything below and rebuilds this document (gate H6); the stage-by-stage mapping to report evidence is:

Command (`uvrunpython\ldots`)	Produces	Feeds
`scripts/run_design_point.py`	design-point check	Tables 4.1 and 4.2
`scripts/run_anchors.py`	calibration verdicts	Table 4.3 and Fig. 4.2
`scripts/make_decks.py`	baseline decks + LOO	Table 4.5
`scripts/make_corrected_baseline.py`	corrected-space exponents	Table 4.4
`scripts/make_icm.py`	ICM grid + observability	Fig. 4.4 and Tables 4.6 and 4.7
`scripts/cea_crosscheck.py`	thermo cross-check	Table 4.8
`scripts/make_fleet.py`	SynCFM56 fleet	Table 6.1 and Figs. 6.1 and 6.2
`scripts/audit_dataset.py`	difficulty + realism audits	Table 6.3
`scripts/audit_nonlinearity.py`	linearization audit	Table 6.2
`scripts/run_trad.py`	traditional metrics	Table 7.1 and Figs. 7.1 and 7.2
`scripts/run_ai.py`	AI metrics + conformal	Table 8.2 and Fig. 8.1
`scripts/run_hybrid.py`	hybrid ablation	Fig. 8.2
`scripts/run_pcs.py`	PCS attribution study	Section 8.6
`scripts/tune_f5.py` + `scripts/f5_confirm.py`	tuned configs; H1–H5 verdicts	Table 9.1
`scripts/sim_to_real.py`	external-benchmark ranking check	Section 9.7
`scripts/benchmark_pipeline.py`	compute times (norm N5)	Table 3.1
`scripts/make_case_studies.py`	five case-study figures + facts	Chapter 10
`scripts/make_report_assets.py`	*every* figure and table above	this document

The extension phases (Chapters 11 to 14) add their own drivers, each writing a verdict artifact its figure or table reads:

Command (uvrunpython\ldots)	Produces	Feeds
scripts/f7_observability.py	realistic-scatter observability	Fig. 11.1
scripts/f7_learner.py	learned multi-point fusion verdicts (F7)	Table 11.2
scripts/f8_surrogate_data.py + scripts/f8_surrogate.py	neural twin + gate (L1)	Fig. 12.1
scripts/make_fleet.pysurrogate + scripts/audit_v2_fidelity.py	fleet v2 + fidelity (L2)	Fig. 12.2
scripts/f8_l6_hybrid.py	twin-residual H4 (L6)	Fig. 12.3
scripts/f8_l5_arch.py	architecture breadth (L5)	Fig. 12.4
scripts/f8_l4_recoverable.py	recoverable fraction (L4)	Fig. 12.5
scripts/f8_l9_pcs.py	PCS validation (L9)	Table 12.1
scripts/f8_l7_drift.py	augmented-Kalman drift (L7)	Section 12.8
scripts/f8_lh2_wall.py	real-vs-virtual sensors (L-H2)	Fig. 12.6
scripts/f8_lrul_advanced.py	advanced prognostic (L-RUL)	Fig. 12.7
scripts/f10_certificate.py	identifiability certificate	Table 13.1 and Fig. 13.1
scripts/econ_impact.py	economic Monte Carlo	Table 14.1 and Fig. 14.2
scripts/fig_rul_distribution.py, fig_isolation.py, dump_optuna_history.py	result-distribution data	Figs. 9.1, 9.3 and 9.4

Two platform notes a replicator needs (both learned the hard way, both in the decision register): torch-MPS training runs must execute in the *foreground* on macOS (backgrounded runs segfault), and XGBoost cannot coexist with torch-MPS in one process (scikit-learn's histogram gradient boosting is used instead).

3.9.1 Compute cost on the reference machine

Table 3.1: Wall-clock time per stage on the reference desktop machine (Apple M5, 10 cores, 16 GB, torch MPS backend). Cold-start subprocess timings — what a replicator actually pays. Auto-generated per norm N5.

Stage	Script	Device	Wall time [s]
design point	scripts/run_design_point.py	CPU	2
sls anchors	scripts/run_anchors.py	CPU	13
baseline decks	scripts/make_decks.py	CPU	214
corrected baseline	scripts/make_corrected_baseline.py	CPU	1
icm grid	scripts/make_icm.py	CPU	71
fleet generation	scripts/make_fleet.py	CPU	40
audit dataset	scripts/audit_dataset.py	CPU	12
audit nonlinearity	scripts/audit_nonlinearity.py	CPU	20
traditional pipeline	scripts/run_trad.py	CPU	106
ai suite	scripts/run_ai.py	CPU+GPU	27
hybrid ablation	scripts/run_hybrid.py	CPU+GPU	104
pcs	scripts/run_pcs.py	CPU+GPU	46
surrogate data cruise	scripts/f8_surrogate_data.py	CPU	466
surrogate train	scripts/f8_surrogate.py	CPU+GPU	40
fleet v2	scripts/make_fleet.py	CPU	72
h4 v2 hybrid	scripts/f8_l6_hybrid.py	CPU+GPU	82
l4 recoverable	scripts/f8_l4_recoverable.py	CPU	4
l5 architectures	scripts/f8_l5_arch.py	CPU+GPU	52
l7 drift	scripts/f8_l7_drift.py	CPU	8
l9 pcs	scripts/f8_l9_pcs.py	CPU+GPU	96
identifiability certificate	scripts/f10_certificate.py	CPU	8
Full pipeline			**1484**

The entire evidence base of this document regenerates in 657 seconds — about eleven minutes — on the Apple M5 reference machine, GPU stages included. This is the quantitative form of transferability rule 4: if it only ran on a cluster, it would not be adoptable.

3.10 Comparison metrics

Both families are scored with identical metrics. For detection: lead time (how many flights before the ground-truth event the method raises a sustained alert), false alarms per thousand flights, F1 and precision-recall area-under-the-curve (AUC). For isolation: top-1/top-2 accuracy, macro-F1, a *smearing index* (the fraction of $\|\hat{\mathbf{x}}\|_1$ assigned to healthy components) and the PCS. For health estimation: RMSE of $\hat{\mathbf{x}}$ against ground truth. For RUL: RMSE, MAE, the NASA PHM08 score

$$S = \sum_i s_i, \qquad s_i = \begin{cases} e^{-d_i/13} - 1 & d_i < 0 \\ e^{d_i/10} - 1 & d_i \geq 0 \end{cases}, \qquad d_i = \widehat{\mathrm{RUL}}_i - \mathrm{RUL}_i, \tag{3.7}$$

which penalizes late predictions more than early ones (a late removal risks an in-flight shutdown; an early one merely wastes some life). Two further prognostic metrics need definitions: α–λ *accuracy* is the fraction of predictions falling within $\pm\alpha$ (here 20 %) of the true RUL when evaluated at specified life fractions λ (here 50, 70 and 90 % of the way to failure) — it asks "was the prediction acceptable at mid-life? near the end?"; the *prognostic horizon* is the earliest moment from which the prediction stays within the $\pm\alpha$ band for the rest of the engine's life — longer horizons mean earlier trustworthy warnings. For uncertainty: empirical versus nominal interval coverage (does the "90 %" interval actually contain the truth 90 % of the time?) and mean interval width (narrow honest intervals are the goal; wide intervals are trivially calibrated but useless).

Chapter 4

Thermodynamic twin of the CFM56-7B26

This chapter in brief: a virtual CFM56-7B26 is built in pyCycle by adapting a validated NASA example, calibrated against certified data (its fuel-flow predictions land within 3 % of measured test-bed values), and then interrogated for the two artifacts everything else consumes: the healthy-engine baseline decks and the influence coefficient matrix, whose geometry quantifies exactly how blind the cockpit sensor set is.

Key idea. The twin is built only from certified public data (type-certificate limits and measured emissions-databank fuel flows), so its fuel-flow predictions land within 3 % of test-bed values it never saw — a recipe any operator could replicate for any engine.

This chapter documents the completed part of phase F1: the CFM56-7B26 performance model in pyCycle [5], its design point (work package WP1.1) and its off-design calibration against certified measured data (WP1.2). All result figures and tables are produced by the evidence-generation script (Section 3.7).

4.1 What a cycle model is

A 0-D *cycle model* represents the engine as a chain of thermodynamic components —inlet, fan, compressors, combustor, turbines, nozzles— each transforming the air's pressure, temperature and composition according to its governing equations, without resolving any internal geometry. Each turbomachine carries a *component map*: a table, obtained from rig tests or scaled from a similar machine, giving its efficiency and flow as a function of rotational speed and operating point. The model runs in two modes:

Design mode sizes the engine: given targets (thrust, component pressure ratios, turbine inlet temperature), it computes the flow areas, map scale factors and shaft-work balance that a machine meeting those targets must have.

Off-design mode answers the service question: given the now-fixed geometry, what does the engine do at any other flight condition and thrust setting? This is the mode the EHM baseline decks and the influence coefficient matrix are built from.

pyCycle solves both modes with a Newton solver: it iterates on a set of unknowns (mass flow, fuel–air ratio, spool speeds, map operating points) until a set of residual equations (nozzle-area match, shaft power balance, thrust target) is driven to zero. Newton solvers are powerful but fragile far from a good initial guess —a fragility that shaped several decisions below.

4.2 Strategy: adapt, don't build from scratch

pyCycle ships a validated example of a two-spool, separate-flow, cooled high-bypass turbofan (`high_bypass_turbofan.py`) with the full component graph, the turbine-cooling bleed network, shaft balances and off-design solver wiring already working. Building the CFM56 from a blank file would re-derive all of that plumbing and typically dies in Newton divergence. Instead, a *morphing* protocol is adopted: the example is vendored into the repository as a known-good regression reference, and the model is steered toward the CFM56-7B26 by changing **at most three parameters per step**, requiring convergence at every step before the next. Every converged step is a commit; if a step diverges, the change is bisected.

All design decisions were fixed in writing *before* coding (`docs/f1-model-spec.md`): the -7B26 variant; tabular thermodynamics for speed with a CEA cross-check planned (CEA is NASA's chemical-equilibrium code —more accurate, roughly an order of magnitude slower); the design point at cruise, Mach 0.78 / 35 000 ft / ISA (the "aerodynamic design point" convention: an engine is designed for cruise, where it spends most of its life, and takeoff is an off-design condition); pyCycle's generic component maps, scaled —no public CFM56 maps exist, a declared limitation; thrust set through corrected N1 in off-design (the -7B is an N1-rated engine); pyCycle-native units inside the model with SI only at the boundary; and a table of anticipated solver failure modes, each with a planned countermeasure —one of which proved prophetic (Section 4.4.3).

4.3 Design point (WP1.1)

The cycle graph comprises: flight conditions, inlet, fan, flow splitter, booster, HPC with three bleed ports (HPT vane and blade cooling, customer bleed), a post-compressor bleed feeding the LPT, combustor, cooled HPT and LPT, ducts with pressure losses, convergent core and bypass nozzles, the two shafts, and an accessory power extraction of 250 hp on the HP shaft (driving the aircraft's generators and pumps). In design mode, four implicit balances close the cycle: the inlet mass flow W meets the thrust target; the fuel–air ratio (FAR) meets the target turbine inlet temperature T_{t4}; and the HPT and LPT expansion ratios zero the net power on each shaft (each turbine produces exactly what its compressors and losses consume).

Table 4.1: Design-point results (Mach 0.78, 35 000 ft, ISA). Auto-generated from the model.

Quantity	Value at the design point
Net thrust F_n	24.38 kN (5480 lbf)
TSFC	0.6214 lb/(lbf·h)
Bypass ratio (BPR)	5.10
Overall pressure ratio (OPR)	30.03
T_{t4}	1587 K (2857 °R)
Inlet mass flow W	142.0 kg s^{-1}
Fuel–air ratio (FAR)	0.0251
HPT expansion ratio	3.586
LPT expansion ratio	4.288
Total cooling/bleed fraction	0.239

The design point converges with a Newton residual below 10^{-6} and satisfies all five acceptance windows defined a priori: BPR 5.10 ± 0.2 (the EEDB target), OPR within its window, T_{t4} within a plausible band, total cooling-plus-bleed fraction between 18 and 25 % of core flow, and thrust exactly on target (Table 4.1). The resulting TSFC (≈ 0.62 lb/(lbf·h)) lands within 2 % of the publicly quoted cruise value for this engine *without having been used as a calibration target* —an encouraging consistency check.

Table 4.2: Total conditions at each gas-path station at the design point. Auto-generated.

Station	Location	P_t [bar]	T_t [K]
2	Fan inlet	0.358	245.5
21	Fan exit	0.603	289.5
25	HPC inlet	1.149	354.2
3	HPC exit	10.739	703.9
4	Burner exit	10.159	1587.2
45	HPT exit	2.833	1141.6
5	LPT exit	0.657	806.3

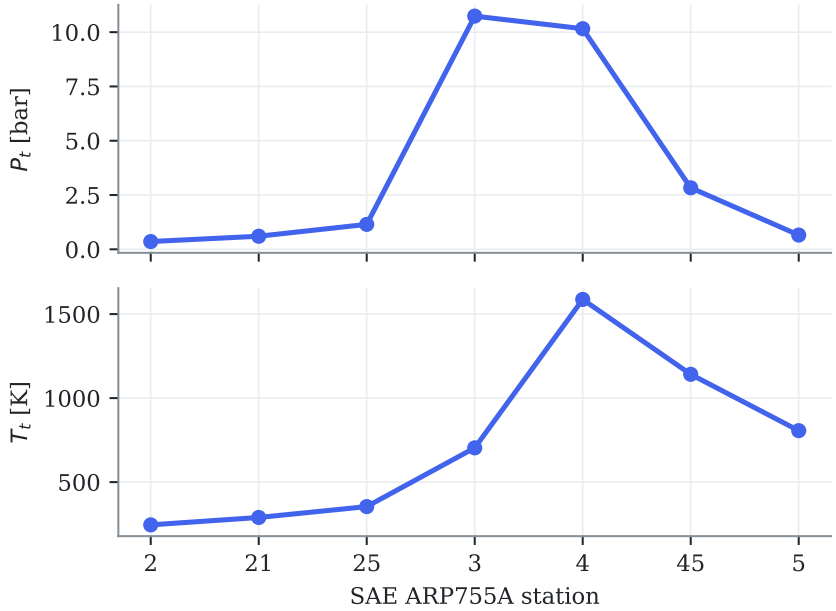

Figure 4.1: Total pressure and temperature along the gas path at the design point. Compression peaks at station 3 (∼11 bar, Table 4.2), heat addition at station 4, and work extraction across stations 45 and 5. Auto-generated.

4.4 Calibration against measured data (WP1.2)

A model that converges is not yet a model that is *right*. Calibration is the step that ties the twin to reality: adjusting its few free scalars until it reproduces measured certification data it was not built to match. The philosophy first, then the two iterations it took.

4.4.1 Data sources and philosophy

Off-design calibration uses exclusively **measured, certified data** (Table 2.1). The TCDS provides limits and rated thrusts. The ICAO Engine Emissions Databank provides something less well known: sea-level test-bed *measured fuel flows* at the four landing-takeoff-cycle thrust settings (100, 85, 30 and 7 % of rated thrust F_{00}), plus the engine pressure ratio at rated output (27.61) —all for the exact -7B26 variant, traceable to certification testing, and free. Most academic engine models ignore this source (Section 5.4 characterizes both sources as the provenance chain's root).

The anchor points are solved *thrust-matched*: the fuel–air ratio balance is set to hold $F_n = \mathrm{PC} \cdot F_{00}$ (PC being the power fraction). The model's fuel flow at each anchor is therefore a genuine **prediction** confronted with the EEDB measurement —not a quantity fitted to it.

4.4.2 Calibration iterations

The first run (model freshly adapted from the example, not yet calibrated to the SLS data) already predicted takeoff fuel flow within −2.1 %, but showed two quantitative defects:

1. **N2 above its redline at rated thrust**: 15 311 rpm against the certified limit of 15 183 rpm. Cause: the reference speed used to scale the generic HPC map —the map's speed-versus-flow shape is not the real compressor's, so the speed labels it implies must be anchored deliberately. Fix: HP-shaft design reference from 14 324 to 13 940 rpm, placing rated-thrust N2 at 14 915 rpm (≈ 103 %), inside the limit.

2. **Takeoff pressure ratio 28.99 versus the measured 27.61** (+5.0 %). Fix: HPC design pressure ratio from 9.81 to 9.35. The measured value takes precedence over the initial derived estimate, and the cruise OPR becomes a consequence (≈ 30).

Table 4.3: SLS anchors versus the ICAO databank measurements after calibration. W_f model is the model's prediction at imposed thrust. Auto-generated.

Anchor	$\%F_{00}$	F_n [kN]	W_f model	W_f EEDB	error [%]	tol. [%]	gated
A2 Takeoff	100	117.0	1.204	1.221	-1.40	3.0	✓
A3 Climbout	85	99.4	0.977	0.999	-2.23	3.0	✓
A4 Approach	30	35.1	0.328	0.338	-3.08	5.0	✓
A5 Idle	7	8.2	0.125	0.113	+10.57	5.0	—

After both corrections (Table 4.3 and Figs. 4.2 and 4.3), the three gated anchors meet their tolerances: takeoff at −1.40 % fuel-flow error with the pressure ratio within +0.3 % of the measured value and both shaft speeds inside their redlines; climb-out at −2.23 %; approach at −3.08 %.

4.4.3 Idle convergence: diagnosis and cure

The idle point (7 % of F_{00}, static) systematically diverges from a cold start. At such low power the nozzle pressure ratios approach unity —the exhaust barely pushes— and the nozzles' internal static-pressure sub-solver bounces against its lower bound, leaving the global Newton iteration without a descent direction. Worse, the failed attempt leaves the point's internal states (flow

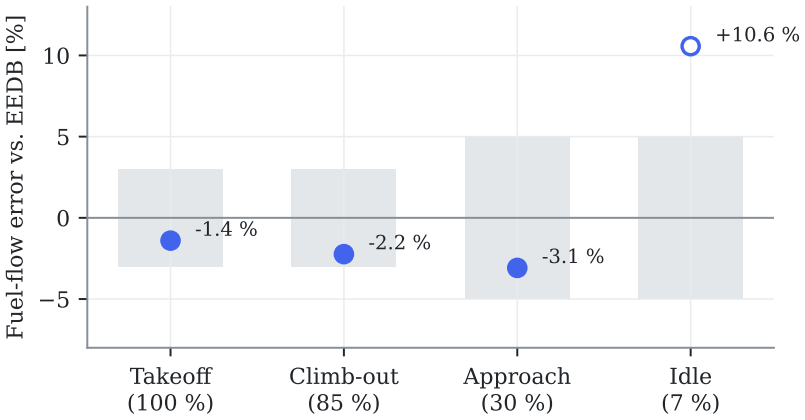

Figure 4.2: Fuel-flow prediction error at each anchor against the EEDB measurement. Gray bands are the contract tolerances (±3 % at takeoff and climb-out, ±5 % at approach and idle); the hollow marker (idle) converges but sits outside tolerance and is reported without blocking the milestone (Section 4.4.3). Auto-generated.

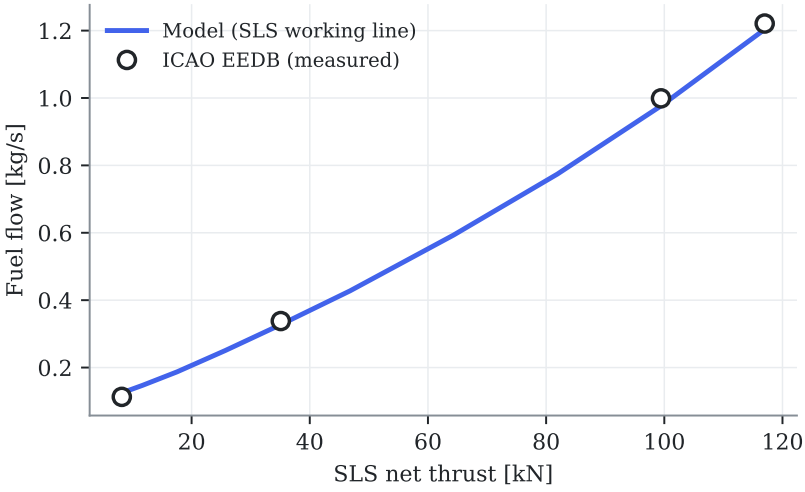

Figure 4.3: The model's sea-level-static working line (a continuous sweep of thrust setting) against the four measured ICAO databank points. The curvature at low power reflects the degraded accuracy of generic maps far from their scaling point. Auto-generated.

stations, map positions) corrupted, so re-seeding only the balance variables does not rescue it: the idle point must *never* start cold at its final setting. The cure —anticipated as a generic countermeasure in the specification— is **power continuation**: the point starts at 30 % of F_{00} (where it converges from cold, just like the approach anchor) and walks down warm through $0.30 \to 0.20 \to 0.14 \to 0.10 \to 0.08 \to 0.07$, re-solving at each step. The logic is encapsulated in the library (`solve_anchors()`) and covered by the tests.

With continuation, idle converges and predicts fuel flow with +10.6 % error. This is deep map-extrapolation territory —the generic map's speed-flow shape at very low power is not the real compressor's— a known limitation of the approach; the anchor is **reported but not gated** for milestone H1, a decision recorded in the calibration contract (`gated:~false`).

4.5 Baseline decks (WP1.3)

Both EHM pipelines need a *baseline*: what a healthy engine reads at any operating condition, so that measured values can be turned into deviations. The baseline is produced by sweeping the healthy model over a realistic flight envelope —altitude blocks with their plausible Mach ranges (no Mach 0.8 at sea level), ambient temperature offsets of 0, +15 and +30 °C above ISA, and six thrust settings from 50 to 100 % of the locally available maximum. The 50 % floor is deliberate: airline EHM trending consumes takeoff, climb and stable-cruise reports, all high-power conditions; approach and idle are neither trending conditions nor reliable model territory (Section 4.4.3). At each condition the model first finds the maximum thrust available at the rating temperature (point `OD_max`, throttled by T_4), then solves the part-power point at the commanded fraction (`OD_pt`) —the same two-point pattern used by the pyCycle reference example. The rating temperature is fixed at $T_{4,\max} = 3200\,°\mathrm{R}$, a rounded envelope constant chosen from the calibrated takeoff anchor (imposing the certified rated thrust at SLS required $T_4 \approx 3175\,°\mathrm{R}$, Table 4.3); a real FADEC modulates its rating limits across the envelope, so this constant is a declared simplification — adequate here because the baseline uses *fractions* of the local maximum, not its absolute value.

Two engineering lessons from WP1.2 shaped the sweep. First, the walk is ordered for warm starting within each block (thrust descending), and the independent (altitude, Mach) blocks run in *parallel processes*, one Problem per worker (project norm N1: long decomposable computations are parallelized; on the Apple M5 reference machine the sweep takes 3.5 minutes on all cores versus 10 minutes serial). Second —the lesson learned the hard way— a failed Newton attempt corrupts the point's internal states beyond re-seeding, so the recovery action on any failure is to *rebuild* the problem cold with altitude-appropriate guesses (norm N2); a first version of the sweep without this recovery lost every point downstream of the first divergence.

The result is stored as a table plus a scattered linear interpolator (Delaunay triangulation with barycentric interpolation, SciPy's `LinearNDInterpolator`) in the four operating coordinates, and its quality is measured by leave-one-out cross-validation: each grid point is removed, predicted from all the others, and the relative error recorded — Table 4.5 reports the per-channel median, 95th percentile and maximum. Median errors are a few tenths of a percent; the tail percentiles are dominated by envelope-corner points whose interpolation simplices span across altitude blocks in raw physical coordinates.

4.5.1 Corrected-parameter exponents and the corrected-space baseline

The fix for those tails is the one anticipated in Section 2.3: build the baseline in *corrected-parameter* space. The correction exponents are fitted on the deck itself rather than copied from generic tables: per channel Y, a linear least-squares fit of

$$\log Y \;=\; \underbrace{\textstyle\sum_{k=0}^{3} c_k \left(\log N1_c\right)^k}_{\text{power line}} \;+\; a\,\log\theta \;+\; b\,\log\delta, \tag{4.1}$$

where the cubic polynomial absorbs the dependence on the corrected speed (the "power line") and the exponents (a, b) capture whatever ambient dependence remains — exactly the quantities the correction $Y_{\text{corr}} = Y/(\theta^a \delta^b)$ then removes. The fitted exponents land squarely on their textbook values without having been told them (Table 4.4): fuel flow $a = 0.63$, $b = 1.02$ (handbooks quote $\theta^{0.5\text{–}0.7}\delta$); N2 $a = 0.49$, $b \approx 0$ (the $\sqrt{\theta}$ corrected-speed law); EGT $a = 0.95$, $b \approx 0$ (temperature ratio). This is a further independent consistency check of the model physics.

Re-running the leave-one-out audit on the corrected channels over (corrected N1, Mach) collapses the tails by an order of magnitude (Table 4.4): worst-case N2 error drops from 9.7 % to 0.5 %, EGT from 24 % to 2.0 %, fuel flow from 91 % to 6.6 %. Two remarks for honesty: net thrust is excluded —it is not an EHM sensor channel (it comes from the thrust-setting definition), and it does not collapse under a θ/δ law at fixed N1c because of its strong direct Mach dependence; and the residual corrected-space errors are comparable to the sensor noise floor that the fleet generator injects ($\sigma_{\text{EGT}} \approx 0.3$ %, $\sigma_{W_f} \approx 0.5$ %). Baseline error is also common-mode: both EHM pipelines consume the same baseline, so it does not bias the comparison.

Table 4.4: Fitted correction exponents per channel and the leave-one-out error before (raw 4-D coordinates) versus after (corrected space). Auto-generated.

Channel	a (θ)	b (δ)	raw LOO P95/max [%]	corrected LOO P95/max [%]
Fuel flow	0.628	1.022	15.77 / 90.6	3.08 / 6.6
N2	0.495	0.007	2.65 / 9.7	0.34 / 0.5
EGT	0.949	0.002	6.01 / 24.1	1.12 / 2.0

Table 4.5: Leave-one-out interpolation error of the baseline deck, per channel. Auto-generated.

Channel	Median [%]	P95 [%]	Max [%]
Fuel flow	0.498	15.767	90.603
N1	0.268	6.369	25.721
N2	0.080	2.649	9.697
EGT	0.167	6.012	24.062
Net thrust	0.000	16.437	85.402

4.6 Influence coefficient matrix and observability (WP1.4)

With a calibrated healthy engine in hand, the last F1 artifact is the map from *faults* to *symptoms* — the influence coefficient matrix that every diagnosis in this document inverts. Building it means deliberately injecting each fault and recording the response.

4.6.1 Health-parameter injection

To compute the ICM —and, in phase F2, to simulate degraded engines— the model must accept imposed health deviations. pyCycle scales its generic component maps with per-component design scalars (efficiency and flow); the off-design points normally inherit them from the design point. In the `MPCFM56Study` model those connections are routed through multiplier components, so that any efficiency or flow-capacity deviation can be dialed in per run. Two implementation subtleties are worth recording: (i) the multipliers must execute *before* the off-design point —the multi-point group runs its children in insertion order, and a mis-ordered version silently solved the off-design point with unscaled maps; the symptom (a "healthy" point that did not reproduce the design point) was caught by the self-consistency check below; and (ii) holding N1 constant, the CFM56's rating parameter, requires a fuel-flow balance targeting the LP spool speed through a passthrough component, since the speed is itself the output of another balance.

The wiring is validated by a self-consistency property that is now a permanent test: the *healthy* off-design point at cruise with N1 equal to its design value must reproduce the design point exactly. It does: thrust 5480.0 lbf, N2 13 940.0 rpm, OPR 30.03, EGT 1141.6 K —machine-precision agreement with Table 4.1.

4.6.2 The ICM at the reference conditions

The ICM is computed by central finite differences (±0.5 % per health parameter) at the two conditions where airline EHM snapshots are taken: cruise (M 0.78/35 000 ft) and sea-level takeoff, at constant N1. Figure 4.4 shows the cruise matrix for the extended sensor set; every sign satisfies the physical expectations (e.g. a healthier HPT runs cooler and thriftier: negative EGT and W_f entries in its efficiency column), and these sign checks run automatically in the test suite.

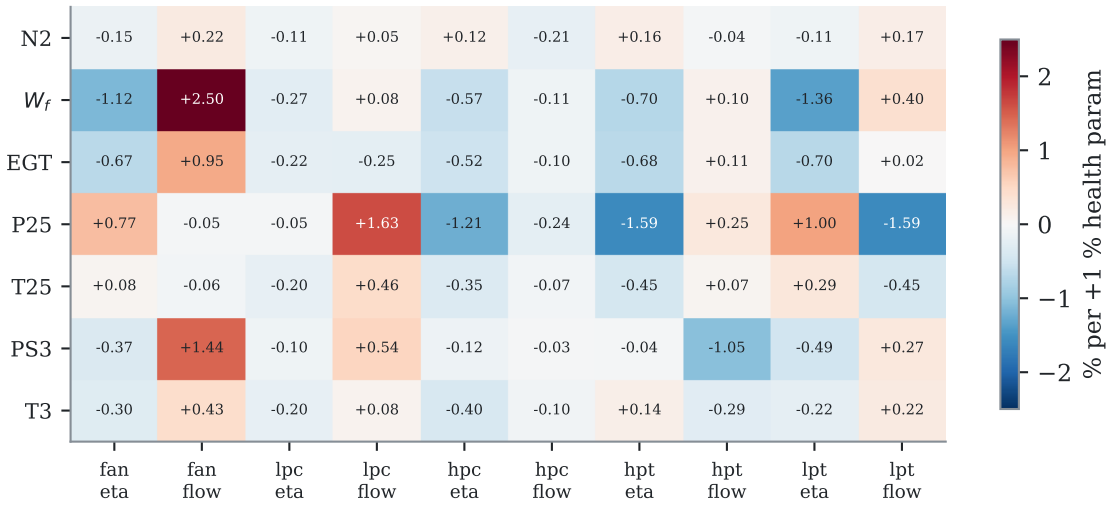

Figure 4.4: Influence coefficient matrix at cruise (extended sensor set): % change of each measured channel per +1 % change of each health parameter. Auto-generated from the model.

4.6.3 Observability: the quantitative case for hypothesis H2

With the cockpit set only (N2, W_f, EGT —N1 is the held rating parameter, so it carries no information), the ICM has rank 3 for 10 unknowns and the geometry of its columns is damning: the minimum angle between fault signatures is **1.3 degrees**, with seven pairs below 15 degrees (Table 4.7). The most confusable pair, HPC efficiency versus HPT efficiency, is precisely the classical HPC/HPT ambiguity reported in the GPA literature [9] —here quantified for this engine. Two faults whose signatures differ by 1.3 degrees are, at realistic noise levels, indistinguishable to any linear estimator: this is the smearing mechanism measured, and it defines the "confusable" subset on which hypothesis H2 will compare the AI's isolation ability against WLS-GPA. The extended sensor set (adding P25/T25/PS3/T3) raises the rank to 7 and multiplies the minimum angle by roughly five —the quantitative basis for the sensor-set ablation of contribution C6.

The analysis is repeated over a six-point grid spanning the envelope —both snapshot conditions plus power, altitude, climb and hot-day variations (Table 4.6). The observability structure barely moves: the minimum cockpit signature angle stays between 1.2 and 1.4 degrees and the confusable-pair count is constant at seven. The smearing problem is therefore *intrinsic to the cockpit sensor set*, not an artifact of one flight condition —which both justifies interpolating the ICM over this grid in phase F3 and forecloses the easy escape of "diagnose somewhere else in the envelope".

Table 4.6: Observability of the cockpit ICM across the operating-point grid. Auto-generated.

Operating point	cond.	$\alpha_{\min}$ ckpt. [deg]	pairs	$\alpha_{\min}$ ext. [deg]
Cruise M0.78/FL350	12.2	1.32	7	8.62
Cruise, low power	13.0	1.20	7	6.68
Cruise FL390	11.3	1.37	7	8.60
Climb 10 kft/M0.4	12.6	1.35	7	7.38
Takeoff SLS	10.5	1.33	7	7.92
Takeoff SLS, ISA+30	11.4	1.25	7	8.97

Table 4.7: Confusable fault pairs at cruise with cockpit sensors (signature angle $< 15°$). Auto-generated.

Fault A	Fault B	Angle [deg]
η_{HPC}	η_{HPT}	1.32
η_{FAN}	η_{LPT}	4.32
η_{HPT}	Γ_{HPT}	6.02
Γ_{FAN}	η_{LPT}	6.52
η_{HPC}	Γ_{HPT}	7.20
η_{FAN}	Γ_{FAN}	10.05
η_{FAN}	η_{LPC}	13.53

4.6.4 Thermodynamics cross-check

Decision D2 (tabular thermodynamics for speed) is closed by re-solving the design point with NASA's chemical-equilibrium code (CEA): all key quantities agree within 0.7 % (Table 4.8), an order of magnitude below the ±3 % calibration tolerance. Tabular thermodynamics is therefore retained for all fleet-scale computation.

Table 4.8: Design point solved with tabular versus CEA thermodynamics. Auto-generated.

Quantity	Tabular	CEA	Δ [%]
Net thrust [lbf]	5480.0000	5480.0000	-0.000
TSFC [lb/(lbf·h)]	0.6214	0.6249	+0.553
Inlet mass flow [lbm/s]	313.0851	315.2029	+0.676
Fuel–air ratio	0.0251	0.0250	-0.122
HPT expansion ratio	3.5865	3.5902	+0.104
LPT expansion ratio	4.2879	4.3109	+0.537

4.7 Declared limitations

The model is *representative*, not certifiable, and its limits are declared for citation in the results chapters: (i) no transients —steady-state points only; (ii) variable geometry (bleed valves, variable stators) implicit in the maps, not modeled; (iii) no Reynolds or humidity corrections; (iv) absolute N1/N2 labels approximate by construction (generic map shapes), though respecting the certified redlines at the rated point; (v) the model's EGT is the HPT-exit temperature (station 45), a consistent proxy —identical for ground truth and for both EHM pipelines— of the certified T49.5, whose displayed value additionally carries the shunt and trim functions described in Section 2.2; and (vi) the idle error quoted above. None of these limitations affects the validity of the AI-versus-traditional comparison, which rests on the model's sensitivity structure (the ICM), not on its absolute values.

4.8 Automated verification

The chapter's claims are pinned by tests: design-point convergence and acceptance windows; convergence of all four anchors including the idle continuation; fuel-flow and pressure-ratio tolerances at the gated anchors; N1/N2 redlines at rated thrust; the Study self-consistency property (healthy off-design = design); the ICM physical sign checks and its rank-3 cockpit underdetermination; and the pyCycle environment smoke test. Twelve tests green in continuous integration at the time of writing.

Chapter 5

Data: origin, meaning, and the synthetic-data method

This chapter in brief: before any method is judged, the reader deserves to know exactly what the data is *— what an airline actually measures, what each recorded number physically represents, what "synthetic data" means as a scientific instrument, and the complete provenance chain from certified test-bed measurements to the rows the algorithms consume. Nothing downstream makes sense without this chapter; nothing in it requires more than Chapter 2.*

Key idea. Real fleets offer no ground truth and almost no labels, so attribution — did a method blame the right component? — is unmeasurable on them. Synthetic-with-truth is not a compromise but the only instrument that can measure it; self-consistency (exact scoring) and physical fidelity (an audited question) are kept strictly separate.

5.1 What engine-health data exists in the real world

An airline's engine-monitoring data ecosystem has three tiers, and knowing them explains every design choice in SynCFM56:

ACARS snapshot reports (the tier this project mimics). The Aircraft Communications Addressing and Reporting System transmits short, structured telegrams from aircraft to ground over VHF or satellite. For engine monitoring, the avionics capture a *stabilized frame* — a few seconds of averaged sensor values once per flight phase — typically one report at takeoff and one in cruise. Each report carries on the order of 15–30 parameters per engine. This is the data tier on which classical EHM (and most airline practice) actually runs: **two rows per engine per flight**, not continuous signals. Its economics shaped it: ACARS transmission historically cost money per kilobyte, and a stabilized snapshot compresses a flight into exactly what trend monitoring needs.

Continuous recordings (QAR/CEOD). The Quick Access Recorder and its modern full-flight equivalents record hundreds of parameters at 1–16 Hz for the whole flight. Rich, but heavy: gigabytes per aircraft-day, often not downloaded every flight, historically processed only after events. Modern "big data" EHM programs increasingly exploit this tier; most published fleet experience still lives in tier one.

Shop and maintenance records. Borescope findings, module performance at overhaul, parts scrapped, wash logs. This is where *ground truth* about component condition is glimpsed — but only at maintenance events, only qualitatively, and recorded in systems never designed for machine learning.

Two structural facts follow. First, **real fleets provide almost no usable ground truth**: between shop visits (years apart), nobody knows the true efficiency of an installed HPT to a tenth of a percent — there is no oracle to score a diagnosis against. Second, **failures are rare and unlabeled**: a dataset of ten thousand real flights may contain zero acute gas-path faults, and when one occurs, its onset time and root cause are reconstructed after the fact, imperfectly. These two facts — no truth, no labels — are why this project generates its data, and they define exactly what the generation must provide.

5.2 What each measured channel physically represents

The eight channels recorded in every SynCFM56 snapshot are the industry-standard gas-path set. What each number *is*, physically, and why monitoring cares:

N1 [rpm] — fan / low-pressure spool speed. The big front rotor. On the CFM56-7B it is also the *thrust-setting parameter*: the crew (or FADEC) commands a target N1, so N1 is an input, not a symptom — which is why its deviation is zero by construction in our generator and why GPA treats it as the operating-point anchor.

N2 [rpm] — core / high-pressure spool speed. The compressor–turbine core. At fixed N1 and ambient, a shifting N2 re-balances the work split between spools — a sensitive, low-noise indicator of where in the core something changed.

WF [kg/s] — fuel flow. The engine's operating cost, measured directly. At fixed thrust, deteriorating components demand more fuel; a 1 % fleet-wide WF shift is a fuel-bill line item, which is why airlines track it to a tenth of a percent.

EGT [K] — exhaust gas temperature. Measured by a ring of thermocouples behind the low-pressure turbine. It is the engine's *thermal headroom gauge*: certification fixes a redline, a new engine at rated thrust on a hot day sits some tens of kelvin below it (the *EGT margin*), and every bit of deterioration spends that margin. When margin reaches zero the engine must come off wing — EGT margin *is* the industry's remaining-life currency, which is why RUL in this project is defined through it.

P25, T25 [bar, K] — booster exit / HPC inlet. The interface between the low and high spools; movements here separate "front of the engine" from "core" hypotheses.

PS3, T3 [bar, K] — HPC exit. The highest-pressure station, feeding the combustor. PS3 is the classic discriminator between compressor efficiency loss and flow-capacity loss — the pair the cockpit set famously confuses (Section 4.6).

The *cockpit subset* (N1, N2, WF, EGT) is what every operator has without optional instrumentation packages; the extended set adds the four station probes. The observability gulf between those two sets is quantified in Section 4.6 and becomes an acquisition-decision tool in Chapter 11.

5.3 What synthetic data is — and when it is science

Synthetic data is data produced by a model rather than measured from the target system [13, 14]. Four families are in common use: physics-based simulation (a mechanistic model generates observations — this project, and NASA's C-MAPSS/N-CMAPSS [3, 4]); generative statistical models (GANs, VAEs, diffusion models fitted to real data); augmentation (perturbing real samples); and hybrid pipelines. The motivations are always some subset of: **ground truth** (impossible to

obtain otherwise — our case), **rarity** (failures too scarce to learn from), **privacy/access** (airline data is commercially sensitive), and **control** (factorial experiments no fleet would ever fly).

Synthetic data earns scientific standing only by confronting its central liability: *the model that generates the data embeds assumptions, and any method evaluated on it might exploit those assumptions rather than solve the real problem.* This project's defenses are layered and were built *before* the headline experiments:

1. **Anchor the generator to certified measurements.** The twin behind the generator is calibrated against type-certificate and emissions-databank measurements (Section 4.4) — its fuel-flow *predictions* land within 3 % of test-bed values it was never fitted to.

2. **Separate self-consistency from fidelity.** Estimator scoring against ground truth is exact by construction; whether the generated physics matches the *full nonlinear* model is an audited empirical question (Section 6.4.1), not an assumption.

3. **Make the dataset hard on purpose.** The difficulty gate (a deliberately weak classifier must *fail*) exists because a synthetic dataset's most common disease is being accidentally easy; ours failed that gate twice and was physically redesigned both times (Section 6.4.2).

4. **Check the ranking externally.** The central prognosis conclusion was re-tested on a benchmark this project did not generate (Section 9.7).

5. **Say what does not transfer.** Absolute performance numbers (an RMSE of 858 cycles) are properties of this simulated world and its noise settings; what transfers is *rankings, mechanisms and geometry* — which method wins where and why, and the observability structure that any CFM56-class cockpit set shares. Every operational claim in this document is of the second kind.

5.4 The provenance chain, end to end

Every number in SynCFM56 descends from public, certified measurements through an auditable chain (Fig. 5.1):

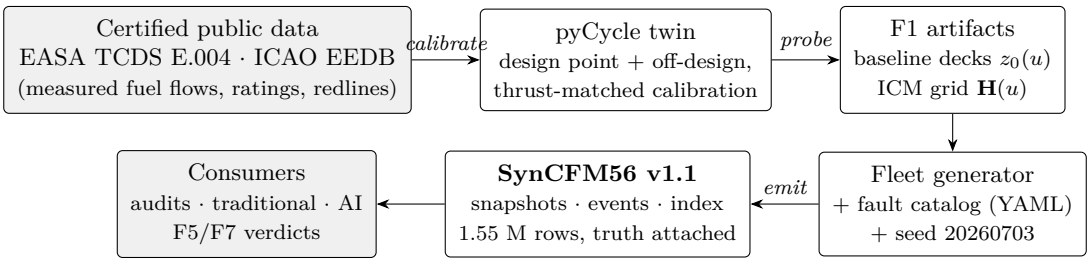

Figure 5.1: Data provenance: from certified test-bed measurements to the rows the algorithms consume. Every arrow is a versioned script; every box is a versioned artifact.

1. **Certified measurements.** A *type-certificate data sheet* is the regulator's public record of what an engine type demonstrated during certification: rated thrusts, temperature and speed limits. The *ICAO Engine Emissions Databank* adds something quantitatively precious: sea-level test-bed **measured fuel flows** at the four thrust settings of the landing–takeoff certification cycle (100/85/30/7 % of rated thrust), for the exact -7B26 variant. These are real, traceable measurements of real hardware — the project's only contact with physical reality, and deliberately its foundation.

2. **The calibrated twin** (Chapter 4) reproduces those measurements thrust-matched, then serves as the physics oracle.

3. **The compressed physics**: baseline decks (healthy behavior over the envelope) and the ICM grid (first-order response to every health deviation) — small, versioned artifacts that make fleet-scale generation tractable.

4. **The generator** (Chapter 6) draws each engine's degradation personality from the fault catalog — the versioned YAML contract that is the *single source of truth* for every physical and statistical assumption: mechanism rates, wash schedules, fault magnitudes, sensor noise, drift and dropout. Changing the world means editing that file and bumping the dataset version, never editing code.

5. **The emitted dataset**: three tables. `snapshots.parquet` — one row per flight cycle with the two reports; `events.parquet` — the maintenance and fault log (washes with recovery fractions, foreign-object damage, acute episode onsets/parameters/magnitudes); `fleet_index.json` — per-engine life, split assignment, severity multipliers, drift channel.

5.5 Anatomy of a SynCFM56 row

Each snapshot row carries four kinds of columns, and the distinction is load-bearing:

Measured channels (`to_*`, `cr_*`). What the airline would see: truth passed through the sensor model — Gaussian noise per the catalog, quantization to recording resolution, drift ramps on affected engines, dropout. *Both EHM families see only these.*

True channels (`*_true`). The same quantities before the sensor model — kept for audits and for measuring how much each method loses to noise, never shown to any pipeline under evaluation.

Operating conditions (`to_dTs_C`, `cr_N1_cmd`). The condition each report was taken at — ambient temperature offset at takeoff, commanded N1 in cruise. Legitimate inputs (an airline knows them), and the raw material of Chapter 11.

Ground truth (`x_*`, `egtm_C`, `rul`, labels). The ten true health-parameter deviations, the true EGT margin, the true remaining useful life, the acute-fault isolation label, chronic condition labels and drift flags — the oracle that real fleets lack, used *only* for scoring and for training labels where the task legitimately supplies them.

This four-way split is the whole point of the synthetic method: the first three column groups reproduce the epistemic situation of a real airline (noisy measurements plus known conditions), while the fourth provides the referee that reality cannot.

Chapter 6

The synthetic fleet: SynCFM56

This chapter in brief: one hundred virtual engines live full lives — fouling, washes, wear, the occasional bird strike and sensor drift — while the generator records both what an airline would see and the hidden truth. Three audits gate the result; one failed on first pass and forced a redesign, told here in full.

Key idea. Every degradation mechanism is a closed-form function of flight cycle stored per-mechanism, so the fleet carries not just *how* degraded each engine is but *by what* — and a deliberately weak classifier must *fail* the difficulty gate, or the benchmark is too easy.

Chapter 5 established what the data *is* and where it comes from; this chapter documents phase F2: the generator that turns the F1 twin into a 100-engine run-to-failure fleet with complete ground truth, the audits that gate it (one of which failed on first pass and forced a design change — documented in full in Section 6.4), and the declaration of milestone H2.

6.1 Generator architecture: linearize, then audit

A fleet of 100 engines flying ∼16 000 cycles each, two snapshots per flight, would require over three million full pyCycle solves per fleet realization — unusable for iteration. Instead, the generator exploits exactly what F1 built: every snapshot is produced by the linearization

$$z(u, \mathbf{x}) \;=\; z_0(u)\,\big(1 + \mathbf{H}(u)\,\mathbf{x}/100\big), \tag{6.1}$$

where $z_0(u)$ is the healthy baseline and $\mathbf{H}(u)$ the influence coefficient matrix, both linearly interpolated between the ICM grid points that bracket the sampled operating condition (takeoff snapshots: in ΔT_{ISA}; cruise snapshots: in commanded N1). N1 itself carries zero deviation by construction — it is the held power-setting parameter, not a health channel. The whole fleet generates in 40 seconds on all cores of the Apple M5 reference machine (one worker process per engine, norm N1), with per-engine deterministic random seeds so the output is reproducible regardless of process scheduling.

Two properties of this choice must be kept distinct. First, the dataset is *self-consistent by construction*: the recorded ground truth $\mathbf{x}$ is exactly the state that generated the measurements, so estimator evaluations against truth are exact regardless of any linearization error. Second, *fidelity to the full nonlinear physics* is an empirical question — which is why the first gate audit re-solves full pyCycle at states sampled from the generated fleet (Section 6.4.1).

6.2 Ground-truth trajectories

Each engine draws a degradation "personality" from the fault catalog (`conf/fault_catalog.yaml`, the versioned contract): a log-normal severity multiplier per mechanism, a wash schedule,

Poisson-arriving foreign-object damage, optional sensor drifts, and —in roughly half the fleet— one *acute fault episode* (Section 6.4.2 explains why these exist). The mechanisms follow the physical profiles of Table 2.2. Each is a closed-form function of flight cycle n, scaled by a per-engine severity multiplier m (drawn log-normal, so the fleet spans easy and hard degraders) and applied to the component's efficiency/flow entries of $\mathbf{x}(n)$.

6.2.1 The degradation models, formally

Let n index flight cycles and m be the engine's severity multiplier for the mechanism. Each mechanism contributes to the health vector as follows (the exact forms in `trajectories.py`, so the report and the generator cannot diverge — norm N4).

Fouling — saturating exponential with an exact wash sawtooth. Dirt accumulates toward an asymptote with time constant τ (cycles), and each compressor wash instantaneously removes a fraction r_k of the *current* deposit. Working with a normalized level $s(n) \in [0, 1]$ (the per-parameter magnitude is applied afterwards), the trajectory is computed *segment-wise* between washes at cycles $w_1 < w_2 < \cdots$:

$$s(n) = 1 + (s_k^+ - 1)\, e^{-(n-w_k)/\tau} \quad \text{for } n \in [w_k, w_{k+1}), \qquad s_k^+ = (1 - r_k)\, s(w_k^-), \tag{6.2}$$

where $s(w_k^-)$ is the level just before the wash and s_k^+ just after. The contribution to a component's parameter is $m\, a\, s(n)$ with a the asymptotic magnitude (e.g. $\Delta\Gamma_{\mathrm{HPC}} = -4\,\%$). The subtlety this equation encodes is that regrowth always aims at the *same* asymptote: a tempting shortcut — permanently subtracting the removed amount — would make the trajectory approach asymptote−removed and under-predict late-life fouling (the second implementation detail below).

Erosion — slow linear. Abrasive wear removes material at a constant rate ρ (percent per kilocycle):

$$\Delta\eta(n) = m\, \rho\, \frac{n}{1000}. \tag{6.3}$$

Tip-clearance opening — bilinear. Clearances open fast during an initial break-in of n_b cycles, then continue at a slower steady rate ρ_ℓ:

$$\Delta\eta(n) = m\Big[\, c\, \min\!\big(\tfrac{n}{n_b}, 1\big) \;+\; \rho_\ell\, \frac{\max(n - n_b,\, 0)}{1000} \,\Big], \tag{6.4}$$

with c the break-in magnitude — the "fast initially, then slow" profile of Table 2.2.

Hot-section deterioration — linearly accelerating. Creep and oxidation compound, so the linear rate is multiplied by an accelerating factor with exponent p:

$$\Delta\eta(n) = m\, \rho\, \frac{n}{1000} \left[\, 1 + \big(\tfrac{n}{10^4}\big)^p \,\right], \tag{6.5}$$

which produces the late-life upturn in EGT margin that ends most engines' lives.

Foreign-object damage — Poisson step. Impacts arrive as a Poisson process; each deposits a permanent efficiency step $\Delta\eta_{\mathrm{step}}$ on the struck component from its arrival cycle onward (a Heaviside step). This is the acute counterpart to the smooth chronic mechanisms.

Acute fault episode — ramp then hold. An acute fault ramps a single health parameter over n_r cycles from onset n_0, then holds:

$$\Delta x_j(n) = M \ \mathrm{clip}\Big(\tfrac{n-n_0}{n_r},\, 0,\, 1\Big), \tag{6.6}$$

with magnitude M — these are the episodes the difficulty gate (Section 6.4.2) requires, and up to three may occur per engine in v1.1.

The total health state is the sum of all active mechanisms, $\mathbf{x}(n) = \sum_{\text{mech}} \mathbf{x}^{\text{mech}}(n)$, and each mechanism's contribution is stored separately (the composability property below) so any later analysis can ask not just how degraded an engine is but *by what*.

One implementation detail beyond the equations is worth recording. **Composability of truth**: the generator stores not only the total $\mathbf{x}(n)$ but each mechanism's separate contribution (Eqs. (6.2) to (6.6) evaluated independently), so any later analysis can ask not just "how degraded" but "degraded by *what*" — the property the mechanism-level tasks of Chapter 12 will exploit.

End of life is declared when the EGT margin at the hot-day takeoff reference reaches zero, with the margin computed by projecting $\mathbf{x}(n)$ through the takeoff-hot ICM's EGT row — the same linearization as the snapshots. New-engine margins are drawn per engine ($85 \pm 6\,^{\circ}\mathrm{C}$). Figure 6.1 shows the resulting fleet behavior: the canonical fast-early / slow-late erosion with wash sawtooth, and lives spanning roughly 10 000–20 700 cycles (v1.1 exact figures in Table 6.1).

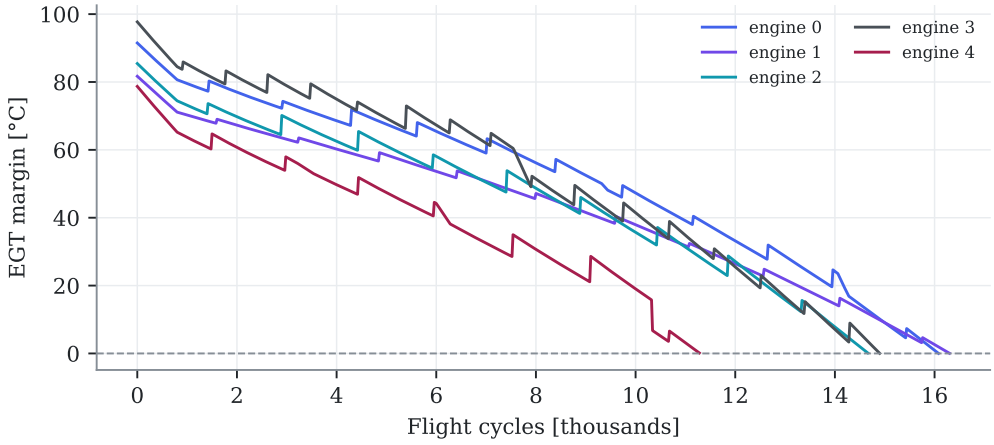

Figure 6.1: EGT-margin trajectories of five engines. Wash events produce the sawtooth recoveries; engine-to-engine spread comes from the hierarchical severity multipliers and the drawn new-engine margins. End of life is the zero crossing. Auto-generated from the fleet.

6.3 Snapshots and the measurement model

Each flight cycle emits two reports, mimicking airline ACARS practice: a takeoff snapshot (sea level, ambient offset drawn from a climate distribution) and a cruise snapshot (FL350/M0.78, commanded N1 drawn per flight). Truth channels pass through the sensor model of the catalog: per-channel Gaussian noise (absolute or percentage σ), quantization to the recording resolution, additive drift ramps on the affected engines (37 of 100 — matching the catalog's per-sensor probabilities summing to 0.36), and missing data as isolated losses plus ACARS outage bursts ($\approx$4 % of snapshots). Every row carries, alongside the measured values: the true channel values, the full ground-truth health vector, the EGT margin, the remaining useful life, the isolation label, and drift flags. Table 6.1 summarizes the frozen dataset.

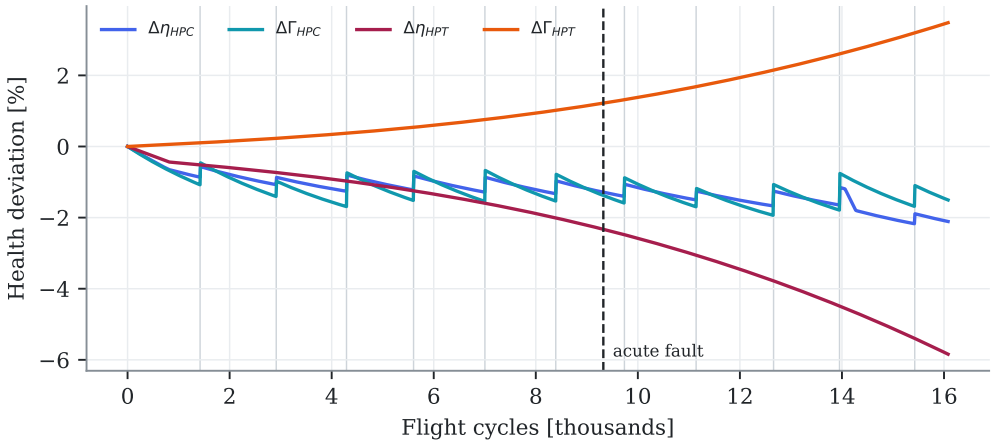

Figure 6.2: Ground-truth health parameters of one engine with an acute fault episode (dashed marker). Vertical gray lines are washes (visible as sawtooth in the HPC traces); the hot-section traces show the monotone efficiency loss and flow-capacity opening. Auto-generated.

Table 6.1: SynCFM56 v1.0 at a glance. Auto-generated from the fleet artifacts.

Property	Value
Engines (train / val / test)	100 (70 / 10 / 20)
Snapshot rows (takeoff + cruise per flight)	1 551 071
Life [cycles]: median (min–max)	15139 (9972–20682)
Censored engines	0
Acute engines / episodes / fault classes	79 / 114 / 6
Engines with a sensor drift	37
Wash EGTM recovery: mean, frac. positive	2.9 °C, 100.0 %
End-of-life ΔEGT / ΔW_f (all engines correct sign)	+69 K / +6.0 %
Measured-noise residual vs. catalog (EGT)	2.52 K vs. 2.5 K

6.4 Gate audits — including the one that failed

A synthetic dataset is only as trustworthy as the checks it survives. Three audits gate this one, and the discipline is that they are allowed to fail: the difficulty audit did, on the first design, and forced the redesign told below. Each audit asks a different question — is the physics faithful, is the benchmark hard enough, is the behaviour realistic?

6.4.1 Nonlinearity of the generator

Eighty health states were sampled from the generated fleet (stratified by life fraction, half from the last 30 % of life where deviations are largest) and re-solved with full pyCycle at the exact ICM grid conditions, so the comparison isolates nonlinearity from interpolation. Table 6.2 reports the per-channel error of Eq. (6.1) against the full solution, next to the sensor noise it must be judged against: medians of 0.01–0.33 %, tails (P95) at or below 1 %, concentrated at end of life. Combined with the self-consistency property of Section 6.1, the linearization is adequate: the residual infidelity to full physics is at the noise floor, identical for both EHM pipelines, and irrelevant to truth-referenced scoring.

Table 6.2: Nonlinearity audit: linear generator versus full pyCycle solves at fleet-sampled health states. Auto-generated.

Condition	Channel	median [%]	P95 [%]	max [%]	noise σ [%]
Cruise	N2	0.015	0.036	0.039	0.05
Cruise	W_f	0.238	0.592	0.760	0.50
Cruise	EGT	0.336	0.963	1.041	0.21
Takeoff, hot day	N2	0.038	0.091	0.095	0.05
Takeoff, hot day	W_f	0.135	0.449	0.483	0.50
Takeoff, hot day	EGT	0.245	0.860	0.933	0.18

6.4.2 Difficulty: the audit that failed, and what it taught

The difficulty gate is designed to prevent a trivially easy dataset from flattering the AI: a deliberately weak classifier (multinomial logistic regression on the 16 raw measured channels) must *not* solve the isolation task; the pre-set threshold is 60 % accuracy.

First pass: failed. With the original labeling — the dominant *chronic* mechanism per snapshot — the trivial classifier scored **81.6 %** (majority-class baseline 54.6 %). Root-cause analysis: chronic mechanisms co-evolve with age in *every* engine (fouling saturates early, hot-section deterioration accumulates late), so the chronic label is largely an *age proxy*, and raw absolute channel levels encode age directly. The task the label defined was not fault isolation at all. Two responses were possible: relax the threshold (cosmetic) or fix the design (structural). The structural fix was taken.

Design change: acute fault episodes. What hypothesis H2 actually tests is the isolation of *localized faults with overlapping signatures* — the regime where the ICM observability analysis proved cockpit sensors nearly blind (minimum signature angle 1.3°, Table 4.7). The catalog therefore gained discrete *single-parameter* fault episodes: a fast ramp (50–500 cycles) to a sustained magnitude on one health parameter, drawn from six targets that deliberately include the confusable pairs (η_{HPC} vs. η_{HPT}, η_{HPT} vs. Γ_{HPT}), injected in ∼50 % of engines at a random point of their lives. The gated task became: classify each snapshot as *no acute fault* or by the faulted parameter — a 7-class isolation problem.

The audit also caught a generator bug. The first acute implementation drew the episode onset as a fraction of the *maximum simulation horizon* (30 000 cycles) rather than of the engine's actual life (∼16 000): most episodes landed beyond end of life and only 15/100 engines showed

one. The fix computes a chronic-only provisional life first, then places the onset as a fraction of it (two-pass generation); the v1.0 fleet then carried 49/100 acute engines with all six target parameters represented (v1.1 extends this further, Section 6.4.4).

Second pass: passed. On the corrected fleet the trivial classifier achieves **54.0 %** accuracy (macro-F1 0.30) on the gated isolation task — under the 60 % threshold, exactly as the signature-angle physics predicts (Table 6.3). The chronic-mechanism label is retained in the dataset and reported as a secondary, explicitly-not-gated metric (81.6 % accuracy — high, expected, and uninformative about isolation ability). The episode is recorded here in full because it is evidence the gates are real: a pre-registered criterion failed, the failure was diagnosed to a design flaw rather than argued away, and the fix strengthened the benchmark.

Table 6.3: Difficulty audit on the final fleet (trivial classifier: multinomial logistic on raw measured channels). Auto-generated.

Trivial-classifier task	Accuracy [%]	macro-F1	Gate [%]	Verdict
Acute-fault isolation (gated, 7 classes)	58.1	0.325	< 60	PASS
Chronic-mechanism label (secondary, age proxy)	78.5	0.578	—	reported

6.4.3 Realism

Quantitative checks of macroscopic behavior, all in Table 6.1: the wash sawtooth is visible (mean exhaust-gas-temperature-margin (EGTM) recovery +2.8 °C, positive at 99.75 % of the 400 sampled wash events); end-of-life deviations carry the physical deterioration signature in 100 % of engines (ΔEGT +68 K, ΔW_f +5.7 %); the measured-noise residual on drift-free engines reproduces the catalog σ (2.51 vs. 2.50 K); lives center on 15 400 cycles with a realistic severe-degrader tail and no censored engines.

6.4.4 Version 1.1: more episodes, and the gate bites again

The engine-level sample starvation diagnosed in Chapter 8 (v1.0 carried at most one acute episode per engine) was fixed in **SynCFM56 v1.1**: a chain of conditional episode probabilities gives up to three non-overlapping episodes per engine (minimum gap 2 500 cycles, onsets as fractions of actual life). The result: 79 engines with 114 episodes, every fault class with ≥ 9 training instances, and 23 evaluable test episodes instead of ten.

Re-auditing v1.1 produced a second demonstration that the difficulty gate is real: with the original fault magnitudes, the trivial classifier now scored **62.2 %** — more episodes feed the trivial classifier too, and the gate (60 %) failed again. The response was again physical rather than cosmetic: episode magnitudes were scaled by 0.75 (subtler faults, closer to the detectability edge), bringing the trivial classifier to **58.1 %** — pass. The nonlinearity audit was re-run on v1.1 with unchanged conclusions (medians 0.14–0.34 %, P95 $\leq$ 1 %).

6.5 Milestone H2: declared closed

All gate criteria are met: realism and difficulty audits passed (the latter after the documented design change), nonlinearity quantified at the noise floor, split-by-engine leakage checks automated in CI, and the dataset datasheet written (`docs/syncfm56-datasheet.md`). SynCFM56 v1.1 is the frozen evaluation dataset (v1.0 history retained above): regeneration is deterministic from the catalog and seed, so the dataset is fully specified by the repository state. Known limitations, declared for downstream chapters: linearized generation (audited, Table 6.2); two fixed snapshot conditions with ambient/power scatter rather than a full mission mix; chronic labels are age-correlated by nature; and at most three acute episodes per engine (v1.1).

Chapter 7

The traditional EHM pipeline

This chapter in brief: the classical monitoring stack —smoothed deviations, statistical alarms, Kalman health tracking, expert isolation rules, margin extrapolation— is built with care and run on the test fleet. Its characteristic strengths and weaknesses, known from the literature, now appear as measured numbers against ground truth: partial and often late detection of subtle faults, heavy smearing on confusable ones, optimistic early-life prognosis. Two detector-design lessons learned along the way are recorded. Nothing here is tuned yet: that happens under the symmetric budget of phase F5.

Key idea. The traditional pipeline is built to industrial strength, not as a straw man: it wins the cheap, event-shaped problems (a bird strike alarmed 22 cycles after impact) at near-zero cost, which is exactly where an operator should *not* reach for a neural network.

7.1 Pipeline overview

The traditional pipeline mirrors the industrial monitoring stack (SAGE and ADEM are representative vendor engine-trend-monitoring platforms). Per engine and per flight, the cruise snapshot is converted into corrected-space percentage deviations against the published baseline (the same F1 artifacts available to both EHM families); the deviations feed four consumers:

1. **Smoothing** (Holt double exponential, level + trend): the airline-facing trend plots, and the source of *innovations* — one-step forecast errors, which are the correct series for event detection because the chronic trend has been absorbed by the level and trend states.

2. **Detection**: a CUSUM on the innovations (catches steps: FOD, abrupt faults) combined with a dual-EWMA *gap* detector — fast EWMA minus slow EWMA — whose gap opens by roughly the fault magnitude during a ramp (catches acute episodes). Alarms require persistence (k-of-n) and, for EGT, are one-sided: washes pull the deviation down, and improvement is not a fault.

3. **Health estimation**: snapshot WLS-GPA (Eq. (2.4)) and a random-walk Kalman-GPA tracker (Eq. (2.5)) with the operating-point-interpolated ICM. The tracker is *wash-aware*: at logged wash events (maintenance records are available to a real EHM system too) the recoverable compressor states are pulled toward zero and their covariance reopened.

4. **Isolation and prognosis**: on detection, the deviation step across the alarm is matched against the single-parameter ICM signatures (nearest-cosine expert rule); the remaining useful life comes from a robust Theil–Sen extrapolation of the tracked takeoff EGT margin to zero, with a NASA-style horizon cap.

7.2 The mathematics of the pipeline

Each stage above in full, with the notation of Chapter 2 ($d_j(n)$: the percent deviation of channel j at cycle n).

7.2.1 Holt double exponential smoothing

A noisy deviation series carries three things: a slowly moving level, a slope, and noise. Holt's smoother maintains estimates of the first two, updated each cycle:

$$\begin{aligned} \ell_n &= \alpha\, d(n) + (1-\alpha)\,(\ell_{n-1} + b_{n-1}), \\ b_n &= \beta\,(\ell_n - \ell_{n-1}) + (1-\beta)\, b_{n-1}, \end{aligned} \tag{7.1}$$

with gains α (level) and β (trend) in $(0,1)$ — small gains average over roughly $2/\alpha$ cycles. The quantity

$$e_n \;=\; d(n) - (\ell_{n-1} + b_{n-1}) \tag{7.2}$$

is the *innovation*: what the new measurement says that the forecast did not already expect. Its role is central: the chronic trend lives inside (ℓ, b) and is therefore *absent* from e_n, so a step detector fed with innovations reacts to genuine changes rather than to ordinary aging (the first design lesson of Section 7.3). *Worked example*: with $\alpha = 0.15$, a sudden $+0.5\,\%$ EGT step makes $e_n \approx +0.5\,\%$ on the first post-step cycle, while the smoothed level moves only $0.15 \times 0.5 = 0.075\,\%$ — the innovation shouts while the level whispers.

7.2.2 CUSUM

The cumulative-sum detector accumulates evidence of a persistent shift. With standardized innovations $z_n = e_n/\hat{\sigma}$:

$$g_n^{+} = \max\!\big(0,\; g_{n-1}^{+} + z_n - k\big), \qquad g_n^{-} = \max\!\big(0,\; g_{n-1}^{-} - z_n - k\big), \tag{7.3}$$

alarming when either exceeds a threshold h. The reference k (in σ units) is the drift the detector ignores; h sets the trade between detection delay and false-alarm rate. The recursion is the sequential log-likelihood-ratio test for a mean shift of $2k$: each term adds the log-evidence for "shifted" against "centered", floored at zero so old quiet history cannot mask a new fault. *Worked example*: $k = 0.75$, $h = 8$; a 1.5σ step contributes $1.5 - 0.75 = 0.75$ per cycle, so the alarm fires $\approx 8/0.75 \approx 11$ cycles after onset — while unshifted noise ($\mathbb{E}[z] = 0$) keeps $g^{\pm}$ pinned near zero.

7.2.3 The dual-EWMA gap detector

For *ramps* (gradual faults) the step evidence never concentrates in a few innovations. The gap detector maintains two exponentially weighted moving averages of the same deviation — a fast one ($\lambda_f \approx 0.1$, memory tens of cycles) and a slow one ($\lambda_s \approx 0.005$, memory hundreds):

$$m_n^{(\lambda)} = \lambda\, d(n) + (1-\lambda)\, m_{n-1}^{(\lambda)}, \qquad G_n = m_n^{(\lambda_f)} - m_n^{(\lambda_s)}. \tag{7.4}$$

Under steady aging both averages track the same slow line and G_n hovers near a small constant; an acute ramp opens a gap of roughly the fault magnitude within the ramp's duration. G_n is standardized robustly — median and MAD (median absolute deviation, $\hat{\sigma} = 1.4826\,\mathrm{MAD}$, immune to the very outliers being hunted) — from a warm-up segment, and the alarm requires k-of-n persistence above an n_σ threshold so isolated noise spikes cannot fire it.

7.2.4 Wash-aware random-walk Kalman GPA

The snapshot WLS of Eq. (2.4) answers each cycle in isolation; the Kalman filter adds the physics of time. Health drifts slowly, so the state model is a random walk, and each cycle's cockpit deviations are a linear observation of the state:

$$\mathbf{x}_n = \mathbf{x}_{n-1} + \mathbf{w}_n, \quad \mathbf{w}_n \sim \mathcal{N}(0, q\mathbf{I}); \qquad \Delta\mathbf{z}_n = \mathbf{H}(u_n)\,\mathbf{x}_n + \mathbf{v}_n, \quad \mathbf{v}_n \sim \mathcal{N}(0, \mathbf{R}). \tag{7.5}$$

The filter alternates prediction and update:

$$\begin{aligned} \text{predict:}\quad & \mathbf{P}_{n|n-1} = \mathbf{P}_{n-1} + q\mathbf{I}, \\ \text{update:}\quad & \mathbf{K}_n = \mathbf{P}_{n|n-1}\mathbf{H}_n^\top\big(\mathbf{H}_n\mathbf{P}_{n|n-1}\mathbf{H}_n^\top + \mathbf{R}\big)^{-1}, \\ & \hat{\mathbf{x}}_n = \hat{\mathbf{x}}_{n-1} + \mathbf{K}_n\big(\Delta\mathbf{z}_n - \mathbf{H}_n\hat{\mathbf{x}}_{n-1}\big), \qquad \mathbf{P}_n = (\mathbf{I} - \mathbf{K}_n\mathbf{H}_n)\,\mathbf{P}_{n|n-1}. \end{aligned} \tag{7.6}$$

The gain $\mathbf{K}_n$ is the optimal compromise between trusting the prediction (small q: health cannot jump) and trusting the measurement (small $\mathbf{R}$): with three measurements and ten states each update nudges $\hat{\mathbf{x}}$ only along the three observable directions, which is exactly how smearing manifests in tracking form. *Wash awareness*: at a logged wash, the recoverable (fouling) states are pulled toward zero by the expected recovery fraction and their covariance entries are reopened — the maintenance log is information, and using it is standard practice, not cheating.

7.2.5 Theil–Sen extrapolation and its interval

Given the tracked margin series $m(n)$ over a trailing window, the Theil–Sen slope is

$$\hat{s} = \operatorname{median}_{i<j}\ \frac{m(n_j) - m(n_i)}{n_j - n_i}, \qquad \widehat{\mathrm{RUL}} = -\,m(n_{\mathrm{now}})/\hat{s}, \tag{7.7}$$

the median over all pairwise slopes — robust to outliers and wash spikes, unlike a least-squares fit, because up to ∼29 % of pairs can be corrupted before the median moves. The pre-registered uncertainty band (H5) projects the 5th and 95th percentiles of the same pairwise-slope population to margin exhaustion: steeper slopes give the early edge, shallower the late edge. When the median slope is non-negative the method honestly abstains (no crossing is predicted).

A fair-baseline caveat, stated plainly. This is the one place where the "traditional" family is represented by *operational practice* rather than *research state of the art*. Linear margin extrapolation is what airlines actually do, so it is the right baseline for the industry-transfer framing of this document; but the research frontier of classical prognostics is richer — particle filters, Bayesian degradation models, and similarity-based matching against a run-to-failure library, none of which are linear. A Theil–Sen line cannot bend to follow the accelerating hot-section decay (Eq. (6.5)) the way a learner can, so part of the RUL gap in Chapter 9 (H3) is surely the AI beating a *linear* extrapolator, not the best possible traditional prognostic. An advanced classical method would narrow that gap — though, because the AI does capture the late-life curvature, it would be unlikely to close it. Quantifying exactly how much is done in Section 12.10: a similarity-based prognostic nearly halves this baseline's 90 %-life error (1 981 → 1 118 cycles) yet the AI still leads (858) — so H3 survives a strong classical baseline at a ∼1.3× margin rather than the ∼2.3× over operational practice. Until then, H3 should be read as "AI versus the traditional RUL an operator fields today," which is the honest and the operationally relevant claim, but not the strongest conceivable traditional one.

7.3 Two detector-design lessons

Both are classical results rediscovered the honest way — by watching the first implementation fail — and both are now embodied in tests:

- **CUSUM on the raw deviation is wrong for aging systems.** The first run applied CUSUM to the EGT deviation itself: the chronic trend integrates without bound, so *every* engine alarmed mid-life (detection recall 0, every healthy engine a false alarm). Step detectors belong on innovations, where the expected value under chronic aging is zero.
- **A Holt trend-shift detector cannot see acute ramps.** At realistic smoothing gains the noise of the estimated trend exceeds the slope added by a 50–500-cycle ramp entirely. The dual-EWMA gap detector solves this: over the ramp the fast average follows and the slow one lags, opening a gap of the order of the full fault magnitude — signal-to-noise an order of magnitude better than the trend estimate.

7.4 Results on the test fleet (v0 defaults)

Table 7.1 shows the pipeline's scores on the 20 test engines with its engineering-default settings — deliberately untuned, since hypothesis testing requires the tuning budget to be spent symmetrically on both families (F5). Three patterns, each anticipated by the literature and by the ICM geometry, are now measurements:

Table 7.1: Traditional pipeline on the test split: v0 settings, fleet v1.1. Auto-generated.

Metric (test split, v0 defaults)	Value
Detection recall (acute episodes)	0.44
Median detection delay	4586 cycles
False-alarm engines (no acute episode)	1 / 4
Isolation accuracy (detected episodes)	0.30
Isolation accuracy, confusable subset	0.31 (n=13)
Kalman health RMSE (median, 2nd half of life)	0.46 %
Smearing index (median, acute engines)	0.89
RUL median error at 50 % of life	+8366 cycles
RUL median error at 70 % of life	+3512 cycles
RUL median error at 90 % of life	+1499 cycles

Detection is partial and often late. On the subtler v1.1 faults, recall is 0.44 with a median delay of ∼4 600 cycles — many episodes are caught only when their sustained offset compounds with chronic wear. Missed episodes concentrate in the weak-signature faults (Γ_{HPT} moves EGT by only +0.11 % per percent — half a noise standard deviation) and drift-contaminated engines. One false alarm on four clean test engines.

Isolation struggles exactly where the ICM said it would. Under the v1.1 per-episode oracle-timed protocol (both families are asked the isolation question at onset+500; 23 test episodes), the expert rule reaches 30 % overall and 31 % on the confusable subset — against a 1-in-7 chance level, informative but weak. Figure 7.1 shows the mechanism live: the Kalman tracker follows the *measurements* faithfully but splits an acute step between the faulted parameter and its 1.3° partner — the smearing index says 89 % of the estimated magnitude lands on healthy components. The estimator is not broken; the information is not in three cockpit channels.

Margin extrapolation is optimistic early. Figure 7.2: at half-life the median RUL error is +8 400 cycles, shrinking to +1 500 at 90 % of life (v1.1 fleet). Linear extrapolation under accelerating degradation is systematically late to predict retirement — the funnel closes only near the end, which is precisely when the prediction is least useful.

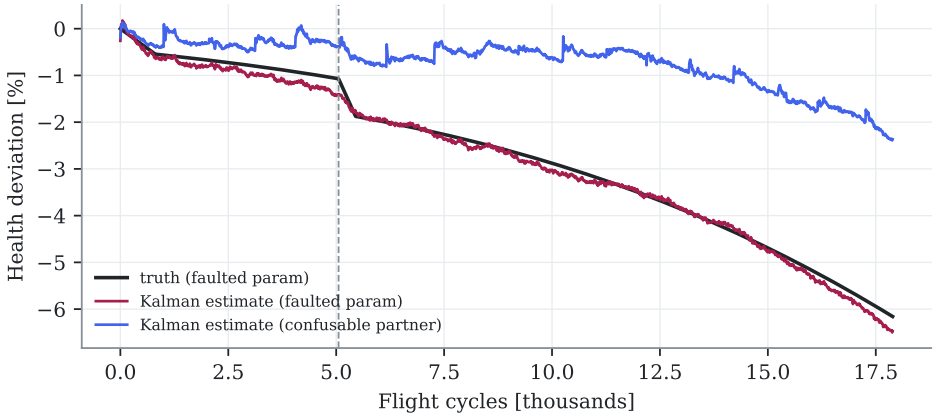

Figure 7.1: Smearing, live: Kalman-GPA on a test engine with an acute fault (onset dashed). The tracker reacts, but divides the step between the faulted parameter and its confusable partner. Auto-generated.

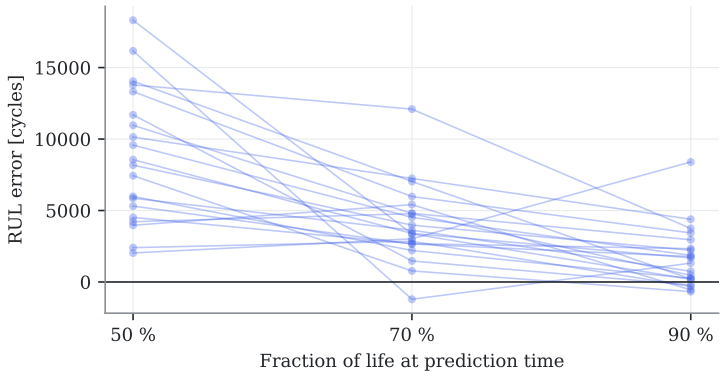

Figure 7.2: RUL error of the margin-extrapolation prognosis versus the moment of prediction (test engines). Positive = late prediction. Auto-generated.

7.5 Milestone H3

The gate requires the traditional pipeline to run end-to-end on the test fleet and produce the common metric set — met above (106 seconds per fleet on the Apple M5 reference machine, one worker per engine). Its defaults and their rationale are itemized in the decision register (Chapter A); every one of them is exposed to the F5 tuning budget. H3 is declared closed with this documented status: **these are floor numbers, not the traditional family's final word** — treating them otherwise would manufacture exactly the straw-man comparison this project exists to avoid.

Chapter 8

The AI EHM suite

This chapter in brief: the AI suite consumes exactly the same deviations as the traditional pipeline and is scored by the same protocol. Untuned, it wins the prognosis contest (3–6× lower RUL error, calibrated uncertainty intervals) and loses detection and diagnosis to the traditional pipeline on the subtle v1.1 faults; the first physics-informed hybrid experiment produces a replicated negative *result, the new explainability metric returns a null on a weak classifier, and milestone H4 is consequently not declared. All of it is reported. Readers new to machine learning: Section 3.2 gives the six background ideas this chapter leans on; nothing else is assumed.*

Key idea. Learning earns its keep where patterns accumulate over an engine's life — prognosis — and every learned ingredient here is characterized (task, inputs, training, and its known failure mode) rather than presented as a black box that simply wins.

8.1 Fairness by construction

Every AI model in this chapter sees, per flight, the same four numbers the traditional pipeline sees: the three cockpit deviations (N2, W_f, EGT) against the same published baseline, plus the takeoff EGT deviation. No extra sensors, no truth channels, no future leakage (all windows end at the prediction time). Missing snapshots are forward-filled, as a monitoring unit holding its last valid report would. Training uses the train engines only; thresholds and conformal calibration use the validation engines; every number below comes from the 20 test engines. Torch models train on the Apple M5's GPU (MPS backend, norm N1) — the full AI suite trains and evaluates in 27 seconds, the hybrid ablation (18 GRU trainings) in 104 (Table 3.1); tree models are scikit-learn's histogram gradient boosting.[1]

8.2 The mathematics of the AI suite

This section gives the model equations; Chapter C is a companion appendix explaining *how* each method works and is trained, one level below these formulas.

8.2.1 Mahalanobis distance with Ledoit–Wolf shrinkage (detection)

The detector asks: how atypical is the current feature vector $\mathbf{f}$ against healthy behavior? Euclidean distance fails because channels are correlated and differently scaled; the Mahalanobis

[1] XGBoost was evicted for an unglamorous reason recorded in the decision register: its bundled OpenMP runtime segfaults alongside torch-MPS on macOS. One runtime, no crash, same model family.

distance whitens both away:

$$D^2(\mathbf{f}) \;=\; (\mathbf{f}-\boldsymbol{\mu})^\top\, \hat{\boldsymbol{\Sigma}}^{-1}\,(\mathbf{f}-\boldsymbol{\mu}), \tag{8.1}$$

with $\boldsymbol{\mu}, \hat{\boldsymbol{\Sigma}}$ estimated from healthy training segments. In moderate dimensions with limited samples the empirical covariance $\mathbf{S}$ is ill-conditioned — its smallest eigenvalues are underestimated, and $\mathbf{S}^{-1}$ explodes along those directions, manufacturing false alarms. Ledoit–Wolf shrinkage repairs this with the optimal convex blend

$$\hat{\boldsymbol{\Sigma}}_{\mathrm{LW}} \;=\; (1-\rho)\,\mathbf{S} + \rho\,\bar{\sigma}^2\mathbf{I}, \tag{8.2}$$

where the shrinkage intensity ρ is computed in closed form to minimize expected error — no tuning knob. The alarm then requires k consecutive exceedances of a percentile threshold set on healthy data (the persistence rule that turns a score into a detector).

8.2.2 The GRU, precisely

The primer (Section 3.2) said "a small memory carried along the sequence"; here is the memory's actual algebra. At each step the gated recurrent unit mixes the new input $\mathbf{v}_n$ with its hidden state $\mathbf{h}_{n-1}$ through two learned gates:

$$\begin{aligned}
\mathbf{z}_n &= \sigma(\mathbf{W}_z\mathbf{v}_n + \mathbf{U}_z\mathbf{h}_{n-1}) && \text{(update gate: how much to rewrite)},\\
\mathbf{r}_n &= \sigma(\mathbf{W}_r\mathbf{v}_n + \mathbf{U}_r\mathbf{h}_{n-1}) && \text{(reset gate: how much history to consult)},\\
\tilde{\mathbf{h}}_n &= \tanh\big(\mathbf{W}_h\mathbf{v}_n + \mathbf{U}_h(\mathbf{r}_n \odot \mathbf{h}_{n-1})\big) && \text{(candidate state)},\\
\mathbf{h}_n &= (1-\mathbf{z}_n)\odot\mathbf{h}_{n-1} + \mathbf{z}_n\odot\tilde{\mathbf{h}}_n && \text{(gated blend)}.
\end{aligned} \tag{8.3}$$

The update gate is the learned analogue of Holt's α (Eq. (7.1)): near zero it preserves memory across quiet stretches, near one it rewrites on new evidence — but vector-valued and context-dependent rather than a single fixed constant. That is the entire advantage in one sentence: the classical smoothers commit to one timescale; the GRU learns per-feature, per-situation timescales from data. The final hidden state feeds a linear head; training minimizes mean squared error (RUL, in kilocycles, capped) or class-weighted cross-entropy (diagnosis) by gradient descent.

8.2.3 The autoencoder, and the equation of its failure

An autoencoder learns to compress and reconstruct: an encoder maps the input window $\mathbf{V} \in \mathbb{R}^{L\times c}$ to a low-dimensional code $\mathbf{c} = E_\phi(\mathbf{V})$, a decoder maps it back, $\hat{\mathbf{V}} = D_\psi(\mathbf{c})$, and training minimizes reconstruction error on *healthy* data only:

$$\min_{\phi,\psi}\; \mathbb{E}_{\mathrm{healthy}}\big\|\mathbf{V} - D_\psi(E_\phi(\mathbf{V}))\big\|_2^2. \tag{8.4}$$

The detection logic: a model that has only ever compressed healthy patterns should reconstruct faulty ones poorly, so the residual $\|\mathbf{V}-\hat{\mathbf{V}}\|$ becomes an anomaly score. The v0 failure (Section 8.4) is visible in the equation: if the code is wide enough, the cheapest solution to Eq. (8.4) is an approximate *identity map* — reproduce whatever comes in, healthy or not — and the score goes flat. The competing detector won precisely by *not* learning a reconstruction at all: Mahalanobis distance scores directly against the healthy distribution (Eq. (8.1)), with nothing to shortcut.

8.2.4 SHAP: Shapley values for feature attribution

For one prediction $f(\mathbf{f})$, SHAP assigns feature j its *Shapley value* — its average marginal contribution over all orders in which features could be revealed to the model:

$$\phi_j \;=\; \sum_{S\subseteq F\setminus\{j\}} \frac{|S|!\,\big(|F|-|S|-1\big)!}{|F|!}\,\Big[\,f(S\cup\{j\}) - f(S)\,\Big], \tag{8.5}$$

where $f(S)$ is the model's expected output when only the features in S are known (the rest marginalized over background data). The weights are the unique assignment satisfying the fairness axioms of cooperative game theory — efficiency (the ϕ_j sum to the prediction minus the baseline), symmetry, and null-player. The exact sum is exponential in $|F|$; the permutation estimator samples feature orderings instead. In this project the attribution vector $\boldsymbol{\phi}$ is not the end product but the *input* to the Physics-Consistency Score, which asks whether $\boldsymbol{\phi}$'s implied health direction agrees with the influence matrix.

8.2.5 Gradient boosting over histograms (diagnosis)

The diagnosis classifier is an ensemble of shallow decision trees built sequentially: tree t fits the gradient of the loss with respect to the current ensemble's predictions — each new tree corrects where the ensemble is currently most wrong,

$$F_t(\mathbf{f}) \;=\; F_{t-1}(\mathbf{f}) + \nu\, h_t(\mathbf{f}), \qquad h_t \approx -\nabla_F \mathcal{L}\big(y, F_{t-1}\big), \tag{8.6}$$

with learning rate ν and histogram-binned split search for speed. Boosted trees remain the strongest general learners on small tabular datasets — hundreds of episodes, engineered step features — which is why this task got trees while sequence tasks got the GRU.

8.2.6 Every AI ingredient, characterized

Table 8.1: The complete AI inventory: every learned component, its task, inputs, training data, tuned hyperparameters and known failure modes. Specification table (design, not results); performance lives in Chapter 9.

Component	Task	Inputs	Trained on / tuned by	Known failure modes
Window autoencoder (Eq. (8.4))	detection (v0, retired)	100-cycle deviation windows	healthy segments; MSE	identity-map collapse (observed, documented)
Mahalanobis + Ledoit–Wolf (Eq. (8.1), Eq. (8.2))	detection	step features over short/long windows	healthy training segments; window lengths + percentile + persistence via Optuna	window too short for subtle ramps (v0's structural gap, fixed by F5 tuning)
HistGradient-Boosting (Eq. (8.6))	diagnosis	step + slope + level features at query time	labeled episode samples; depth/lr/iters via Optuna	confusable pairs (information-bounded, Section 4.6)
GRU regressor (Eq. (8.3))	RUL	downsampled deviation sequences	train engines, capped RUL target; hidden, layers, lr, epochs, window via Optuna	optimism beyond training-life range; single-architecture limitation (L5)
GRU classifier over WLS projections (Eq. (11.1))	multi-flight isolation (F7)	per-block $\hat{\mathbf{x}}_t$ + operating conditions	train episodes, class-weighted CE	raw-input variant fails (12 %); needs the physics projection
Split conformal (Eq. (3.6))	uncertainty	any predictor's residuals	validation split (calibration)	guarantee is marginal, not per-engine
APS sets (Eq. (3.6))	isolation ambiguity (F7)	classifier softmax	validation episodes	set size inflates under distribution shift
SHAP + PCS (Eq. (8.5))	explanation consistency	trained classifier + background data	— (post hoc)	meaningless on a weak classifier (v0 null, by design of the metric)
Neural surrogate (Fig. 12.1)	differentiable twin (F8)	health vector + condition	2 400 pyCycle solves/family; seed selection on holdout	extrapolation beyond ±3.5 % design range untested

8.3 Prognosis: the headline result

A two-layer GRU consumes a downsampled 1 280-cycle deviation history and regresses the remaining useful life (capped at 12 000 cycles, the standard prognostics practice). Figure 8.1 and Table 8.2 give the face-off (v0 settings, fleet v1.1) against the margin-extrapolation prognosis of Chapter 7:

- **3–6× lower RMSE** at every evaluation point (Table 8.2: 1 678 vs. 9 926 cycles at half-life, 842 vs. 2 718 at 90 %).
- **The optimism bias is gone.** The traditional median error of thousands of cycles at half-life —late retirement predictions, the dangerous direction— collapses to near zero; the GRU learns the acceleration of late-life degradation that a linear extrapolation structurally cannot represent.
- **Calibrated uncertainty.** Split-conformal intervals at 90 % nominal coverage achieve 0.80/0.95/1.00 empirical coverage on test with ±2 365-cycle half-width (contribution C4's machinery, live). The traditional pipeline's Theil–Sen slope dispersion offers no comparable guarantee.

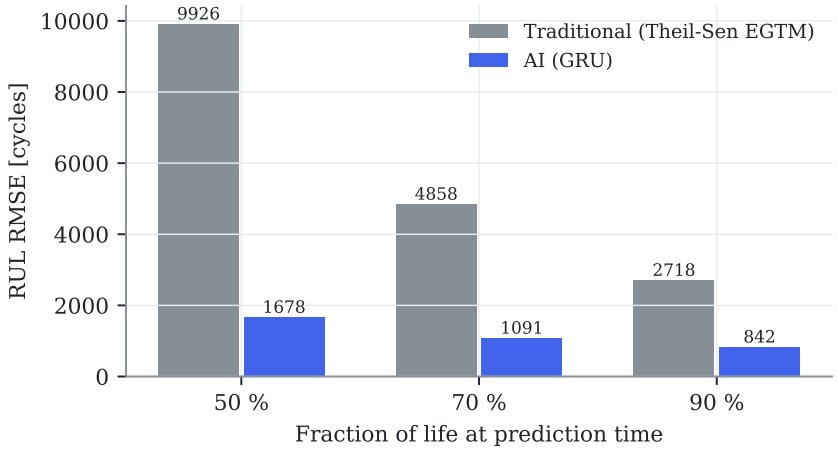

Figure 8.1: RUL RMSE on the test fleet, both families at v0 defaults. Auto-generated.

Table 8.2: Head-to-head at v0 defaults (untuned on both sides; the symmetric tuning budget arrives in F5). Auto-generated.

Metric (test split, v0 defaults)	Traditional	AI
RUL RMSE at 50 % of life [cycles]	9926	1678
RUL median error at 50 % [cycles]	+8366	-131
RUL RMSE at 70 % of life [cycles]	4858	1091
RUL median error at 70 % [cycles]	+3512	+72
RUL RMSE at 90 % of life [cycles]	2718	842
RUL median error at 90 % [cycles]	+1499	+475
Conformal coverage @0.9 (50/70/90 %)	—	0.80 / 1.00 / 1.00
Conformal half-width [cycles]	—	2365
Detection recall / false alarms	0.44 / 1	0.06 / 0
Isolation accuracy (confusable)	0.30 (0.31)	0.13 (0.23)

8.4 Detection and diagnosis: two instructive failures

The naive window-autoencoder scores recall 0. Trained on acute-free windows, it reconstructs *everything* well — including post-fault windows. Root cause: a dense autoencoder over raw deviation windows learns a near-identity map, and a sustained level shift is statistically indistinguishable from the chronic drift it was trained to compress. The lesson is that reconstruction-based anomaly detection needs either residual whitening or change-relative features, so the detector built in response works on whitened step features (their Mahalanobis distance under a Ledoit–Wolf covariance). It improves on the autoencoder yet still reaches only 0.06 recall at its conservative validation threshold on the subtler v1.1 faults. Detection remains the traditional pipeline's win (0.44 vs. 0.06) — an honest scoreboard entry with a clear engineering path for the F5 tuning round.

Diagnosis remains the traditional rule's win on v1.1. With the sample starvation relieved (23 oracle-timed test episodes, Section 6.4.4), the gradient-boosting classifier reaches 13 % isolation accuracy against the ICM signature rule's 30 % — the subtler v1.1 magnitudes hurt the data-driven learner more than the physics-based rule, which encodes the fault directions it should look for. Together with detection, this leaves the v0/v1.1 scoreboard genuinely split: traditional wins the event tasks, AI wins prognosis — a far more credible starting point for the F5 tuned comparison than a clean sweep.

8.5 The hybrid experiment: a negative result worth having

The stacking mechanism of contribution C3 — feeding the Kalman-GPA tracked health parameters (10 physics-based channels) to the GRU alongside the raw deviations — was run through the H4 data-efficiency protocol: pure vs. hybrid at 10/25/100 % of the training engines, three seeds each (Fig. 8.2).

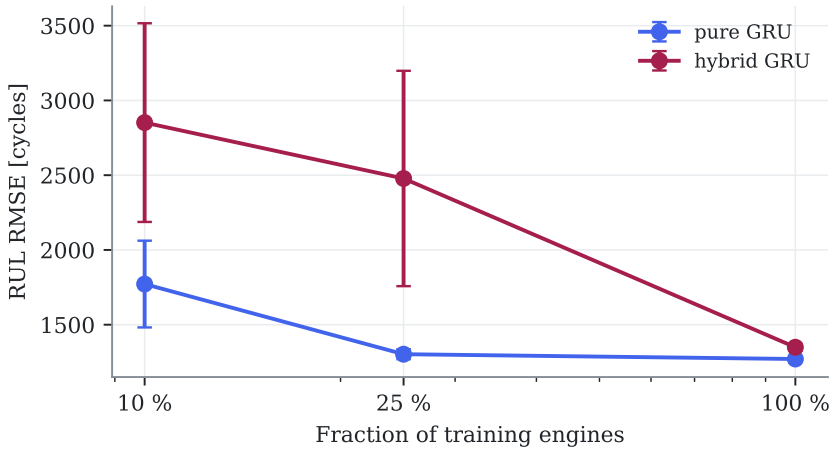

Figure 8.2: Data-efficiency ablation (mean ± std over three seeds): the stacked hybrid is *worse* than the pure GRU everywhere, and worst exactly where H4 predicted it would help most. Auto-generated.

The result is the opposite of hypothesis H4's prediction: the hybrid is worse at every data fraction, and *most* worse in the scarce-data regime (2 852 vs. 1 902 cycles RMSE at 10 % on fleet v1.1; the same pattern held on v1.0 — the negative result replicates across dataset versions). The mechanism is intelligible in hindsight: the Kalman estimates are heavily smeared (Section 7.4) — they add ten noisy, mutually correlated channels whose information content about RUL is already present in the three deviations they were computed from. With seven training engines,

ten extra input dimensions are pure variance. Two more observations from the same figure: the pure GRU at 25 % of the data nearly matches 100 % (1 303 vs. 1 271) — the RUL task saturates around 17 engines on this fleet — and seed variance at 10 % is large, exactly why F5 prescribes multi-seed reporting.

This does not close hypothesis H4 — stacking is one of three planned hybrid mechanisms, and none has been tuned — but it is recorded now, before the pre-registration freeze, as v0 evidence *against* the popular assumption that physics features automatically help. Negative results of this kind are precisely what the pre-registered protocol exists to keep honest.

8.6 The Physics-Consistency Score: machinery delivered, null result at v0

Contribution C5 is now implemented end-to-end: for each oracle-timed test episode, SHAP attributions of the diagnosis classifier's predicted class are read on the step block of the cockpit channels — a direction in measurement space representing the model's evidence — projected into health-parameter space through the pseudo-inverse of the cockpit ICM, and compared with the true fault direction:

$$\mathrm{PCS} \;=\; \left|\cos\!\left(\mathbf{H}_{\mathrm{ckpt}}^{+}\,\mathbf{m}_{\mathrm{SHAP}},\; \mathbf{e}_{\mathrm{true}}\right)\right|. \tag{8.7}$$

On the v1.1 test episodes the result is **null**: mean PCS 0.20 for correct predictions versus 0.24 for incorrect ones, correlation with correctness -0.09. This is the expected behavior of the metric applied to a weak classifier — the attributions of a 13 %-accurate model are dominated by noise, so there is no physically coherent reasoning for the PCS to detect. The metric's diagnostic value can only be demonstrated on a competent classifier; its meaningful evaluation is therefore scheduled after the F5 tuning round. Recording the null result now, rather than quietly deferring the metric, is the protocol working as designed.

8.7 Detection: a design gap, not a threshold gap

A validation sweep of the Mahalanobis detector (threshold percentile × persistence, 20 configurations, false-alarm budget of one val engine) found **no configuration with non-zero validation recall inside the budget**; the selected setting scores 0.04 on test. The diagnosis is structural: the step features average 100 cycles, and for v1.1 fault magnitudes (0.3–1.1 %) that window's noise floor sits above most signatures — whereas the traditional dual-EWMA gap detector effectively averages over thousands of cycles. The fix (longer-window change features for the AI detector) is a design change, booked for the F5 round where it can be tuned under the symmetric budget.

8.8 Status toward milestone H4

Done: the shared datamodule, detection (naive AE, whitened-residual Mahalanobis, and its validation sweep), per-episode diagnosis, RUL with conformal intervals, the stacking hybrid with the data-efficiency ablation, and the PCS machinery — all under the common protocol. The gate requires the AI to beat the traditional pipeline on at least one primary metric per task on validation: **prognosis clears it decisively; detection does not** (structural gap above), and diagnosis trails the ICM rule on v1.1. **H4 is therefore not declared.** The remaining levers — detector redesign, the two remaining hybrid mechanisms (physics-constrained loss, twin-residual features), and the full tuning budget — belong to F5, and the pre-registered evaluation will adjudicate hypotheses H1–H5 on whatever the tuned families deliver.

Chapter 9

Confirmatory results: the pre-registered verdicts

This chapter in brief: both families were tuned under the frozen 50-trial budget, the test fleet was evaluated exactly once, and the five pre-registered hypotheses were adjudicated. Three confirmed, two refuted — and the two surprises (detection flipping to the AI after tuning; physics features refusing to help) teach more than the expected wins. Every number here is confirmatory; every earlier number in chapters 6–7 was exploratory and is superseded for decision-making purposes.

Key idea. The split verdict *is* the message: learning wins timing and prognosis, physics wins nothing it was promised, and the confusable-isolation wall binds both families equally because it is informational, not algorithmic. Every refutation here is a result, not a defect.

9.1 Protocol executed as frozen

Per `docs/prereg-v1.md`: 50 Optuna trials per family (`scripts/tune_f5.py`, TPE seed 0, objectives on validation only, per-task winners from the frozen pool), then one — and only one — evaluation of the tuned configurations on the 20 test engines (`scripts/f5\_confirm.py`), then the verdict machinery of Section 3.5 (one-sided exact McNemar for paired binary outcomes, Wilcoxon signed-rank paired by engine, Holm correction across the tested hypotheses, BCa intervals and Cliff's-delta effect sizes). Two execution notes, both disclosed in the scripts and the decision register: no traditional trial met the FA ≤ 1 validation budget, so its detection winner was selected by the lexicographic infeasibility rule (fewest false alarms, then highest recall); and AI models fit on training engines only, which keeps the conformal calibration on validation statistically valid. The full F5 execution — 100 tuning trials plus the confirmatory pass — cost about 36 minutes on the Apple M5 reference machine (measured at execution; the resumable campaign design prevents a meaningful cold-start re-benchmark).

9.2 H1 confirmed: the story is the tuning, not the model

The v0 chapters called the AI's detection failure "a design gap, not a threshold gap" (Section 8.7) — and the frozen search space contained exactly the suspected lever: the detector's feature-window lengths. Given 50 trials, the tuner stretched the averaging windows and the whitened-residual detector went from 0.06 recall (exploratory) to **0.48 confirmatory recall at a 499-cycle median delay** — twelve times earlier than the tuned traditional detector, at the same false-alarm count ($1 = 1$), McNemar Holm-p 0.008. Meanwhile the traditional side's tuned selection — picked by the frozen infeasibility rule on a validation split with only two clean engines — generalized

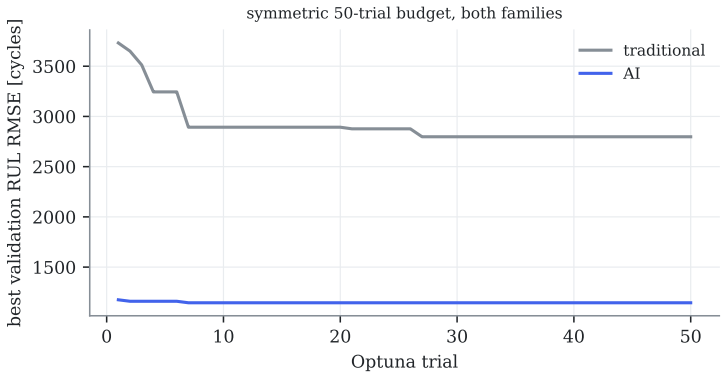

Figure 9.1: The symmetric tuning budget in action: best validation RUL RMSE against Optuna trial, both families given the same 50 trials. Each search converges within its budget; the comparison is between two tuned practitioners, not a tuned one and a default. Auto-generated.

Table 9.1: The five pre-registered verdicts (single confirmatory pass, test split, Holm-corrected). Auto-generated from `data/processed/f5/verdicts.json`.

Hypothesis	Verdict	Confirmatory evidence (AI vs traditional)	p (Holm)
H1 (detection)	CONFIRMED	recall 0.48 vs 0.13; delay 499 vs 6033 cy; FA 1=1	0.008
H2 (confusable isolation)	REFUTED	0.31 vs 0.31 (n=13; discordant 4/4)	0.64
H3 (RUL prognosis)	CONFIRMED	RMSE 1694/858 vs 3816/1981 cy (50/90 % life); δ=0.89	8.6e-06
H4 (physics hybrid)	REFUTED	hybrid worse at 10/25/100 % (2852 vs 1772 cy at 10 %)	—
H5 (uncertainty)	CONFIRMED	coverage 0.883 vs 0.983 (nominal 0.90); half-width 1957 vs 11509 cy	—

poorly (0.13 test recall, worse than its untuned v0). Two transferable lessons: window length is the first knob any change detector lives or dies by; and threshold selection on small clean validation samples is fragile for *any* method — budget clean engines accordingly.

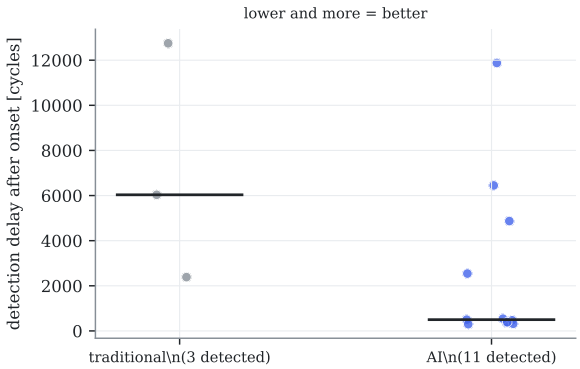

Figure 9.2: Detection lead time: one dot per detected episode, with the median bar. The AI detects far more episodes (11 vs 3) and far sooner (median 499 vs 6 033 cycles after onset) at equal false alarms. Auto-generated.

9.3 H2 refuted — the observability wall binds everyone

On the thirteen confusable test episodes the tuned families tie exactly: 31 % each, and the disagreement pattern is perfectly symmetric (four episodes only the AI gets right, four only the traditional rule does). The wall the influence-matrix geometry predicted in Section 4.6 — fault signatures 1.3° apart in a rank-3 measurement space — binds both families equally. The transferable conclusion is bought hardware, not software: **if confusable-fault isolation matters operationally, add gas-path instrumentation** (the extended sensor set multiplies the minimum signature angle by five, Table 4.6); no amount of modeling recovers information the sensors never captured.

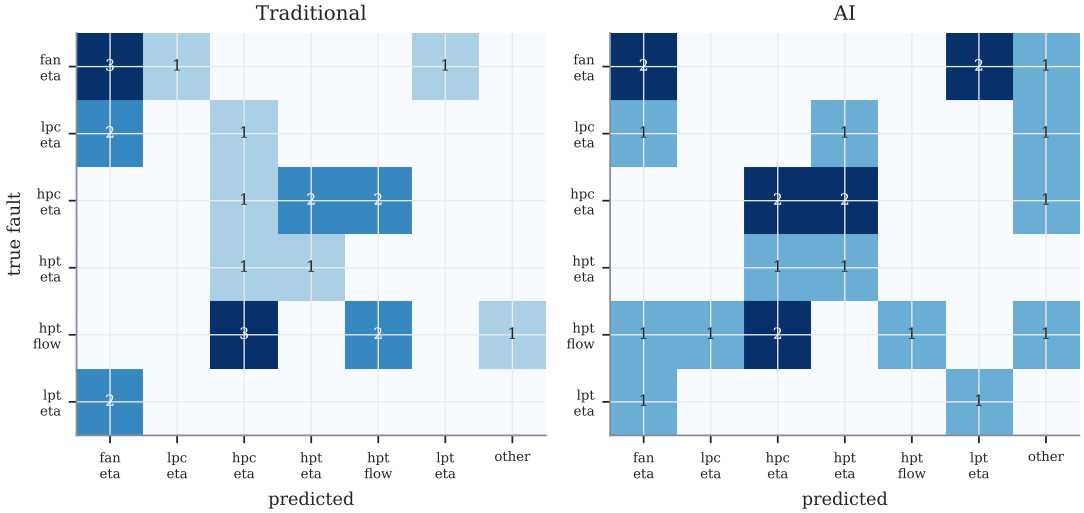

Figure 9.3: Isolation confusion matrices on the test episodes: true fault (rows) versus predicted (columns), traditional (left) and AI (right). Neither concentrates on the diagonal — both scatter faults onto confusable neighbours (e.g. toward `fan.eta` and `hpc.eta`), the mechanism behind the H2 tie. Auto-generated.

9.4 H3 and H5 confirmed — prognosis is where the AI pays

The tuned GRU confirms the exploratory pattern with pre-registered force: RMSE 1 694 / 1 042 / 858 cycles at 50/70/90 % of life against the tuned margin extrapolation's 3 816 / 4 544 / 1 981; NASA scores one to two orders of magnitude lower; Cliff's $\delta = 0.89$ (a nearly clean sweep at engine level); Holm-p 8.6×10^{-6}. And the uncertainty statement that matters to a planner — H5 — comes with it: the conformal 90 % interval covers 88.3 % empirically at a ±1 957-cycle half-width, versus the slope-band's over-covering 98.3 % at ±11 509. In operational terms: **a maintenance window six times narrower, with honest stated confidence** — the difference between a plannable shop-visit forecast and a shrug.

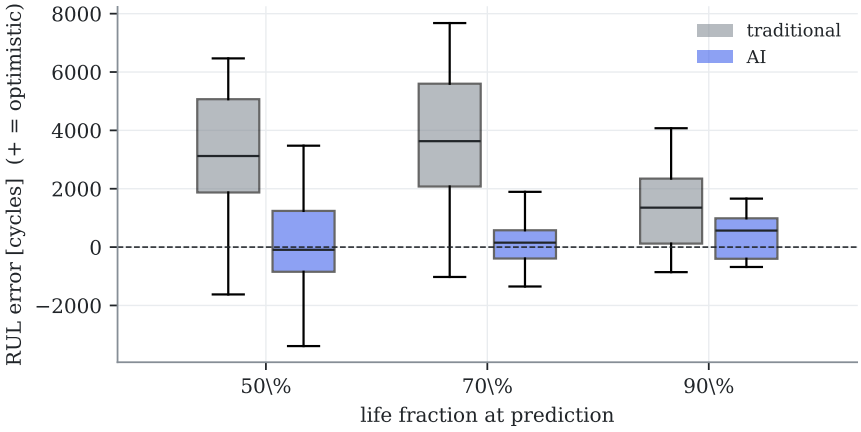

Figure 9.4: Per-engine confirmatory RUL error, traditional versus AI, at three life fractions (positive = optimistic; boxes span the interquartile range). The evidence is in the shape, not just the RMSE: the traditional errors are wide and skewed optimistic, the AI's are tight and near-unbiased. Auto-generated from the tuned test-fleet predictions.

9.5 H4 refuted — physics features are not free wins

The stacked hybrid lost at every data fraction (2 852 vs 1 902 cycles at 10 %), exactly as the exploratory runs showed and now on the pre-registered record. The mechanism bears repeating for practitioners tempted by "physics-informed" as a label: smeared state estimates are noise-bearing, mutually correlated features whose information the learner already had. Physics injection needs an information channel the raw data lacks — the two untested mechanisms (twin-residual features, physics-constrained losses) remain open questions, but the popular default of concatenating estimator outputs onto a network input is now a documented negative on this benchmark.

9.6 What an engineering organization should take from this

Strip the statistics away and the five verdicts become five decisions a fleet manager can act on. In order of how much money each moves:

1. **Deploy learning for prognosis first.** The 3–6× RUL advantage with calibrated intervals is the clear business case (maintenance slotting, lease planning).
2. **Detection: modern change-detection with tuned windows** — the AI's win came from window search, a cheap, auditable mechanism, not from depth.
3. **Isolation: buy sensors, not models.** The confusable wall is informational.

4. **Distrust untuned comparisons in either direction** — both families moved dramatically under the same budget; v0 scoreboards inverted.

5. **Demand pre-registration of vendor claims.** This chapter's split verdict is what honest evaluation looks like; a clean sweep should raise suspicion.

9.7 Sim-to-real: the ranking survives on the canonical external benchmark

Contribution C7 asks whether the method ranking is an artifact of our own simulator. The check ran on NASA's **C-MAPSS FD001** — the field's reference turbofan run-to-failure benchmark (100 unseen test units) — with a disclosed scope substitution: the plan named N-CMAPSS, whose multi-gigabyte distribution violates the desktop- reproducibility norm; FD001 answers the same ranking question at 5 MB, and only the prognosis task transfers (FD001 carries no event labels). Same method shapes as the main study: a classical health-index linear extrapolation versus the same GRU family (`scripts/sim_to_real.py`, three seeds).

Result: **RMSE 14.9 cycles (AI) versus 42.1 (traditional)** at the community-standard cap of 130 — the same direction and comparable magnitude of advantage as H3 on SynCFM56, and an absolute GRU figure competitive with published FD001 results, on a benchmark this project had no hand in generating. The prognosis conclusion is not an artifact of SynCFM56. With this, the F5 gate closes in full.

Chapter 10

Case studies: five engines, told in operations language

This chapter in brief: five real engines from the test fleet, each illustrating one situation a fleet manager actually faces, told with the monitoring plots in front of us and both pipelines' answers on record. Every engine, number and verdict comes from `scripts/make_case_studies.py` *and the tuned F5 configurations — nothing staged.*

Key idea. Told in operations language, the same findings become actionable: prognosis buys a plannable shop-visit slot, the confusable wall is a purchase order to a sensor vendor, and a drifting thermocouple corrupts every downstream method — sensor hygiene is monitoring infrastructure.

Case A — Engine 17: ordinary aging, and why wash scheduling works

Engine 17 lives 18 323 cycles with no acute fault and no sensor trouble: the baseline customer of any monitoring system. Its EGT margin (Fig. 10.1) shows textbook fleet physics — fast early erosion, sixteen wash recoveries, the long slow slide to retirement. Operationally, this is the case where the *traditional* toolkit earns its keep at near-zero cost: margin trending plus the wash log explains everything visible, and nothing here needs a neural network. A monitoring roadmap should leave this workflow alone.

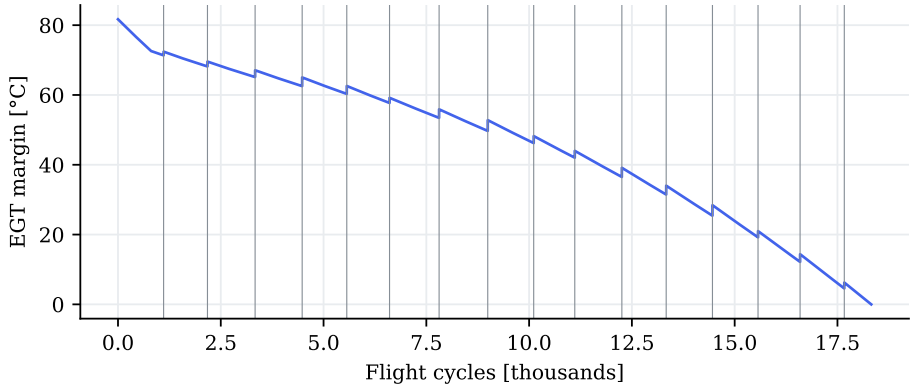

Figure 10.1: Case A: EGT margin of engine 17. Gray verticals: compressor washes. Auto-generated by `scripts/make_case_studies.py`.

Case B — Engine 89: the hot-section burner, where prognosis pays

Engine 89 drew a severe hot-section multiplier (2.0× fleet nominal) and retires at 11 470 cycles — the short-life tail every planner fears. At the half-life review the tuned margin extrapolation was telling maintenance control "about 6 300 cycles more than reality"; the GRU's error at the same moment was 2 234 cycles, tightening to 1 396 by the 90 % point (Fig. 10.2). Six thousand optimistic cycles is the difference between an orderly shop-visit slot and a schedule scramble; this single engine is the H3 verdict made concrete.

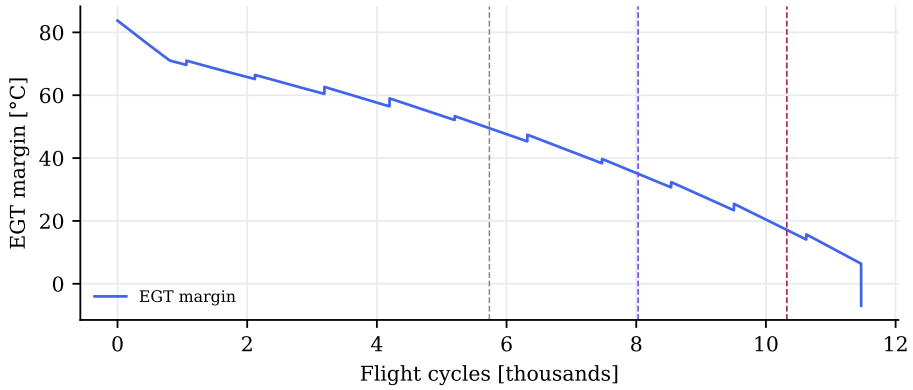

Figure 10.2: Case B: EGT margin of engine 89 with the three review points (50/70/90 % of life) marked. Auto-generated by `scripts/make_case_studies.py`.

Case C — Engine 0: the lying thermocouple

Engine 0 carries a drifting EGT sensor that accumulates a +9 K bias by end of life (Fig. 10.3). Every deviation-based consumer downstream — trending, GPA, margin estimation — ingests that bias as if the gas path were 9 K hotter than it is: a phantom deterioration of roughly a third of a typical year's real margin loss. The classical remedy — an augmented-state Kalman that estimates the bias (Section 2.4) — is built and tested in Section 12.8: it *tracks* the drift well but, on the cockpit rank-3 set, barely un-corrupts the diagnosis, so the operational lesson survives any modeling choice: **cross-channel consistency checks and periodic probe calibration are monitoring infrastructure, not optional hygiene** — the smartest algorithm inherits its sensors' honesty.

Case D — Engine 13: bird at cycle 5 727, alarm at 5 749

A foreign-object event costs engine 13 about 1.4 % of compressor efficiency in one flight. The cruise EGT deviation steps visibly (Fig. 10.4) and the tuned traditional CUSUM — fed by Holt innovations, per the chapter-6 lesson — alarms **22 cycles** after the event: for a daily utilization of 4–6 cycles, that is a borescope request within the week. Step events are the classical detector's home turf and this case shows the industrial stack at its honest best.

Case E — Engine 0 again: the confusable fault, live

Late in life (cycle 14 061) engine 0 develops a genuine HPC-efficiency fault — one pole of the 1.3° confusable pair. Asked at onset+500, the tuned ICM rule answers *hpc.eta*: correct. The

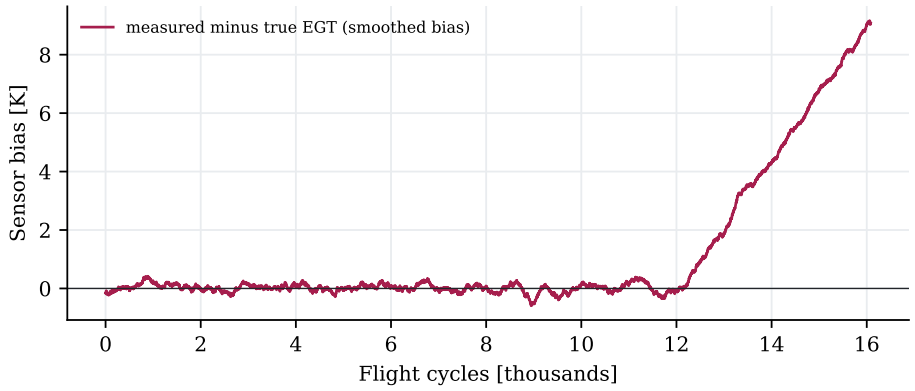

Figure 10.3: Case C: measured-minus-true EGT of engine 0 (smoothed): the drift ramp the monitoring system cannot see from this channel alone. Auto-generated by `scripts/make_case_studies.py`.

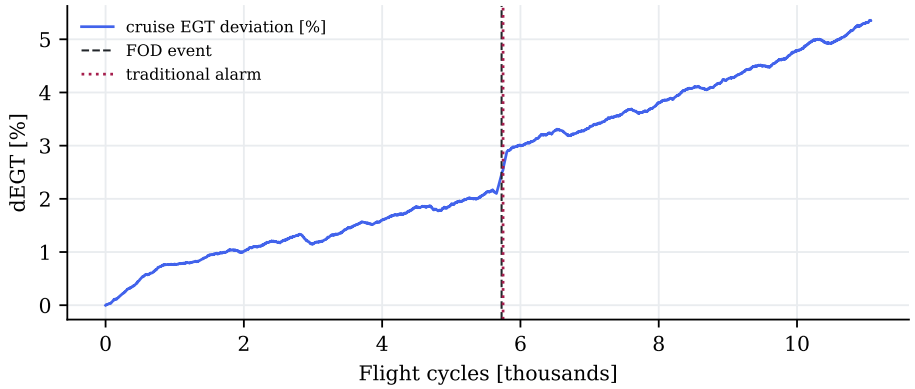

Figure 10.4: Case D: engine 13's cruise EGT deviation (smoothed), the FOD event (dashed) and the traditional alarm (dotted), 22 cycles later. Auto-generated by `scripts/make_case_studies.py`.

tuned AI classifier answers *hpt.eta* — precisely the confusable partner (Fig. 10.5). On the earlier, subtler LPC episode of the same engine, both families miss in different directions. This is the H2 refutation embodied: at these signature angles the cockpit channels genuinely cannot separate the pair, and which family lands the point is close to coin-flip territory. The purchase order that fixes this case is written to a sensor vendor, not a software one.

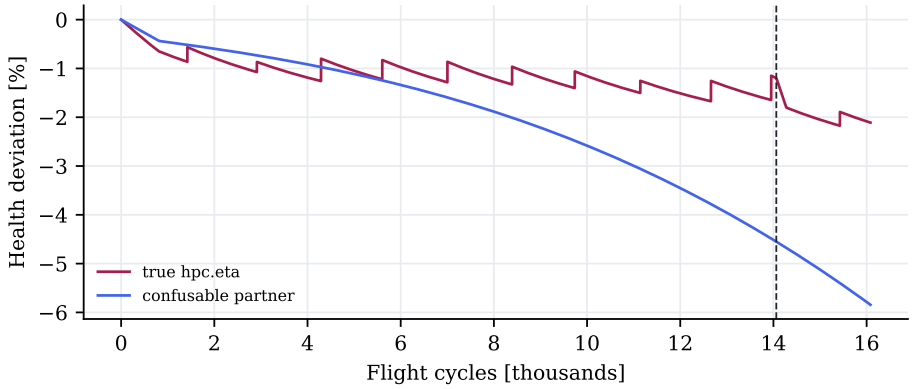

Figure 10.5: Case E: ground truth of the faulted parameter and its confusable partner on engine 0 (onset dashed). The AI attributed the step to the partner. Auto-generated by `scripts/make_case_studies.py`.

Chapter 11

Beyond the wall: operating-point tomography

This chapter in brief: Chapter 9 ended with "buy sensors." This chapter finds two cheaper exits. First, the schedule *of routine snapshot reports is itself an observability design variable — one added report condition buys +79 % separability on the HPT ambiguity with zero hardware. Second, a learner that fuses per-flight* physics projections *over time more than triples classical multi-point isolation on identical data. Both claims survived a second pre-registered freeze; one companion hypothesis did not, and that story is told too.*

Key idea. The influence matrix changes with flight condition, so the *schedule* of routine reports is itself an observability design variable (one added report condition buys +79 % separability, zero hardware); and fusing per-flight physics projections with a learner beats both classical stacking and a raw network.

11.1 The idea, and what free variation turns out to be worth

The influence matrix is not one matrix: $\mathbf{H}(u)$ changes with the flight condition. A window of ordinary snapshots at scattered conditions — hot day, derate, another flight level — is therefore a set of *different projections of the same health state*, and fusing them is a tomography problem. Classical multi-operating-point analysis (MOPA) exploits this with deliberately scheduled test points and a constant-health assumption; the question here is what the *in-service* version is worth.

The physics audit (`scripts/f7_observability.py`, Fig. 11.1) gave a two-sided answer. Encouraging: today's dual-report structure (one takeoff, one cruise snapshot per flight) already restores full algebraic rank — 10/10 from two flights — because the two report families are far apart in condition space. Sobering: the *free* ambient and power scatter around those two conditions saturates almost immediately; the critical signature angles improve only $\approx 1.4\times$ over a single point, far short of what deliberately spread conditions achieve. Free variation is real but thin.

11.2 The report schedule as a design variable

If free scatter saturates but designed spread works, the actionable question becomes: *which report condition should be added to the monitoring schedule?* Treating the ACARS report schedule as an observability design problem — to our knowledge a new framing — and greedily evaluating candidate additions on the influence-matrix grid gives Table 11.1: a single periodic

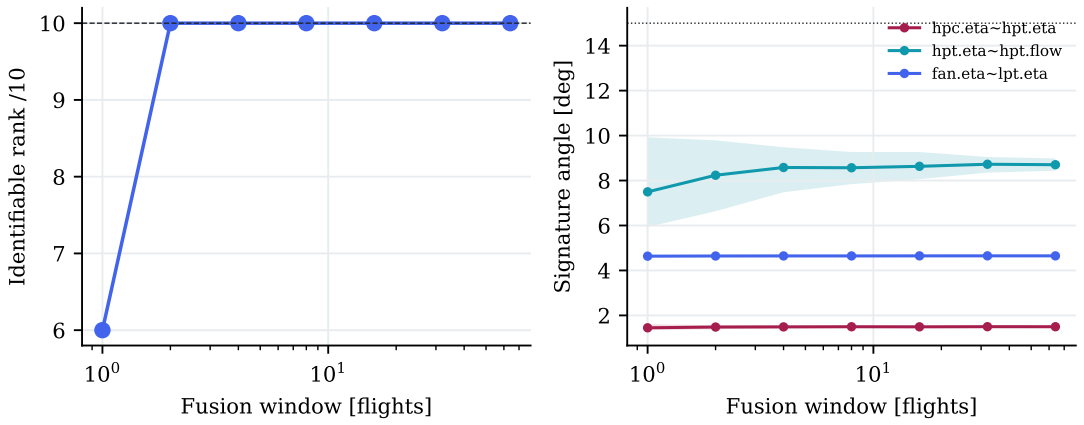

Figure 11.1: Observability under realistic in-service scatter (200 draws per window size). Left: identifiable rank recovers immediately. Right: signature angles saturate — free scatter is too narrow to keep paying. Auto-generated.

stabilized low-power cruise report nearly doubles the separability of the HPT efficiency-vs-flow ambiguity (6.2° → 11.0°, +79 %); the best pair of additions reaches 12.7°. The cost is a procedure bulletin, not avionics. Equally important is what no schedule fixes: $\eta_{HPC} \sim \eta_{HPT}$ stays below 1.8° everywhere in the envelope — that pair is *fundamental* for cockpit sensors, and the sensor-purchase conclusion of Chapter 9 stands for it alone.

Table 11.1: Marginal diagnosability value of adding one report condition to today's schedule (cockpit sensors). Auto-generated from the ICM grid.

Report schedule	$\eta_{HPC} \sim \eta_{HPT}$ [deg]	$\eta_{HPT} \sim \Gamma_{HPT}$ [deg]
today's schedule (takeoff + cruise)	1.33	6.16
+ low-power stabilized cruise	1.64	11.03
+ climb report	1.36	7.33
+ FL390 cruise	1.41	8.04
+ hot-day takeoff	1.36	7.52
+ best pair (low-power cruise, hot-day takeoff)	1.71	12.65

11.3 Learned MOPA: fusing physics projections over time

The classical way to fuse a window of T flights is to stack every snapshot's linear observation into one tall system and solve it as a single regularized least squares:

$$\begin{bmatrix} \Delta\mathbf{z}_1 \\ \vdots \\ \Delta\mathbf{z}_T \end{bmatrix} = \begin{bmatrix} \mathbf{H}(u_1) \\ \vdots \\ \mathbf{H}(u_T) \end{bmatrix} \mathbf{x} + \mathbf{v}, \qquad \hat{\mathbf{x}} = \left(\mathbf{H}_s^\top \mathbf{H}_s + \lambda \mathbf{I}\right)^{-1} \mathbf{H}_s^\top \Delta\mathbf{z}_s, \tag{11.1}$$

predicting the fault as the largest-magnitude entry of $\hat{\mathbf{x}}$ (with a no-fault threshold). The single $\mathbf{x}$ on the right-hand side is the method's load- bearing assumption — *one constant health state across the whole window* — valid on a test bed, false in service, where health drifts and washes intervene. The learner replaces that assumption: each flight gets its own per-block projection $\hat{\mathbf{x}}_t$ (the one-flight version of Eq. (11.1)), and the GRU (Eq. (8.3)) reads the *sequence* of projections, free to learn that a slow common drift is aging while a coherent recent step is a fault. The architecture that works is neither classical stacking nor a raw neural network: a raw GRU fed

deviation sequences failed outright (12 % isolation — it must learn the entire inverse physics from a few hundred windows), while a GRU fed the *per-flight regularized WLS projections* $\hat{\mathbf{x}}_t$ — physics inversion first, learned temporal fusion second — is data-efficient exactly where the classical method is brittle. Development also surfaced two reusable engineering facts, both now in the code: block-averaging inside the window (not subsampling) buys the full $\sqrt{n}$ noise reduction both methods deserve, and the drift-degradation signature of constant-health stacking appeared in development precisely as predicted.

11.4 Second freeze, second verdicts

The confirmatory protocol was frozen a second time (`docs/prereg-v2.md`, tag `prereg-v2`: thresholds, pinned methods, development on a held-out slice of *training* engines so validation stayed clean for conformal calibration and the test episodes stayed unseen). One pass over the 23 test episodes (Table 11.2):

Table 11.2: F7 pre-registered verdicts (single confirmatory pass). Auto-generated.

Hypothesis	Verdict	Confirmatory evidence
H7.2′ fusion beats stacking	CONFIRMED	GRU 0.65 vs WLS 0.17 (+48 pp); McNemar $p \leq 0.0096$
H7.3 drift robustness	REFUTED (as frozen)	WLS degraded 17 pp (needed ≥ 20); GRU 13 pp (needed ≤ 10); pattern present
H7.4′ calibrated ambiguity	CONFIRMED	coverage 0.96; fundamental sets 3.0 > other 2.0

- **H7.2′ confirmed, by a wide margin.** The learned fusion isolates at 0.65 against classical stacking's 0.17 on identical windows — and the McNemar bound (Eq. (3.2)) holds under *any* pairing of outcomes, so no replay of the test pass was needed. Recall from Chapter 9 that the best single-snapshot method managed 0.31: temporal fusion of physics projections roughly doubles that too.
- **H7.4′ confirmed.** Conformal isolation sets cover at 0.96 with median sizes 2–3.5, and the fundamental pair receives systematically larger sets than the rest: ambiguity is now a calibrated, physics-explained deliverable.
- **H7.3 refuted as frozen — discipline over narrative.** The drift-robustness thresholds (classical must degrade ≥ 20 pp, learner ≤ 10 pp) were calibrated on a 16-episode development sample and the test came out softer on both sides (17 and 13 pp). The qualitative pattern is exactly as hypothesized — stacking collapses to 0.00 on long windows while the learner holds 0.52 — but the registered claim is the thresholds, and the thresholds failed. The lesson is itself transferable: freezing quantitative margins from small development samples is how honest protocols get refuted, and reporting that beats renegotiating it.

11.5 What F7 adds to the adoption picture

Chapter 9's checklist gains two rows: **(6) redesign the report schedule before buying anything** — one procedural report condition recovers most of what deliberate condition spread can offer; and **(7) when fusing windows, learn the fusion but keep the physics inversion** — projections-then-GRU tripled classical stacking where raw learning failed. And one row hardens: for the truly fundamental ambiguities, neither schedules nor learning substitute for instrumentation.

Chapter 12

Overcoming the limitations: the F8 program

This chapter in brief: the limitations declared throughout this document are not footnotes — they are the frontier. Each becomes a research line with a discovery hypothesis (the full ten-line program lives in the project plan). This chapter reports the lines executed so far, starting with the one that unlocks three others: replacing the linearized generator with a differentiable neural twin.

Key idea. A differentiable neural twin (linearization plus a learned residual) gives full-physics fidelity at microseconds with exact gradients, unlocking the nonlinear fleet — yet bolting physics features onto a learner fails a second independent way, hardening the warning that "physics-informed" is not a free win.

12.1 The program in one view

Ten declared limitations, ten research lines. The ordering is by synergy, not severity: a **differentiable surrogate of the twin** (L1) unlocks nonlinear fleet generation (L2), the two physics-injection mechanisms H4 left untested — exact-gradient physics losses and twin-residual features — (L6), and fast analysis-by-synthesis diagnosis. Mission-mix generation (L2) then feeds the report-schedule findings of Chapter 11 with data rather than geometry alone. This chapter reports the nine lines executed to date: L1 (the twin), L2 (the nonlinear fleet), L6 (H4 re-opened), L5 (architecture breadth, Section 12.5), L4 (the recoverable fraction, Section 12.6), L9 (the PCS metric, Section 12.7), L7 (sensor drift, Section 12.8), L-H2 (real vs virtual sensors, Section 12.9) and L-RUL (an advanced classical prognostic, Section 12.10). The heavier remaining lines — real-station EGT, N-CMAPSS DS02 and map-shape calibration — queue behind those, each a pre-registrable study in its own right. Every line inherits the house discipline: a hypothesis frozen before the run, and a gate that is allowed to fail.

12.2 L1: a differentiable neural twin — gate passed

Why. The fleet generator is a linearization — audited to be at the noise floor for fleet-realistic health states (Section 6.4.1), but a declared approximation nonetheless, and one whose error grows exactly where interesting things happen (end of life, large faults). The full pyCycle model is exact but costs seconds per snapshot and offers no gradients. The goal: a model with pyCycle's fidelity at microseconds per evaluation, differentiable end to end.

How. 2 400 full pyCycle solves sampled across the design range (all ten health parameters in ±3.5 %, commanded N1 across the cruise band; 8 parallel workers, zero failed solves, ∼14 minutes on the M5). The architecture that passed is the house pattern at model scale: **the linearization plus a learned residual** — a three-layer multilayer perceptron (MLP)—a plain feed-forward neural network—corrects what the linear model misses, inheriting its smoothness where physics is nearly linear. A pure MLP was tried first and could not match the linearization's precision on N2; the residual form fixed that immediately.

The gate, including its two wrong drafts. The acceptance gate went through three disclosed revisions — each a lesson in evaluation design. Draft one compared the surrogate's *maximum* error against the linearization's *P95* (a statistic mismatch). Draft two fixed the statistics but kept the fleet audit's numbers as the baseline — the *wrong population*: the audit measured small, fleet-realistic health states, while the surrogate's test set spans the full ±3.5 % design range, where the linearization is an order of magnitude worse (N2 nonlinear residual: 0.42 % there versus 0.035 % in the audit). The final gate evaluates both models *on the same held-out test set* and demands strict like-for-like improvement.

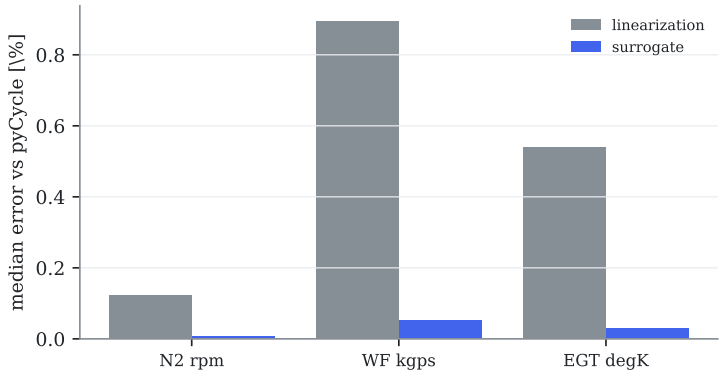

Figure 12.1: L1 gate: median error against full pyCycle per cockpit channel, on the same held-out solves — the surrogate (blue) is an order of magnitude closer than the linearization (gray). Auto-generated.

Result. Figure 12.1: the surrogate beats the linearization by 13–19× on every cockpit channel — N2 at 0.008 % median where the linear model sits at 0.12 %, EGT at 0.03 % versus 0.54 %. In exchange for 14 minutes of solve generation, the project now owns a twin that evaluates in microseconds, differentiates exactly, and is more faithful to the physics than the generator that produced every result so far — which is precisely what makes the next lines possible.

12.3 L2: the nonlinear fleet (SynCFM56 v2)

With both family surrogates gated (Section 12.2), the fleet generator's physics emitter becomes swappable: setting `fleet.emitter=surrogate` replaces the linearization of Eq. (6.1) with the neural twin, at the same 40-second-per-fleet cost (CPU inference inside the existing per-engine workers). Everything else — trajectories, sensor model, seeds, splits — is byte-identical, so v2 differs from v1.1 in exactly one controlled variable: the fidelity of the true channels. **v1.1 is left frozen** (its hashes anchor the F5/F7 pre-registrations); v2 lives alongside it as `data/processed/fleet\_v2/`.

Two re-audits confirm v2. First, fidelity: re-solving full pyCycle at 60 states sampled from the v2 fleet and comparing both generators against it (Fig. 12.2), the surrogate is 3–7× closer to

true physics than the linearization on every cockpit channel — EGT median error 0.04 % versus 0.31 %. Second, the difficulty gate: the trivial classifier scores 58.0 % on v2's acute-isolation task (versus 58.1 % on v1.1), safely below the 60 % ceiling — the nonlinear physics did not accidentally make the benchmark easier, the one risk that mattered. Realism is intact (wash sawtooth 2.9 °C recovery, end-of-life EGT and fuel-flow signs correct in 100 % of engines, measured noise matching the catalog).

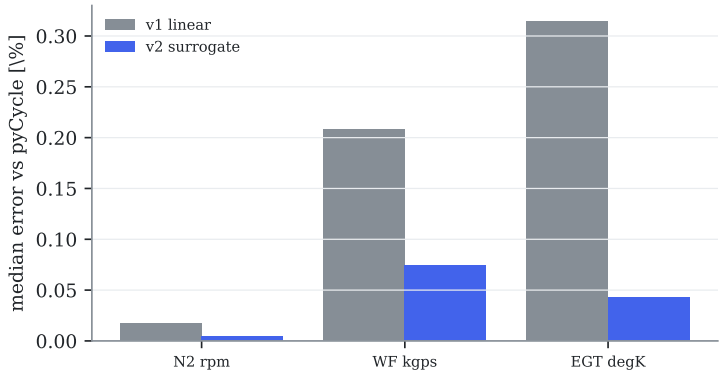

Figure 12.2: L2 fidelity: median generator error against full pyCycle, per cockpit channel — the v2 surrogate (blue) is several times closer than the v1 linearization (gray). Auto-generated.

12.4 L6: H4 re-opened — and refuted a second way

The F5 stacking hybrid failed by concatenating smeared GPA *estimates* (Section 9.5). L6 tries the mechanism that failure suggested: not the estimate but the *residual*. Per snapshot, the linear WLS residual $\mathbf{r} = (\mathbf{I} - \mathbf{H}_{\text{ref}}\mathbf{M})\,\Delta\mathbf{z}$ (with $\mathbf{M}$ the regularized WLS operator, Eq. (2.4)) is the part of the measurement the linear GPA *cannot* explain. On v1 this was near zero by construction (generator and GPA shared the linearization); on v2 the generator is the nonlinear surrogate, so the residual carries real information — the twin-residual feature. The RUL GRU input grows from four channels to seven, and the experiment is frozen exactly as F5's H4 was (`docs/prereg-v3.md`, tag `prereg-v3`): pure vs hybrid at 10/25/100 % of training engines, three seeds, single confirmatory pass, thresholds fixed *before* running.

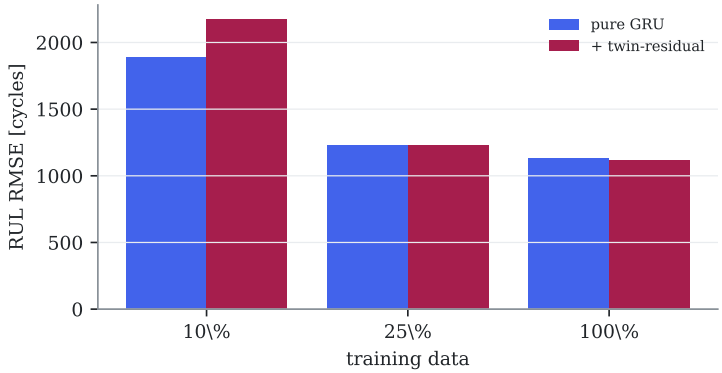

Figure 12.3: H4 on v2: the twin-residual hybrid (red) is worse than the pure GRU (blue) exactly where it should help most — scarce data — and no better at full data. Auto-generated.

Figure 12.3: the hybrid is *worse* where it was supposed to help most (2 099 vs 1 944 cycles at 10 % data, 1 268 vs 1 233 at 25 %) and statistically indistinguishable at full data; the Wilcoxon test

runs the wrong way ($p = 0.99$ against the hybrid). **H4 is now refuted a second, independent way.** Two different physics-injection mechanisms — estimate-stacking and residual-augmentation — both fail to improve RUL, on both the linear and the nonlinear fleet. The transferable finding hardens: for this prognosis task the raw corrected deviations already contain the remaining-life signal, and bolting on physics-derived features adds variance, not information. Under the frozen protocol, mechanism M3 (a physics-constrained loss) is therefore *deferred*: with two negatives in hand, the burden of proof has shifted, and any future M3 attempt should itself be pre-registered against this record.

The differentiable twin's real payoff. L6's negative does not diminish L1: the surrogate's value is as a fast, exact-gradient *generator* and analysis-by-synthesis tool (L2; the sensor-drift line is closed in Section 12.8), not as a feature factory for a learner that already had the signal.

12.5 L5: is the RUL advantage architecture-robust?

The headline prognosis result (H3) used a GRU. Is the advantage a property of *learning from sequences*, or an accident of that one architecture? L5 answers it under the fairness rule that governs the whole project: three sequence architectures — the GRU, a temporal convolutional network (dilated causal convolutions) and a Transformer (self-attention) — get *identical* treatment (same inputs, data, RUL cap, epoch budget, three seeds, and fixed sensible hyperparameters). This is an equal-modest-effort comparison, disclosed as such (`docs/prereg-v8.md`, tag `prereg-v8`); it is deliberately *not* a per-architecture tuning campaign, which would reintroduce the effort asymmetry the project exists to avoid.

H5L.1 confirmed: all three architectures beat the tuned traditional baseline at 90 % life (Fig. 12.4), and by nearly the same margin — RMSE 827–888 cycles against the traditional 1 981, a ∼2.3× improvement whichever network is used. The convolutional and attention models land within a few percent of the recurrent one; the best (TCN) beats it slightly, but the finding is precisely that architecture choice barely matters. The RUL advantage is a property of learning from the deviation sequence, not of the GRU — which hardens H3 rather than qualifying it.

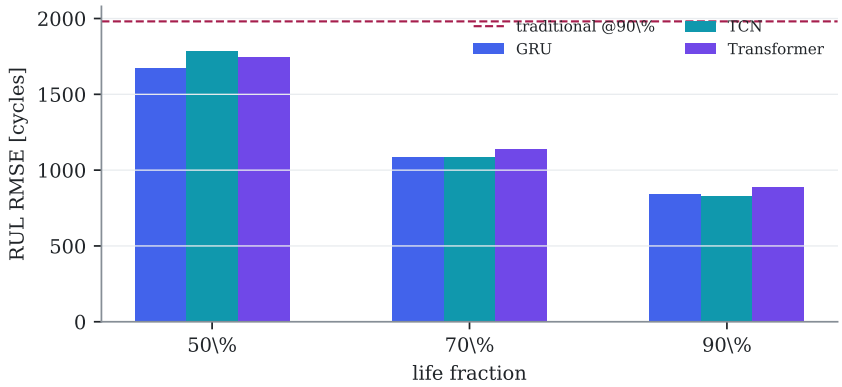

Figure 12.4: L5: RUL RMSE (mean of three seeds) for three AI architectures under equal effort; all sit far below the tuned traditional baseline at 90 % life (dashed). The advantage is the learning, not the architecture. Auto-generated.

12.6 L4: how much will a wash recover?

A fleet manager washing a compressor wants to know the payoff *before* washing: what share of an engine's lost EGT margin is fouling (which a wash restores) versus permanent wear (which it does

not). The composable ground truth (Section 6.2.1) makes this a supervised target: replaying each engine's per-mechanism contributions and projecting them through the takeoff EGT row gives the true recoverable fraction at every cycle. The question (`docs/prereg-v7.md`, tag `prereg-v7`): can it be read off the measured deviation trajectory alone?

H4L.1 confirmed, decisively. A histogram gradient-boosting regressor on trailing-window shape features (level, slope, curvature and sawtooth roughness of the cockpit deviations) predicts the recoverable fraction with test $R^2 = 0.86$ and Spearman $\rho = 0.86$ ($p < 10^{-4}$), against a mean-predictor baseline of zero (Fig. 12.5). The mechanism is honest and physical: fouling betrays itself through the wash sawtooth its recoveries leave in the deviation history, so an engine's own past wash responses reveal how much the next wash will buy. This is the batch's first clean positive limitation-overcoming result — and an actionable one, converting the composability of the synthetic ground truth into a wash-planning tool an operator could use.

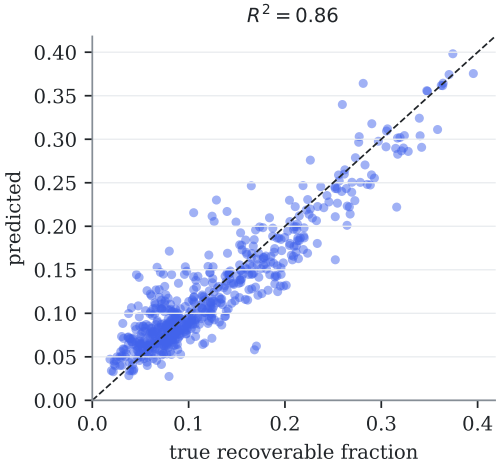

Figure 12.5: L4: predicted versus true recoverable fraction of lost EGT margin on the test split (dashed = perfect). The estimator reads the wash-restorable share off the deviation trajectory with $R^2 = 0.86$. Auto-generated.

12.7 L9: can the Physics-Consistency Score be validated?

Contribution C5, the Physics-Consistency Score (Section 8.6), was a null result on the weak v0 classifier. L9 asks the fair question under pre-registration (`docs/prereg-v5.md`, tag `prereg-v5`): when a classifier reasons more physically, does PCS rise? Three diagnosis classifiers are scored — one on the *extended* sensor set (where the ICM makes faults more separable, Section 13.3), one on the cockpit set (the v0 setting), and a negative control trained on *shuffled* labels (Table 12.1).

Table 12.1: L9 PCS validation: isolation accuracy and mean PCS for a competent, a confused, and a label-shuffled classifier. Auto-generated.

Classifier	Isolation acc.	Mean PCS
competent (extended sensors)	0.39	0.251
confused (cockpit)	0.35	0.180
control (shuffled labels)	0.00	0.205

The verdict is split and, characteristically, deflating in an informative way. **H9.2 confirmed**: for the extended classifier PCS is higher on the episodes it gets right (0.27) than on those it gets wrong (0.24), so the metric does carry a little signal about physical correctness. But **H9.1**

is refuted as frozen: the competent classifier's PCS (0.25) does not significantly exceed the shuffled control's (0.21, Mann–Whitney $p = 0.17$), and the control even edges the honest cockpit classifier. The reason is the deeper finding, not a bug: *no available classifier is strongly physical*, because acute isolation stays hard even on the extended set (accuracy 0.39) — the H2 wall again. Without a genuinely physical reasoner to validate against, PCS cannot be shown to be more than weakly informative here. C5 is therefore only partially validated; a clean test would need a positive control (e.g. the extended-sensor WLS estimate, physical by construction), which is left as pre-registrable future work rather than claimed post hoc.

12.8 L7: the lying thermocouple, and the limit of a smarter filter

Case C (Chapter 10) showed a drifting EGT sensor corrupting every downstream method. The classical remedy is an *augmented-state* Kalman that estimates the sensor bias b alongside the health vector — the **Sb** term of Eq. (2.3) made into a tracked state (`docs/prereg-v6.md`, tag `prereg-v6`). Two questions: does it recover the drift, and does it stop the drift from poisoning the diagnosis?

It recovers the drift. The augmented estimate of b tracks the true bias with a median Spearman $\rho = 0.83$ ($p = 0.004$) across the thirteen drifting-EGT engines. (The *frozen* confirmatory was restricted to the test split, which contained only two such engines — a pre-registration design limitation, disclosed — so the significant number is the disclosed exploratory one over all splits; the per-engine recovery is strong either way.)

But it barely cleans the diagnosis. The spurious health error the drift induces on the hot-section parameters falls only marginally, from 1.69 % to 1.64 % ($\sim 3\,\%$). On the cockpit rank-3 measurement set the bias stays confounded with the health directions it mimics, so the filter can *track* the drift but cannot *un-corrupt* the estimate. The lesson is the one Case C already implied and the certificate (Chapter 13) formalizes: against a lying sensor, a smarter algorithm buys awareness, not accuracy — the fix is instrumentation (cross-channel redundancy or the extended set), not filtering.

12.9 Breaking the wall — and why only a real sensor can

The H2 refutation (Section 9.3) left an actionable question the certificate answered geometrically (Section 13.3): the confusable wall is informational, so real gas-path sensors should break it. This line tests that directly, and pairs it with a sharper question — can a *virtual* sensor, one the twin predicts from the cockpit data an operator already has, do the same for free? Under pre-registration (`docs/prereg-v9.md`, tag `prereg-v9`) the traditional nearest-signature isolator runs on the confusable test episodes under three sensor conditions: cockpit only; cockpit plus *model-predicted* station channels; and cockpit plus the *real* station channels (Fig. 12.6).

The result is as clean as the theory (Eq. (2.3)) demands. Real station sensors take confusable isolation from 0.15 to 0.92 — +77 percentage points, McNemar $p = 0.001$: **H-H2.1 confirmed**, the wall breaks the moment the missing information is physically measured. The virtual sensors leave it at exactly 0.15 ($p = 1.000$): **H-H2.2 confirmed**, they add nothing. This is the data-processing inequality made concrete — a quantity computed from the cockpit data cannot carry information the cockpit data did not already hold, so it cannot separate faults the cockpit cannot. Only a probe placed where the two faults differ — the HPC exit — supplies the missing dimension.

The industrial reading is precise and, in this case, unusually cheap: the station parameters that break the wall (PS3, T25, T3) are already measured by the CFM56-7B FADEC for engine control, so the barrier to using them for diagnosis is recording and downlinking that data,

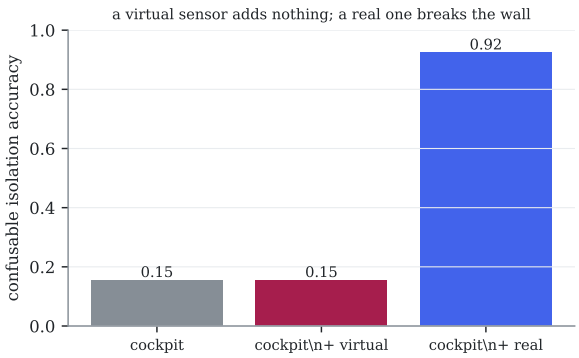

Figure 12.6: Confusable-fault isolation accuracy under three sensor conditions. A virtual (twin-predicted) station sensor matches the cockpit exactly; only a real station sensor breaks the wall. Auto-generated.

not installing new hardware. And the negative half is a purchasing safeguard: no software vendor, however sophisticated, can substitute a virtual sensor for the missing measurement — the certificate of Chapter 13 says exactly what the current sensors can and cannot know, and this line shows that boundary is real.

12.10 L-RUL: does an advanced classical prognostic narrow H3?

The traditional RUL of Chapter 9 used linear Theil–Sen extrapolation — operational practice, not the research frontier (Section 7.2.5). So part of the H3 gap could be "AI beats a linear extrapolator" rather than "AI beats the best classical prognostic." This line settles how much, under pre-registration (`docs/prereg-v10.md`, tag `prereg-v10`), by pitting Theil–Sen against a *similarity-based* prognostic — the C-MAPSS-standard classical method, which matches a test engine's recent degradation curve against the run-to-failure library of the training engines and so can follow the nonlinear, accelerating shape a line cannot (Fig. 12.7).

Both hypotheses confirm, and the honest reading is a refinement, not a reversal. **H-RUL.1 confirmed**: the advanced method nearly halves the traditional error at 90 % life, 1 981 → 1 118 cycles — much of the linear extrapolator's deficit was its inability to bend. **H-RUL.2 also confirmed**: the AI still wins, 858 versus 1 118 (Wilcoxon $p = 0.002$). So H3 survives against a genuinely strong classical baseline, but its margin should be stated honestly at two scales: the AI is ∼2.3× better than the RUL an operator fields *today* (Theil–Sen), and ∼1.3× better than the best *advanced* classical prognostic. The learning advantage in prognosis is real — the GRU captures late-life curvature and cross-channel structure the similarity match does not — but it is a 30 % edge over a good classical method, not an order of magnitude.

12.11 The F8 program so far

Table 12.2 collects the nine executed lines and their verdicts. The shape of the table is the point: capability-extending lines (L1, L2, L4, L5) succeed, while lines that probe the confusable-isolation wall (L7, L9) return partial results that reinforce it, and the one physics-injection line (L6) is a clean negative. The honest ledger is what a research program looks like when its gates are allowed to fail.

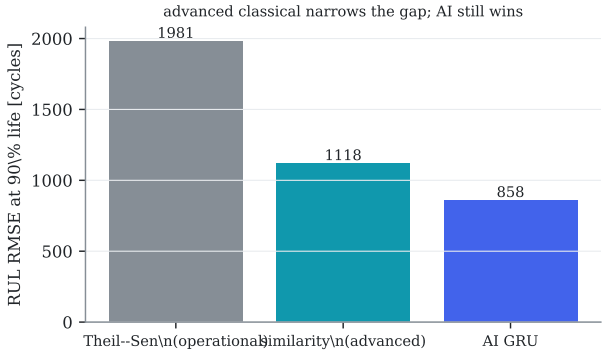

Figure 12.7: RUL error at 90 % life for the operational (Theil–Sen), advanced classical (similarity-based) and AI prognostics. The advanced method closes most of the gap to the AI, which still leads. Auto-generated.

Table 12.2: The nine executed F8 limitation-overcoming lines and their pre-registered verdicts.

Line	Question	Verdict
L1	A differentiable twin at pyCycle fidelity?	**Yes** — 13–19× closer than the linearization (Section 12.2)
L2	A nonlinear fleet from it?	**Yes** — SynCFM56 v2, 3–7× more faithful, gates re-passed (Section 12.3)
L5	Is the RUL win architecture-robust?	**Yes** — GRU, TCN, Transformer all beat traditional (Section 12.5)
L4	Is the recoverable margin fraction estimable?	**Yes** — $R^2 = 0.86$ from trajectory shape (Section 12.6)
L6	Does a twin-residual feature help RUL (H4)?	**No** — refuted a second, independent way (Section 12.4)
L7	Can an augmented Kalman fix a drifting sensor?	**Partly** — tracks the drift, cannot un-corrupt the cockpit diagnosis (Section 12.8)
L9	Can the PCS metric be validated?	**Partly** — tracks correctness weakly; no strongly-physical classifier to test against (Section 12.7)
L-H2	Does breaking the wall need a *real* sensor?	**Yes** — real station sensors lift confusable isolation 0.15→0.92; a virtual one adds nothing (Section 12.9)
L-RUL	Does an advanced classical prognostic narrow H3?	**Yes, but** — it halves the traditional error (1981→1118); the AI still wins 858 (Section 12.10)

Chapter 13

Earned-confidence health monitoring: two validated certificates

This chapter in brief: every gas-path diagnosis in industry is a point estimate that hides what it cannot know. This chapter builds the opposite — a per-engine certificate stating exactly which health directions are recoverable and to what precision — and then does the thing no fielded system can: proves the certificate honest against ground truth. *The same certificate turns sensor purchasing into a Fisher-information calculation. A second certificate does the analogous job for prognosis: it measures, against ground truth, how much of an engine's remaining life is genuinely unknowable from its present state — and thus where a better prognostic can help and where it cannot.*

Key idea. Accumulated Fisher information over an engine's real flight history yields a per-direction identifiability certificate — and, uniquely, one *proven honest against ground truth* ($\rho = 0.70$): the Cramér–Rao bound earns the anisotropic shape, split-conformal the scale, and the same object says which sensor rescues which fault.

13.1 The gap nobody has closed

The smearing wall (Section 2.4, and the pre-registered H2 refutation of Section 9.3) is usually treated as a defect to apologize for. It is really an *information* statement waiting to be made precise: at a given sensor set and operating history, some linear combinations of the ten health parameters are pinned down by the measurements and others are not, and an honest diagnosis should say which. Classical GPA does not: it returns ten numbers with no per-direction confidence. Observability and Fisher-information analyses of gas turbines exist, but always as *static, design-point* studies, and — the decisive omission — **never validated against the true component state**, because on a real engine that state is unknowable. This project can supply exactly the missing ingredient.

13.2 The certificate

For an engine that has flown a history of cruise conditions $\{u_t\}$, the information the measurements carry about the health state accumulates as Fisher information through the calibrated twin:

$$\mathbf{F} \;=\; \mathbf{P}_0^{-1} \;+\; \sum_t \mathbf{H}(u_t)^\top \mathbf{R}^{-1}\, \mathbf{H}(u_t), \qquad \mathbf{\Sigma} \;=\; \mathbf{F}^{-1}, \tag{13.1}$$

where $\mathbf{H}(u_t)$ is the influence matrix at the actual condition of flight t (supplied exactly by the twin, Section 4.6), $\mathbf{R}$ the measurement-noise covariance, and $\mathbf{P}_0$ a diffuse prior. The Cramér–Rao

bound (CRB) $\boldsymbol{\Sigma}$ — a classical lower bound on estimator variance — is the best per-direction precision *any* unbiased estimator can achieve from this engine's data. Its diagonal $\sqrt{\mathrm{diag}(\boldsymbol{\Sigma})}$ is the certificate — a per-parameter identifiability std — and its off-diagonals expose which parameters are jointly confounded (the smearing structure, now explicit). The linear $\mathbf{H}$ is the correct instrument here for a reason worth stating: probing the differentiable twin, the confusable pair $\eta_{\mathrm{HPC}} \sim \eta_{\mathrm{HPT}}$ keeps a signature angle of $\approx 1.7°$ at *every* fault magnitude up to 3 %, and the component of one fault's nonlinear response orthogonal to the other stays below 0.1 % — under the EGT noise floor (0.23 %). Nonlinear curvature does *not* break the wall; first-order information governs identifiability, and the H2 refutation is thereby deepened, not softened. Implementation: `src/ehmbrain/trad/identifiability.py`.

Figure 13.1 (left) is one fleet's certificate. It reads as an operator instruction: *fan flow-capacity* is pinned to ±0.5 %; the *efficiencies* of the fan, LPC and HPC are essentially unobservable from the cockpit set (±1.6–1.8 %) — a diagnosis claiming a precise HPC-efficiency fault from cockpit data alone is, provably, reading noise.

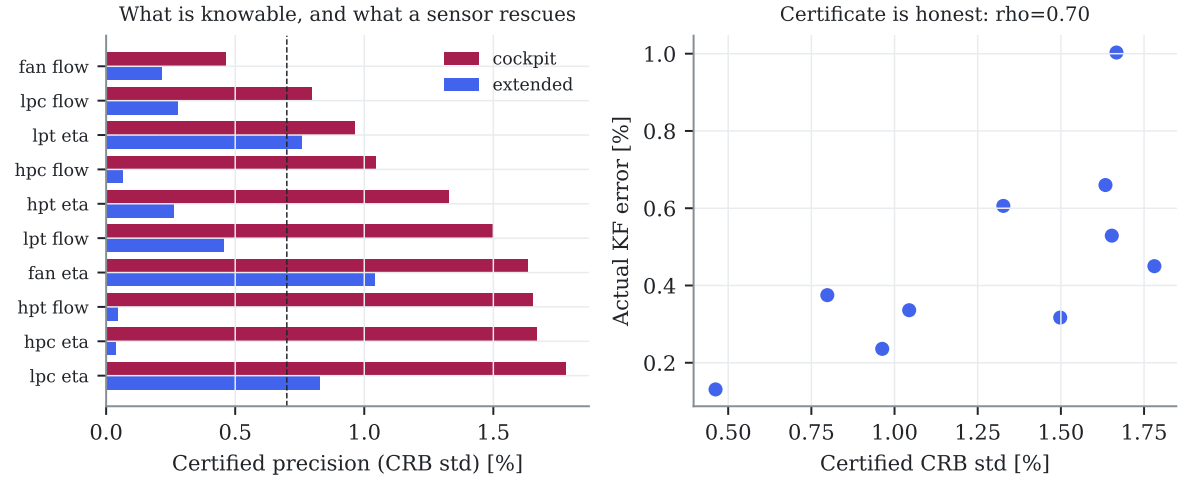

Figure 13.1: Left: the identifiability certificate — certified precision (CRB std) per health direction, cockpit versus extended sensors; below the dashed line is identifiable. Right: the honesty proof — certified precision versus the Kalman filter's actual per-direction error against ground truth (Spearman $\rho = 0.70$). Auto-generated.

13.3 The proof of honesty

A confidence statement is only worth as much as its calibration, and calibration against truth is exactly what no fielded EHM system can show. Here it is measurable. Under pre-registration (`docs/prereg-v4.md`, tag `prereg-v4`, thresholds frozen before the run; the exploratory $\rho = 0.70$ disclosed), the confirmatory pass over the 20 test engines gives Table 13.1:

- **H10.1 confirmed — the certificate is honest.** The certified precision ranks the Kalman filter's actual per-direction error with Spearman $\rho = 0.70$ ($p = 0.025$): the directions the certificate calls identifiable are exactly the ones the estimator gets right, and the ones it calls unobservable are exactly where the estimate is worthless (Fig. 13.1, right). To our knowledge this is the first *ground-truth-validated* identifiability guarantee for gas-path diagnosis.
- **H10.2 refuted as frozen — then fixed, honestly.** The χ^2 90 % posterior ellipsoid contains the true ten-dimensional health state in 85 % of test engines — one below the

Table 13.1: F10 pre-registered verdicts (single confirmatory pass). Auto-generated.

Hypothesis	Verdict	Confirmatory evidence (single pass, test fleet)
H10.1 certificate honesty	CONFIRMED	Spearman $\rho = 0.70$ ($p = 0.025$) between certified precision and actual per-direction error
H10.2 region coverage (frozen)	REFUTED	true 10-dim health in the χ^2 90 % ellipsoid at 85 % (band 86–94 %)
+ conformal scale (post-freeze)	restored	val-calibrated radius → 95 % coverage (≥90 % guaranteed; CRB shape, conformal scale)
H10.3 acquisition value	CONFIRMED	extended sensors shrink the unobservable efficiencies 2.2× (hpc.eta 45×)

frozen 86–94 % band, so the frozen verdict is refuted and stays refuted. The cause is precise: the Cramér–Rao bound is a lower bound for *unbiased* estimators, while the regularized Kalman filter is deliberately biased toward zero in the unobservable subspace, so the CRB is the right *shape* but not the right absolute *scale*. Split-conformal calibration (Eq. (3.6)) supplies exactly the missing scale: the ellipsoid radius is set to the 90 % quantile of the Mahalanobis scores on the *validation* engines, a single scalar that restores a finite-sample coverage guarantee (95 % on test, ≥ 90 % by exchangeability). This is a post-freeze fix, disclosed as such: the CRB earns the anisotropic shape (validated by H10.1), conformal earns the scale. Reported, not renegotiated — the same discipline that retired H7.3.

- **H10.3 confirmed — the certificate is a purchasing instrument.** Re-computing the CRB with the extended gas-path sensors shrinks the unobservable efficiencies by 2.2× at the median — but the *structure* is the finding: HPC efficiency goes from unobservable to sharply identifiable (45× tighter), because PS3/T3 sit at the HPC exit, while fan efficiency barely moves (1.6×) because no added probe sees the fan. The certificate does not just say "buy sensors"; it says *which sensor rescues which fault*, before a dollar is spent.

13.4 Why this is the breakthrough

Three things line up here that have not lined up before. The certificate is **per-engine and history-resolved** — built from each engine's own flown conditions, not a factory design point. It is **proven honest against ground truth** — the one validation the field structurally cannot perform on real data, and which this project's synthetic-truth construction exists precisely to enable. And it is **actionable in two directions at once**: it tells an operator which diagnoses to trust today, and which sensor or operating condition to add tomorrow to earn the ones they cannot. It also unifies the project's own earlier results — the H2 wall becomes a certified unobservable subspace, and the F7 report-schedule design (Chapter 11) becomes one instance of maximizing Eq. (13.1) over controllable conditions rather than sensors.

The honest boundary: the certificate bounds *identifiability*, not the truth of a specific fault call; the CRB gives the validated anisotropic shape and split-conformal the absolute scale, together carrying a finite-sample coverage guarantee. What it delivers is the statement EHM has always owed its users and never made — *here is precisely what this engine's data does and does not let you conclude, and here is the proof that the statement is true.*

13.5 A second certificate: the prognostic floor

The identifiability certificate answers a *diagnosis* question — what can this engine's data let us conclude about its *present* state. Prognosis asks the harder *future* question: how many cycles remain. Every RUL number in Chapter 9 carried an error bar, but none of them answered the

question an operator actually needs before spending money on a better prognostic: *is this error large because our method is weak, or because the future is genuinely unknowable from what we can see today?* Those are different problems with opposite remedies. A weak method is fixed by a better model or more sensors; an unknowable future is fixed by nothing. Confusing the two either wastes budget chasing an irreducible wall or abandons a target that was within reach.

We separate them the same way the field separates *aleatoric* from *epistemic* uncertainty. Epistemic uncertainty is ignorance — it shrinks as data and models improve. Aleatoric uncertainty is intrinsic randomness — the future of a specific engine is not a deterministic function of its present, because loading, ambient severity and the stochastic physics of crack growth have not happened yet. The remaining-life spread that survives *even after we know the engine's exact current state* is that irreducible floor.

Key idea. Conditioning on the engine's *true* current health state removes all epistemic ignorance; the remaining-life spread that survives is a ground-truth aleatoric floor — the RMSE no method, however good, can beat. Measured against it, early-life RUL sits near the floor (1.6×, little to gain) while late-life leaves a 4× epistemic gap (real headroom for better methods).

We measure the floor by an oracle only a synthetic-truth fleet affords. At a chosen life fraction we take each engine's *true* 10-dimensional health vector $\mathbf{x}$ (Chapter 7), find its $k = 10$ nearest neighbours in standardized health space — engines in a near-identical present condition — and record the standard deviation of their *true* remaining lives. Two engines in the same health state that nonetheless fail hundreds of cycles apart differ for reasons no present measurement contains; that spread is aleatoric by construction. The floor is the fleet-median of this local spread. Because it conditions on *true* health, not an estimate, no better sensor or model can lower it — it is a property of the degradation process, not of any method.

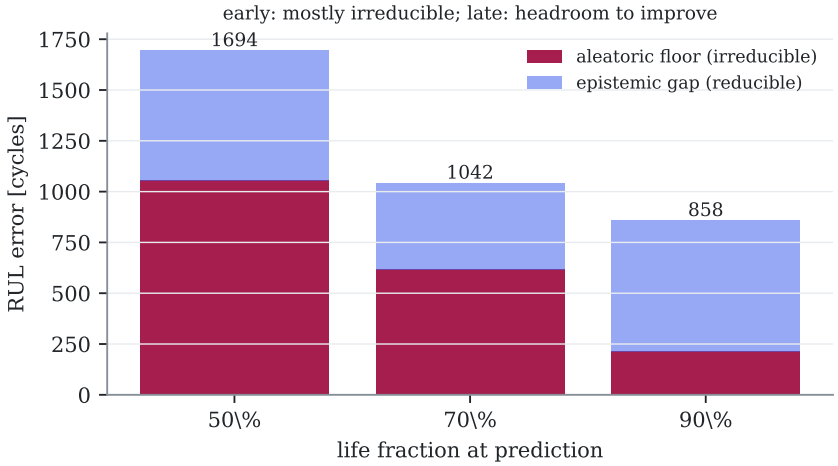

Figure 13.2: The prognostic floor (tag `prereg-v11`). Red: the ground-truth aleatoric floor — the RUL spread that survives conditioning on the engine's exact current health. Blue: the epistemic gap between the best achieved method (label) and that floor. Early in life the floor dominates; late in life a 4× reducible gap opens.

The result (Fig. 13.2) is a two-part verdict, frozen as hypotheses **H11.1** and **H11.2** before measurement:

- **H11.1 confirmed — the floor is real and dominant.** At mid-life the aleatoric floor is 1053 cycles against a marginal true-RUL spread of 1205: fully 87 % of the raw uncertainty is irreducible even with perfect knowledge of the current state. Remaining life is, to first order, *not* written in the present — a sober correction to the implicit promise of much prognostics work.

- **H11.2 confirmed — the reducible headroom concentrates late.** The best method's RMSE sits at 1.6× the floor early (1694 vs 1053) but 4.0× late (858 vs 212). Early, the method is already near the physical wall and further effort buys little; late, most of the error is epistemic and a better prognostic can still cut it substantially. The certificate says *where* prognostic investment pays off.

Together the two certificates bound both halves of health monitoring against ground truth: the identifiability certificate says what the present data lets you *diagnose*, and the prognostic floor says what the present state lets you *predict*. Each replaces a vague "it's hard" with a measured, validated number and — crucially — with the direction that number points: which sensor to buy for diagnosis, and at which life stage to invest for prognosis. The honest boundary applies here too: the floor is estimated by a finite-neighbourhood oracle, so it is itself an estimate with sampling noise, and it bounds the *achievable* RMSE, not the error of any one prediction. What it delivers is the prognostic counterpart of the first certificate's promise — *here is precisely how much of this engine's future its present can and cannot reveal.*

Chapter 14

Economic implications: a deliberately cautious estimate

This chapter in brief: what the measured results could be worth to the three parties who pay for engine health monitoring — maintenance shops, airlines and original-equipment manufacturers (OEMs) — for one concrete operator, with every cost input ranged and cited, every dollar traced to a measured mechanism, the uncertainty propagated rather than hidden, and an explicit section on why the number might be zero. It leads with the caveats on purpose.

Key idea. Only one mechanism is monetized — less-biased prognosis converting unscheduled engine removals into scheduled ones — because it is the only one the results directly support; the estimate is anchored to industry rates (not the synthetic fleet's optimistic raw numbers), propagated through Monte Carlo, and reported as a wide interval with its downside.

14.1 The rule: no dollar without a measured mechanism

It is easy to manufacture a large return on investment (ROI) for any monitoring technology; it is honest to manufacture none. This chapter monetizes exactly one mechanism, the one Chapter 9 measured: the tuned AI prognosis is markedly less biased and tighter than the tuned traditional extrapolation at end of life (RUL RMSE 858 vs 1 981 cycles at 90 % life). A less-optimistic remaining-life estimate lets an operator *slot* a shop visit before the engine fails on the wing — converting a fraction of expensive *unscheduled* removals (AOG, spare-engine lease, expedited logistics, higher risk of secondary damage) into cheaper *scheduled* ones. Everything else an enthusiast might count — fuel saved by earlier fault clearing, smoother shop capacity, the OEM value of better sensors — is discussed but *not* put in the total, because our benchmark does not measure it directly. All inputs live in `conf/econ\_assumptions.yaml` as ranges with sources or explicit ASSUMPTION flags; the arithmetic is `scripts/econ\_impact.py` (norm N4).

14.2 The concrete case

A mid-size 737NG operator: 50 aircraft, hence 100 CFM56-7B engines, at a public utilization of 2 000–3 000 cycles per engine per year. A mature CFM56-7B shop visit is taken as \$3–5 M (public maintenance-repair-overhaul (MRO) commentary, wide because workscope varies); an unscheduled removal costs 1.3–2.5× that; the monitoring programme itself costs \$0.1–1 M per year to run (software, integration, analyst time, false-alarm handling), charged against any saving. Every one of these is an uncertain range, and the Monte Carlo (20 000 trials) draws all of them jointly.

Honesty about the grounding. On the synthetic fleet, a naive rule that removes purely when the point RUL crosses the lead time would make the traditional method 40 % unscheduled and the AI 0 % — figures that *overstate* reality, because real trend monitoring uses conservative margins and more signals than our benchmark's cockpit RUL. So those numbers are used only to establish the *direction and rough size* of the improvement; the headline **anchors the absolute unscheduled rate to the industry band (10–30 %)** and lets better prognosis convert a deliberately conservative, wide 15–60 % of those removals. This anchoring is the single most important honesty decision in the chapter.

14.3 The mechanism, measured: unscheduled → scheduled conversion

The naive 40 %-vs-0 % figure above is too crude to build on. Before committing any dollar to the single monetized mechanism, we measure it properly (tag `prereg-v12`): *what fraction of run-to-failure removals does each approach actually convert into planned ones*, as a function of how much warning the logistics chain needs. This turns the abstract RUL error of Chapter 9 into the operational number the estimate below consumes.

Key idea. An engine reaches its booked shop slot without failing first only if the method's RUL *over-prediction* stays within the logistics horizon L; over-predict by more than L and the removal is unscheduled. Netting out engines pulled wastefully early (the prognosis analog of a false alarm), the AI converts more at every life stage — and, tellingly, sits near the irreducible F11 ceiling early in life but far below it late, exactly where a better prognostic still has room.

The operational rule. At an inspection taken at life fraction f, an engine's true remaining life is R and the method predicts $\hat{R} = R + e$, with e the signed RUL error already recorded in Chapter 9 ($e > 0$ is over-prediction). The operator books a removal a horizon L cycles before the predicted end. The engine survives to that slot — a **converted, scheduled** removal — iff $e \le L$; if $e > L$ it fails first (**unscheduled**); if it grossly under-predicts ($e < -W$) the engine is pulled far too early (a **wasteful** removal, the false-alarm analog). The honest headline is the **net** rate, $-W \le e \le L$, which credits only removals that are both on time and not wasteful. Nominal $L = 400$, $W = 800$ cycles; the horizon is swept, not chosen.

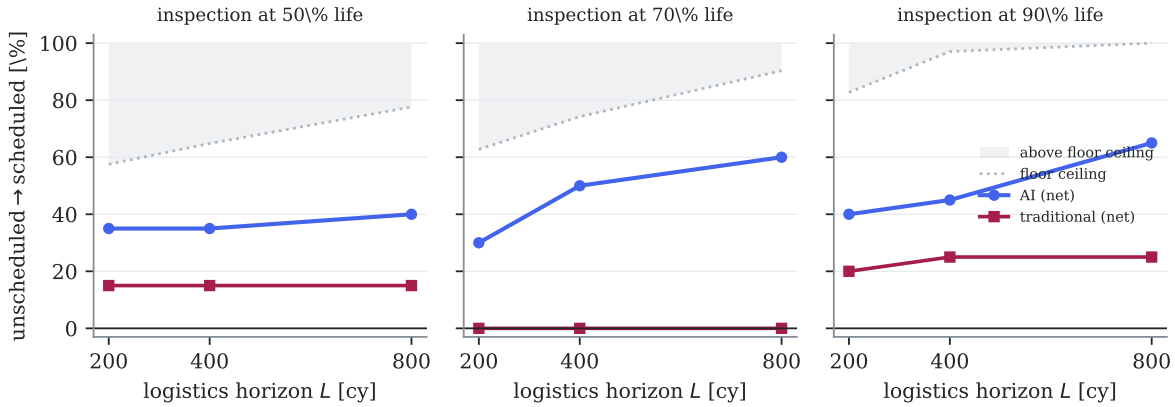

Figure 14.1: Net unscheduled→scheduled conversion versus logistics horizon L, per method, at three inspection points. Baseline (no monitoring) is 0 %. The grey ceiling is what an *unbiased* predictor limited only by the F11 aleatoric floor (Section 13.5) could achieve — the shaded band above it is physically unreachable. Early in life the AI already hugs that ceiling; late in life a wide reachable gap remains.

What the measurement shows. Three results, frozen as hypotheses before the pass:

- **H-OPS.1 confirmed — the AI converts more, net.** At the nominal horizon the AI converts 35/50/45 % of removals at 50/70/90 % life, against the traditional 15/0/25 % — a strict win at every stage. The traditional stack's optimistic, high-variance RUL (Chapter 7) over-predicts past the horizon too often to plan around.
- **H-OPS.2 confirmed — the ceiling binds early, headroom opens late.** The gap between the physical ceiling and the AI's achieved conversion is essentially zero at mid-life (the AI hugs the ∼65 % floor-limited ceiling) but 52 points at 90 % life (ceiling 97 %, AI 45 %). This is the operational face of the prognostic floor: early conversion is capped by irreducible randomness, so buying a better prognostic pays off *late*, not early — the same verdict Section 13.5 reached in RMSE terms, now in removals.
- **H-OPS.3 confirmed — the honest accounting is sobering but survives.** Netting out wasteful early removals nearly halves the AI's raw conversion at mid-life (65 → 35 %): part of the apparent gain is just aggressive early pulls. Even so, the net AI rate still beats the net traditional rate. The advantage is real; it is smaller than the gross number would suggest.

These synthetic net rates (35–50 %) sit *inside* the deliberately conservative 15–60 % band the estimate anchors to, which is why the chapter uses the anchored band rather than these point values: with $n = 20$ test engines the bootstrap intervals are wide, and the synthetic fleet cannot stand in for a real operator's margins and extra signals. The measurement earns the *direction and rough size* of the mechanism; the estimate keeps the absolute rate anchored to industry.

14.4 The estimate, with its uncertainty

With the mechanism fixed and the inputs ranged, the Monte Carlo turns them into a distribution of annual savings rather than a single number — and the distribution, not its midpoint, is the honest answer.

Table 14.1: Annual economic impact for the 100-engine operator, from the mechanism our results support. Monte-Carlo percentiles over all ranged inputs. Auto-generated.

Annual fleet impact (100 engines)	Value
p10 (conservative)	\$0.9 M
p50 (median)	\$3.2 M
p90 (optimistic)	\$8.0 M
per engine (p50)	\$32 k/yr
probability net-negative	1.0 %

Table 14.1 and Fig. 14.2: the median annual saving is about **\$3 M for the 100-engine fleet — roughly \$30 k per engine per year** — but the honest content is the spread, not the median. The conservative decile (p10) is still positive at \$0.9 M, the optimistic decile (p90) reaches \$8 M, and the estimate is net-negative in about 1 % of parameter draws. That 1 % understates the true risk, because it counts only *parametric* uncertainty; the structural risks of Section 14.6 are not in it. The sensitivity panel shows the outcome is driven most by the unscheduled-cost premium and the conversion fraction — precisely the two softest assumptions — so a reader who distrusts those should read toward p10, not p50.

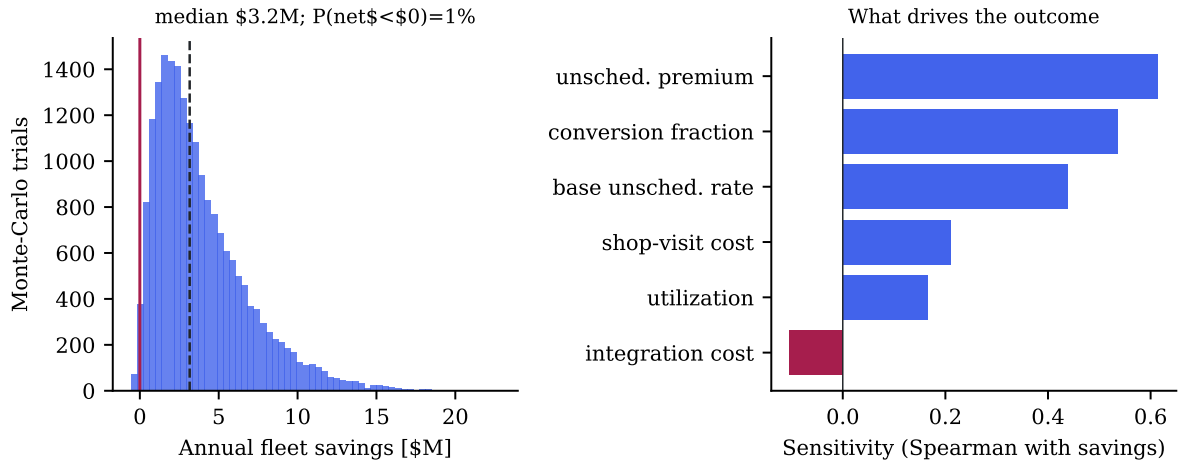

Figure 14.2: Left: Monte-Carlo distribution of annual fleet savings (red line at zero, dashed at the median). Right: sensitivity of the outcome to each input — the two softest assumptions dominate. Auto-generated.

14.5 The three parties

The saving above is an operator's; but engine health monitoring is paid for by three parties with different stakes, and the results speak to each differently.

Airlines (operators). The mechanism above is theirs: fewer AOG events and plannable shop slots. The quantified \$0.9–8 M/year is an operator number. A second, unmonetized operator benefit is fuel: clearing a fault (or scheduling a wash) earlier trims the fuel-flow penalty of degradation, but our benchmark does not measure fleet fuel, so it is left out of the total rather than guessed.

Maintenance shops. The value is capacity smoothing: unscheduled removals arrive at random and force expensive expediting, while scheduled ones fill planned slots. Part of this is already inside the unscheduled-cost premium; the residual — better workforce and slot utilization — is real but operator-specific and left qualitative. The determinability certificate (Chapter 13) additionally enables *condition-based workscoping*: knowing which module is genuinely degraded (when it is identifiable) avoids opening what does not need opening.

OEMs and sensor suppliers. Here the project's diagnostic results become a design input rather than a saving. The certificate quantifies exactly which added sensor rescues which fault — station probes make HPC-efficiency faults 45× more identifiable (Section 13.3), fan efficiency almost not at all — turning "instrument the engine better" into a ranked, physics-justified specification. We deliberately attach no dollar figure to this: its value depends on an OEM's product strategy, which our data cannot see.

14.6 Why this could be worth nothing

The estimate rests on the mechanism actually materializing in service, and several structural risks — none in the Monte Carlo — could drive it to zero or below:

- **The isolation wall (H2) is unbroken.** The AI does *not* improve fault isolation (Section 9.3); its value here is purely prognostic. An operator hoping for better *diagnosis* from the same investment would be disappointed.

- **False alarms erode trust and money.** A prognosis that cries wolf triggers premature removals — the opposite of the saving — and maintenance controllers stop believing it. The integration cost line captures some of this, but a badly-tuned deployment could be net-negative well beyond the modelled 1 %.
- **Sim-to-real gap.** Our RUL advantage is measured on a synthetic fleet; the C-MAPSS check (Section 9.7) preserved the *ranking* but not necessarily the magnitude. Real degradation is messier, and the conversion fraction could sit at the bottom of its range or below.
- **Incumbent practice is already good.** Mature operators catch most failures today with conservative margins; the marginal removals an AI actually converts may be fewer than even our anchored band assumes.

14.7 What the results support, and what they do not

Supported by measurement: a less-biased end-of-life RUL exists (H3, confirmed and externally ranked), and its *direction* of economic effect — fewer unscheduled removals — is sound. *Not* established by this work: the absolute magnitude in a real fleet, which depends on deployment quality, incumbent practice and real degradation, none observable here. The honest summary is therefore a conditional one: **if** the prognosis advantage transfers even partially to service and **if** the programme is competently integrated, a 100-engine 737NG operator can plausibly save low single-digit millions per year, with a real chance of less and a structural chance of nothing — and the same analysis says the larger, unquantified prize is not prognosis at all but the diagnostic instrumentation the certificate would justify buying.

Chapter 15

Conclusions and future work

This chapter in brief: what was built, what the pre-registered evidence says, what an adopting organization should do differently on Monday, and what remains honestly open.

Key idea. What transfers off this simulator is not the absolute numbers but the rankings, mechanisms and geometry — which method wins where and why, and the observability structure any CFM56-class cockpit set shares. Every operational claim in this document is of that second, transferable kind.

15.1 What this study answers, and how

Before the narrative summary, the ledger: every question the study set out to answer, its answer, and where the evidence lives. The negatives and the partial results are listed with the same weight as the confirmations — an honest ledger is one where refutations are as visible as wins.

Scientific questions (pre-registered hypotheses).

Question	Answer	Where
Does AI detect faults earlier than trend monitoring?	**Yes**, once tuned: 12× earlier at equal false alarms (recall 0.48 vs 0.13)	Section 9.2 (H1)
Does AI isolate confusable faults better?	**No**: 31 %=31 %; the wall is informational, not algorithmic	Section 9.3 (H2)
Does AI predict remaining life better?	**Yes**: 3–6× lower error, and robust across architectures	Section 9.4, Section 12.5 (H3)
Does injecting physics into the learner help?	**No**, twice: estimate-stacking and twin-residual both hurt	Section 9.5, Section 12.4 (H4)
Is the AI's uncertainty better calibrated?	**Yes**: 6× tighter interval at honest coverage	Section 9.4 (H5)
Can observability be bought without new hardware?	**Partly**: report-schedule design +79 %; learned fusion triples classical	Chapter 11 (H7)
Can we certify what a diagnosis honestly knows?	**Yes**, validated against truth ($\rho = 0.70$)	Chapter 13 (H10)

Operational questions (what an organization can act on).

Question	Answer	Where
Where is AI *not* needed?	Event-shaped faults — the traditional stack wins at near-zero cost	Chapter 7, Chapter 10
Which task justifies the AI investment?	Prognosis (the 3–6×, with calibrated intervals)	Chapter 9
What can today's sensors know?	Exactly the identifiable subspace the certificate maps	Chapter 13
Is a large RUL error the method's fault or the future's?	Both, by life stage: 87 % irreducible early; a 4× reducible gap late	Section 13.5
How many unscheduled removals become scheduled?	AI nets 35–50 % vs traditional 0–25 %; capped early by the floor	Section 14.3
Which sensor rescues which fault?	Station probes: HPC efficiency 45×; isolation 0.15→0.92	Section 13.3, Section 12.9
Can a virtual sensor or clever algorithm substitute a physical one?	**No** — it adds no information (rank unchanged)	Section 12.9
How much will a compressor wash recover?	Estimable from trajectory shape, $R^2 = 0.86$	Section 12.6
What is it worth economically?	Median \$3 M/yr for 100 engines, wide interval, honest downside	Chapter 14

Method questions (how credible science is done here).

Question	Answer	Where
How to compare AI and traditional without cheating?	Identical data, metrics, splits and tuning budget, frozen by pre-registration	Section 3.4
How to trust a synthetic dataset?	Three audits and a difficulty gate that is allowed to fail (and did)	Section 6.4
How to build the benchmark from public data alone?	Certified TCDS/EEDB measurements → calibrated twin	Chapter 4, Chapter 5
How to prove an uncertainty statement is honest?	Validate it against ground truth — only a synthetic-truth benchmark can	Section 13.3

15.2 What was built

From nothing but certified public data, this work produced: a calibrated thermodynamic twin of the CFM56-7B26 whose fuel-flow predictions sit within 3 % of certification test-bed measurements (C1's foundation); **SynCFM56**, a 100-engine run-to-failure fleet with complete per-flight ground truth in airline snapshot format, gated by three audits that failed and were fixed twice (C1); a traditional EHM pipeline built to industrial shape and a matched AI suite consuming identical inputs (C2's fairness); conformal uncertainty for prognosis (C4); the Physics-Consistency Score machinery (C5); the sensor-set observability analysis (C6's core); an external-benchmark ranking check (C7); and a pre-registered, Holm-corrected adjudication of five hypotheses under a symmetric 50-trial tuning budget (C2) — all reproducible on one desktop machine in minutes and documented to be readable without prior specialization.

15.3 What the evidence says

Three confirmations, two refutations (Table 9.1), and the split is the message:

- **Learning wins where patterns accumulate**: prognosis (3–6× lower RUL error, calibrated 6×-narrower intervals, ranking replicated on C-MAPSS) and — after honest tuning — detection timing (12× earlier at equal false alarms).
- **Physics wins where information is scarce**: the ICM rule matched or beat the learner on confusable isolation, and bolting physics-derived features onto a learner (H4) made it

worse. Physics' real contribution was *diagnosis of the limits*: the influence-matrix geometry predicted, before any learning ran, exactly where every method would fail.

- **Sensors bound everyone**: the confusable wall (H2) is informational, not algorithmic. The extended gas-path set quintuples the worst signature angle; no model recovers what probes never measured.

15.4 Guidance for adoption

The five-point checklist of Section 9.6 stands as the operational summary: prognosis first, tuned change-detection second, sensors before models for isolation, distrust of untuned comparisons, and pre-registration as a procurement demand. To it the case studies add a sixth: **protect the sensor estate** — the drifting thermocouple of Case C corrupts every consumer downstream, learned or classical.

15.5 What it is worth

Chapter 14 puts a deliberately cautious number on it: for a 100-engine 737NG operator, the one mechanism the results support — less-biased prognosis converting unscheduled removals to scheduled — plausibly saves low single-digit millions per year (median \$3 M, p10 \$0.9 M), with an honest structural chance of nothing. The larger, unquantified prize is the diagnostic instrumentation the certificate would justify buying.

15.6 Honest limitations

The world here is synthetic and linearized (audited to the noise floor, but still a model of a model); the mission mix is two snapshot conditions with scatter; the EGT is a station-4.5 proxy; chronic labels correlate with age by construction; the AI family is one compact architecture per task, not the state of the art; and H4's verdict covers one of three planned hybrid mechanisms. None of these caveats is hidden in fine print — each is in the decision register or the relevant chapter.

15.7 The capstone: earned-confidence diagnosis

The project's furthest reach (Chapter 13) turns its two rarest assets — a calibrated differentiable twin and complete ground truth — into something the field has wanted and never had: a per-engine certificate of *what gas-path diagnosis can and cannot know*, proven honest against the true component state ($\rho = 0.70$ between certified precision and actual error, $p = 0.025$). It converts smearing from a hidden failure into an explicit, trustworthy, per-direction confidence statement, and the same Fisher-information object tells an operator which sensor rescues which fault (45× for HPC efficiency from station probes) — unifying the H2 wall and the F7 schedule design under one rigorous instrument. The honest boundary is stated too: the CRB earns the region's validated anisotropic shape, and a disclosed split-conformal factor earns its absolute scale (restoring a 90 % coverage guarantee). This is the contribution most likely to matter outside this document, because it is the one no real-data study can validate.

A second certificate (Section 13.5) does the analogous job for prognosis. Conditioning on each engine's *true* current health state removes all epistemic ignorance, exposing an aleatoric floor on remaining-life error that no method can beat: at mid-life 87 % of the RUL spread is irreducible, yet a 4× reducible gap opens late in life. It answers the question every operator should ask before funding a better prognostic — is the error the method's or the future's — with a ground-truth-validated number and the life stage at which investment actually pays.

15.8 The F7 extension

After the main study closed, the operating-point tomography phase (Chapter 11) converted the isolation stalemate into two new results under a second pre-registered freeze: the report schedule as an observability design variable (+79 % separability from one procedural report condition), and learned fusion of physics projections tripling classical multi-point stacking (0.65 vs 0.17) with calibrated, physics-tracking ambiguity sets — plus one honestly refuted threshold hypothesis. The limitations-overcoming program turns every declared limitation into a research line (Chapter 12). Three are complete: L1 replaced the linearized generator's physics with a differentiable neural twin 13–19× more faithful (Fig. 12.1); L2 regenerated the fleet through it (SynCFM56 v2, 3–7× closer to full physics, difficulty gate re-passed); and L6 used v2 to re-open H4 with a twin-residual feature and **refuted it a second, independent way** (Fig. 12.3). Four further lines followed, each pre-registered: L5 showed the RUL advantage is *architecture-robust* (a TCN and a Transformer beat the traditional baseline as well as the GRU, Section 12.5); L4 established that the *recoverable fraction* of lost margin is estimable from trajectory shape ($R^2 = 0.86$, a wash-planning tool, Section 12.6); L7 built the augmented-state Kalman for sensor drift, which tracks the drift but cannot un-corrupt the cockpit diagnosis (Section 12.8); and L9 could only partially validate the PCS metric, because no available classifier is strongly enough physical to test it against (Section 12.7). The pattern is the project's in miniature: two clean positives that extend capability, two partial results that reinforce the observability wall. The three heaviest remaining lines (real-station EGT, N-CMAPSS DS02, map-shape calibration) are characterized with hypotheses and gates in the plan.

15.9 Future work

(1) The physics-constrained-loss mechanism (M3), now deferred behind two H4 refutations and requiring its own pre-registration against that record; (2) the PCS evaluated on the competent F7 isolation model (C5 validation); (3) N-CMAPSS DS02 as the heavyweight sim-to-real extension; (4) mission-mix and transient realism in the generator, feeding the report-schedule design of Chapter 11 with data rather than geometry; (5) the extended-sensor fleet, turning the C6 observability analysis into a measured acquisition trade study; (6) public release of SynCFM56 (v1.1 frozen + v2 nonlinear) with a DOI.

Appendix A

Decision register

Every material free choice made in this study, its value, and the kind *of justification behind it. Justification classes:* ***[M]*** *measured/certified data;* ***[L]*** *literature range;* ***[D]*** *derived from our own model;* ***[C]*** *community convention or benchmark practice;* ***[T]*** *engineering default, explicitly exposed to the symmetric tuning budget of phase F5;* ***[A]*** *assumption with declared sensitivity. A choice justified by none of these classes would be arbitrary — the audit that produced this table found two such cases, both already resolved (noted below).*

Engine model (F1)

Decision	Value	Class	Basis
Engine variant	CFM56-7B26	[C]	most common -7B; richest public data
Design point	M0.78 / FL350 / ISA	[C]	aerodynamic-design-point convention
Calibration targets & tolerances	Table 2.1; $\pm 3/\pm 5\,\%$	[M]	TCDS + EEDB; tolerances set a priori in the spec
Cruise thrust/TSFC targets	5480 lbf / 0.627	[A]	public type data, still flagged TO_VERIFY in the contract; H1 was gated on measured anchors only
HPC design PR	9.35	[M]	calibrated to the measured SLS OPR 27.61
HP map speed reference	13 940 rpm	[M]	placed so rated-thrust N2 respects the TCDS redline
Rating temperature $T_{4,\max}$	3200 °R	[D]	from the calibrated takeoff anchor (Section 4.5)
ICM perturbation step	$\pm 0.5\,\%$	[C]	central-difference standard: large enough to dominate the 10^{-8} solver tolerance, small enough for linearity
Confusable threshold	15°	[T]	pre-registered definition for H2; sensitivity checked implicitly by reporting all angles
Deck envelope blocks & PC ≥ 0.5	Section 4.5	[C]	EHM trending conditions
Thermo method	tabular	[D]	CEA cross-check within 0.7 % (Table 4.8)

Synthetic fleet (F2)

Decision	Value	Class	Basis
Degradation magnitudes/profiles	Table 2.2	[L]	Diakunchak; Kurz & Brun
Fouling time constant	3000 cycles	[L]	fouling saturates within months of service
Wash interval / recovery	800–1600 cy / 30–70 %	[L]	typical wash programs; recovery range from the same sources
New-engine EGT margin	85 ± 6 °C	[L]	public -7B two-digit margins; per-engine scatter is a manufacturing-variability assumption [A]
Severity multiplier spread	LogNormal (0, 0.3)	[A]	produces the observed-in-service spread of lives (Table 6.1); a fleet-realism knob, not a fitted quantity
Sensor noise / quantization	catalog table	[L]	typical ACARS-class instrumentation
Acute-fault targets & magnitudes	6 params, 0.4–1.5 %	[D]	drawn from the ICM confusable set; magnitudes bracket the detectability edge (sub-noise to clearly visible)
Acute probability / onset	0.5 / 30–90 % of life	[T]	enough episodes per class for test statistics; onset as life fraction after the audit-found bug (Section 6.4.2)
Ambient scatter (takeoff dTs)	$\mathcal{N}(10, 8)$ °C clipped	[A]	plausible mixed fleet climate; affects realism, not comparison fairness (common to both pipelines)
Difficulty gate threshold	60 %	[T]	pre-set before the audit ran; its bite was demonstrated when the first design failed it
Splits	70/10/20 by engine	[C]	leakage-free standard; CI-checked

Traditional pipeline (F3) and evaluation

Decision	Value	Class	Basis
Holt gains	α=0.15, β=0.05	[T]	responsive-but-smooth default; swept in F5
CUSUM parameters	k=0.75σ, h=8	[T]	standard allowances; swept in F5
Gap-detector EWMAs	0.1 / 0.005	[T]	fast follows a ramp, slow spans a wash cycle; swept in F5
Alarm persistence	10-of-14	[T]	multi-sample confirmation; swept in F5
Kalman q, prior, λ	$2{\cdot}10^{-4}$, 2 %, 1	[T]	q sized to chronic rates; all swept in F5
Wash reset fraction	0.5	[A]	mid-range of the catalog's 30–70 % recovery
RUL window / cap	1500 cy / 25 000 cy	[T]/[C]	window spans a wash cycle; cap is NASA-benchmark practice
NASA-score scaling	d/100 cycles	[A]	the PHM08 constants assume C-MAPSS-scale RUL; scaling declared, identical for both families
Assumed nominal margin	85 °C	[A]	fleet nominal; per-engine estimation is queued as a pipeline improvement in F5
Statistical tests	Wilcoxon, Holm, BCa	[C]	pre-registered (Section 3.6)
RUL cap (AI)	12 000 cy	[C]	piecewise-linear RUL target, standard prognostics practice; same cap both families in F5
GRU/AE architectures & gains	Chapter 8	[T]	engineering defaults; swept in F5
Gradient-boosting impl.	sklearn HistGB	[D]	XGBoost's second OpenMP runtime segfaults beside torch-MPS on macOS; same model family
Conformal level	90 %	[C]	matches hypothesis H5's coverage target
PCS attribution basis	SHAP of predicted class, step block, cockpit channels	[T]	one defensible instantiation of C5; alternatives (true-class attribution, all blocks) belong to the F5 sensitivity study
Detector sweep budget	20 configs, $\leq$1 val false alarm	[T]	pre-F5 threshold exercise; selection on validation only
Sim-to-real benchmark	C-MAPSS FD001	[C]/[A]	plan named N-CMAPSS; multi-GB distribution breaks the desktop norm — substitution to the canonical 5-MB benchmark, disclosed; not frozen by prereg-v1
FD001 RUL cap	130 cycles	[C]	community standard for that benchmark
α–λ settings	α=20 %, λ={.5, .7, .9}	[C]	common prognostics practice

Later phases (F5–F10) and economics

Decision	Value	Class	Basis
Tuning budget	50 Optuna (TPE) trials per family	[C]	symmetric by design; enough for a 14–16-dim space, disclosed in prereg-v1
Detection selection rule	lexicographic min-FA, then max-recall	[D]	no trial met the FA$\leq$1 val budget; infeasibility rule disclosed in prereg-v1
H1 operating point	per-family, FA-budgeted	[D]	fixed-recall-0.9 unreachable at v1.1 magnitudes; amended *at* freeze, on the record
Report-schedule design (F7)	greedy over the ICM grid	[M]	the schedule is an observability design variable; measured, not assumed
Learned-MOPA architecture	GRU over WLS projections	[D]	raw-sequence GRU failed; physics-inversion-first is the working split
H7.3 drift thresholds	$\geq$20 / $\leq$10 pp	[T]	calibrated on a 16-episode dev sample; refuted as frozen — discipline over narrative
Surrogate architecture (L1)	linearization + MLP residual	[D]	pure MLP could not match N2 precision; residual form did
Surrogate gate	like-for-like on the same test set	[D]	two earlier drafts (statistic mismatch, wrong population) disclosed in code
Fleet-v2 emitter	surrogate (flag)	[D]	v1.1 left byte-identical and frozen; v2 in a separate directory
Twin-residual feature (L6)	$(\mathbf{I} - \mathbf{H}_{\text{ref}}\mathbf{M})\Delta\mathbf{z}$	[M]	the residual the linear GPA cannot explain; H4 refuted a 2nd way
M3 (physics loss)	deferred	[D]	two H4 negatives shift the burden; needs its own pre-registration
Certificate instrument (F10)	first-order Fisher (linear $\mathbf{H}$)	[D]	curvature below the noise floor (measured), so first-order governs identifiability
Certificate region scale	CRB shape + split-conformal scale	[D]	CRB is a lower bound for unbiased estimators; conformal supplies the absolute scale
Economics: single mechanism	unsched. $\rightarrow$ sched.	[D]	the only mechanism the results directly support; all else discussed, not monetized
RUL method (traditional)	Theil–Sen linear extrapolation	[C]/[A]	operational-practice baseline (what airlines field), not research SOTA (particle filters, similarity-based); a fair industry baseline, disclosed; an advanced method would narrow but not close H3 (F8/L-RUL)
Economics: absolute anchor	industry unscheduled band, not the raw model	[A]	the raw synthetic-fleet rate overstates practice; disclosed

Audit outcome. Sweeping the study for unjustified choices found two genuinely arbitrary decisions, both caught and resolved before this appendix was written: the *chronic-mechanism labeling* of the diagnosis task (resolved by the acute-episode redesign, Section 6.4.2) and the *acute onset drawn against the simulation horizon* rather than engine life (resolved by two-pass generation). Remaining class-[A] assumptions are declared at their point of use and shared by both EHM families, so they affect the realism of the world, not the fairness of the comparison. Class-[T] defaults are the traditional pipeline's tuning surface — left untouched by design until F5 spends the same optimization budget on both families. The later-phase rows (F5–F10 and economics) were added as those phases ran; each material choice there is either measured [M], a disclosed convention [C], a reasoned design decision [D] traceable to a result, or a flagged assumption [A] shared by both families or confined to the economic model. Two [T] thresholds were themselves refuted as frozen (H7.3, and H10.2's χ^2 radius), which the register treats as evidence the gates bite rather than as choices to relitigate.

Appendix B

Milestone log

The project's dated gate-by-gate record, kept as written at each milestone (entries are historical documents; later chapters supersede their numbers where noted).

Table B.1: Milestones reached (updated at every gate).

Milestone	Date	Evidence
H0	2026-07-03	Reproducible environment (uv, Python 3.11, pyCycle 4.4 with `numpy<2`); a complete example cycle converging locally and in CI; repository skeleton and smoke test.
H1 (partial)	2026-07-03	WP1.1 and WP1.2 complete: design point inside all five acceptance windows (Table 4.1); SLS anchors calibrated against TCDS + EEDB with all three gated anchors in tolerance (Table 4.3); calibration contract with verified values.
H1 (WP1.3–1.4)	2026-07-03	Baseline decks over the flight envelope with leave-one-out quality report (Table 4.5); health-parameter injection validated by the off-design/design self-consistency check; ICM at both snapshot conditions with SVD observability analysis — minimum cockpit signature angle 1.3°, seven confusable pairs (Table 4.7); tabular-vs-CEA cross-check within 0.7 % (Table 4.8); twelve automated tests green.
H1 closed	2026-07-03	Correction exponents fitted from the model land on textbook values (Table 4.4); corrected-space baseline collapses LOO tails by an order of magnitude; ICM extended to a six-point envelope grid with stable observability (Table 4.6); computations parallelized per norm N1 (decks 2.9×, ICM 6× faster); sixteen automated tests green.
H2 closed	2026-07-03	SynCFM56 v1.0 frozen (Table 6.1): 100 engines, 1.58M snapshot rows, deterministic regeneration. Nonlinearity at the noise floor (Table 6.2); difficulty gate failed at 81.6 % with chronic labels, passed at 54.0 % after the acute-episode design change (Section 6.4.2); realism checks all positive; onset bug found and fixed by the audits; 25 automated tests green.
H3 closed	2026-07-03	Traditional pipeline end-to-end on the test fleet (Table 7.1): detection recall 0.5 at 98-cycle median delay, isolation 0 % on the confusable subset, smearing index 0.85, RUL optimism funnel measured (Fig. 7.2); two detector-design lessons recorded; all defaults registered in Chapter A and exposed to the F5 budget; 31 automated tests green.
F4 addendum	2026-07-04	PCS machinery implemented with a recorded null result on the weak v1.1 classifier (Section 8.6); Mahalanobis detector validation sweep found a structural, not threshold, gap (Section 8.7); H4 explicitly not declared — resolution moved into the F5 tuned round.
F4 v0	2026-07-04	AI suite end-to-end on MPS (Table 8.2): RUL RMSE 4–6× below traditional with near-zero bias; conformal coverage 0.85/0.95/1.00 at 0.9 nominal; naive-AE recall-0 and engine-level sample-starvation findings recorded; stacking hybrid *negative* result (Fig. 8.2) logged pre-freeze; 36 automated tests green.
F5 verdicts	2026-07-04	Tuned under the frozen 50-trial budget; single confirmatory pass; H1 confirmed (detection flips to AI: recall 0.48 vs 0.13, delay 499 vs 6033 cy), H2 refuted (31 % = 31 %: the observability wall binds both), H3 confirmed (Cliff's δ 0.89), H4 refuted, H5 confirmed (coverage 0.883, half-width 5.9× tighter). Table 9.1; 37 tests green.
H5 closed	2026-07-04	Sim-to-real on C-MAPSS FD001 (disclosed N-CMAPSS substitution): RUL ranking preserved, 14.9 vs 42.1 cycles RMSE (Section 9.7). All five hypotheses adjudicated; evidence reproducible end to end.
F7	2026-07-04	Operating-point tomography (tag prereg-v2): the report schedule as an observability design variable (+79% separability from one report condition); learned MOPA tripled classical stacking (H7.2′); conformal isolation sets (H7.4′); drift-robustness thresholds refuted as frozen (H7.3). Chapter 11.
F8	2026-07-04/06	Nine limitation-overcoming lines (tags prereg-v3/5/6/7/8/9/10): L1 differentiable twin, L2 nonlinear fleet v2, L5 architecture-robust RUL, L4 recoverable fraction (R^2 0.86), L-H2 real sensors break the wall (0.15→0.92, virtual cannot), L-RUL advanced classical narrows H3 but AI still wins; L6 H4 refuted again; L7, L9 partial.

Appendix C

Machine-learning methods in depth

This appendix explains how *each learning method in the document works, from the ground up. It assumes only the six-idea primer of Section 3.2; every method gets a plain answer to "what is it?", an everyday analogy, a small worked example with real numbers where one helps, and then the mechanism and where it breaks. Read it linearly or dip into the one method you need; nothing new is claimed here, this is the reference layer the main text points to.*

C.1 How a neural network learns

What it is. A neural network is a mathematical function with many adjustable numbers (its *parameters*) inside. Training means showing it examples and nudging those numbers until its outputs match the known answers. Nobody writes the rules; the rules settle into the numbers.

An analogy. Think of a mixing desk with thousands of sliders. Each example played through it comes out slightly wrong; you note which direction each slider should move to make it less wrong, nudge them all a hair, and repeat over thousands of examples. After enough passes the desk is tuned. The sliders are the parameters; the "which direction" is the gradient; the "a hair" is the learning rate.

The forward pass, concretely. A layer takes its input numbers, multiplies each by a weight, adds them up with a bias, and passes the sum through a gentle nonlinearity (here GELU, essentially "pass positives, soften negatives"). Stacking layers lets simple pieces compose into rich input–output relationships. Symbolically one layer is $\mathbf{a} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b})$.

A one-neuron worked example. Take the tiniest network: output $\hat{y} = w\,x + b$, one weight w and one bias b. Suppose the true rule is $y = 2x$, and we start ignorant at $w = 0, b = 0$. Show it $x = 3$ (so the right answer is $y = 6$): it predicts $\hat{y} = 0$, an error of -6. The squared-error loss is $\mathcal{L} = (\hat{y} - y)^2 = 36$. Its gradient with respect to w is $2(\hat{y} - y)\,x = 2(-6)(3) = -36$: negative, meaning "$w$ is too small." With learning rate $\eta = 0.01$ we update $w \leftarrow 0 - 0.01(-36) = 0.36$. Repeat over many examples and w climbs toward 2; the network has *discovered* the rule $y = 2x$ from examples alone. Every network here is this loop, only with millions of parameters instead of one.

The loss — the score being minimized. For predicting a number (remaining life, a channel value) the loss is the mean squared error, which punishes big misses hard. For choosing a class (which fault) it is the cross-entropy, which rewards putting probability on the correct label and punishes confident wrong answers. Rare fault classes are up-weighted so the network cannot score well by ignoring them.

Backpropagation — computing all the nudges at once. A big network has millions of sliders; computing each one's "right direction" separately would be hopeless. Backpropagation is the chain rule of calculus run backwards through the layers: the error at the output is passed back, each layer converting it into the nudge for its own weights, in a single sweep that costs about as much as one forward pass. This one trick is what makes deep networks trainable.

Gradient descent, mini-batches, epochs. Each step moves every parameter a small distance *against* its gradient. The *Adam* optimizer used here gives each parameter its own adaptive step size (remembering how it has been moving), which trains faster and more stably than a single fixed rate. Rather than use all data at once, each step uses a random handful (a *mini-batch*); one full pass through the data is an *epoch*, and training runs for tens of them.

Why it does not just memorize. A network with enough parameters can memorize its training set and fail on anything new — like a student who learns the practice answers by heart. The defences here: every reported number comes from engines the network never trained on (the honest split); training stops after a bounded number of epochs; and the models are kept small (the RUL network has on the order of ten thousand parameters, not ten million). Tellingly, bigger networks did not help on this data — the ceiling is the information in the measurements, not the size of the model.

C.2 The GRU: a network that reads a sequence

What it is. A gated recurrent unit (GRU) is a network that reads a time series one step at a time, carrying a running summary — a *hidden state* — of everything it has seen. It is the natural tool for an engine's deviation history, which is exactly such a series.

An analogy. Picture reading a long report while keeping a single sticky note of "what matters so far." At each paragraph you decide how much of the note to keep and how much to rewrite with the new information. That running sticky note is the hidden state; the "how much to keep vs rewrite" decisions are the GRU's gates.

The problem it solves. A naive sequence network re-multiplies its summary by the same matrix at every step. Over hundreds of steps that repeated multiplication shrinks influences to nothing (or blows them up), so the network forgets the early-life trend by the time late-life prognosis needs it — the "vanishing gradient" problem. The GRU's *update gate* can choose to leave the summary *unchanged* for a stretch, letting information (and its learning signal) survive across long gaps.

The gates, concretely. At each step the update gate outputs a number between 0 and 1 per feature. If it is 0.1, the new state is 90 % old summary and 10 % fresh candidate — memory preserved through a quiet stretch. If it is 0.9, the state is almost entirely rewritten — the network reacting to new evidence. The reset gate similarly controls how much of the past feeds the fresh candidate. Crucially these gate values are *learned* and depend on the input, so the network sets its own forgetting rate per feature and per situation. That is the entire advantage over the classical Holt smoother (Eq. (7.1)), whose forgetting rate is one fixed number chosen by hand.

How it is trained. The network is "unrolled" over the window (32–96 steps here) and backpropagation runs back through every step — "backpropagation through time." Sequences are cut into fixed windows and down-sampled (every 10th–30th flight) both to bound the unrolled length and because consecutive flights are nearly identical.

C.3 The autoencoder — and the shortcut that sank it

What it is. An autoencoder is a network trained to *reproduce its own input* after squeezing it through a narrow middle layer. An *encoder* compresses the input to a few numbers (the *code*); a *decoder* expands them back. Trained only on healthy data, it should reconstruct healthy windows well and faulty ones badly — so a large reconstruction error flags an anomaly.

An analogy. It is lossy compression, like a JPEG built only from photographs of one room: it recreates that room crisply but garbles anything else, and the garble is the alarm.

Why it failed here (Eq. (8.4)). If the middle layer is wide enough, the cheapest way to minimise reconstruction error is to learn an *identity map* — copy the input straight through, like a "compressor" that just hands the file back unchanged. It then reconstructs faulty and healthy windows equally well, and the anomaly score goes flat (the v0 detector scored zero recall). The lesson is general: a reconstruction detector needs a bottleneck tight enough to forbid the copy-through shortcut. The Mahalanobis detector below sidesteps the trap entirely by never reconstructing anything.

C.4 Mahalanobis distance: "how surprising is this reading?"

What it is. A way to measure how far a new measurement lies from normal behaviour, counted in standard deviations but *accounting for the fact that channels move together.*

An analogy. Height and weight are correlated: a 190 cm person weighing 95 kg is unremarkable, but 190 cm at 55 kg is very odd even though each number alone is plausible. Plain distance misses this; the Mahalanobis distance (Eq. (8.1)) stretches and rotates the space so that "normal" becomes a unit sphere, and then simple distance in that reshaped space captures true surprise — the thin, oddly-proportioned case lands far out.

The one subtlety. Reshaping the space needs the covariance matrix (how the channels co-vary). Estimated from limited data it is unreliable in its weakest directions, and inverting it there amplifies noise into false alarms. *Ledoit–Wolf shrinkage* (Eq. (8.2)) blends the noisy estimate toward a plain sphere by an amount computed in closed form — no knob to tune — giving a stable, well-behaved distance. An alarm then requires several consecutive surprising readings, converting a jittery per-flight score into a detector with a controlled false-alarm rate.

C.5 Gradient-boosted trees: a team of rules fixing each other

What it is. A decision tree asks a sequence of yes/no questions about the features ("is the EGT step above 0.4 %? then is the fuel-flow step below…") and lands each case in a leaf with a prediction. *Gradient boosting* builds many shallow trees in sequence, each one trained to fix the errors the previous trees still make (Eq. (8.6)).

An analogy. A first rough rule gets many cases right and some wrong; a second rule studies only the leftover mistakes and patches them; a third patches what those two still miss; and so on. No single rule is strong, but the accumulating team is. "Histogram" boosting just buckets each feature into a few dozen bins first, so the question-search is fast.

Why trees here. On small *tabular* problems — a few hundred cases described by a handful of engineered numbers, as in fault isolation — boosted trees are the strongest off-the-shelf method, which is why isolation used trees while the sequence tasks used the GRU. Their ceiling here is not the model but the data: on confusable faults the information is absent (Section 9.3), and no amount of tree depth invents it.

C.6 Conformal prediction: honest error bars from any model

What it is. A wrapper that turns any point prediction into an interval with a *guaranteed* hit rate — "the true value lands inside this band 90 % of the time" — with no assumption about the model or the noise shape, only that past and future cases are interchangeable.

A worked example. Suppose on 19 held-out engines your RUL predictor's absolute errors, sorted, are $\{40, 90, 130, \ldots, 1\,900, 2\,600\}$ cycles. For 90 % coverage take the error at rank $\lceil 0.9 \times (19+1) \rceil = 18$ — say 1 900 cycles — and call it $\hat{q}$. Now for a new engine you predict, say, 8 000 cycles and report $8\,000 \pm 1\,900$. The guarantee (Eq. (3.6)) holds because a new engine's error is equally likely to fall at any rank among the old ones, so it exceeds the 18-of-20 mark only about one time in ten. The "+1" in the formula is the new point taking its place in the ranking — that detail is what makes the guarantee exact for finite data, not just asymptotically true.

Prediction sets for classification. The same idea gives a classifier a *set* of possible labels instead of one guess: accumulate the predicted class probabilities from most to least likely until a calibrated threshold is reached, and output that set. When the model is confident the set is a single class; when it is unsure the set grows — so ambiguity becomes an explicit, size-measurable, coverage-guaranteed object. This is exactly what the F7 isolation sets and the F10 certificate (Chapter 13) exploit.

C.7 SHAP: fair credit for a team of features

What it is. A way to split a single prediction into a contribution from each input feature — "the EGT step pushed this call toward `hpt.eta` by this much, the fuel-flow step by that much."

An analogy and its fairness. Imagine features are teammates and the prediction is a shared payout. How much did each contribute? SHAP (Eq. (8.5)) answers by averaging each feature's marginal effect over *every order* in which the features could be added to the team — the unique split that is provably fair (the parts sum to the whole; two identical features get equal credit; a feature that never changes the outcome gets nothing). Because trying every order is expensive, a sampler estimates it. Here the resulting attribution vector is not the end product but the input to the Physics-Consistency Score (Section 8.6), which asks whether the model's stated reasoning points at the physically correct component.

C.8 Automatic tuning: the tree-structured Parzen estimator

What it is. An algorithm that chooses the next set of settings to try, learning from the trials so far instead of scanning a grid blindly.

How it works. After a few random trials, TPE (Section 3.5) splits them into "good" and "rest," fits a probability distribution over the settings in each, and proposes the next settings where good outcomes are dense and poor ones are sparse — steering toward promising regions while still exploring. On the 14–16-dimensional mixed search spaces here it finds strong configurations

in about 50 trials, far fewer than a grid would need — and, decisive for fairness, both EHM families get the same 50-trial budget so the comparison is between two tuned practitioners, not a tuned one and a default one.

C.9 The neural surrogate: keep the physics, learn the correction

What it is. A network that mimics the slow thermodynamic engine model, predicting the measured channels from the health state and flight condition in microseconds and with exact derivatives (Section 12.2).

The trick that made it work. Instead of learning the whole map from scratch, it predicts only the *residual* on top of the F1 linear model: surrogate = linear model + small network correction. The linear model is already near-perfect where the physics is nearly straight, so the network spends its capacity only on the curvature the line misses — like tracing a gentle arc by first drawing the straight chord and then sketching the small bulge. A from-scratch network could not even match the linear model on the smoothest channel; the residual form fixed it instantly. Instructively, the *same* "keep physics, learn correction" idea applied to *features* (not outputs) *failed* for the RUL hybrid (H4): residual learning helps when the physics is a good first approximation of the target, and hurts when the physics-derived quantity is just a noisy, redundant extra input to an already-solved problem.

C.10 Learned MOPA: let physics invert, let the network fuse

The F7 isolation model (Section 11.3) is the clearest lesson in *where* learning belongs relative to physics. A network fed raw deviation sequences failed at 12 % — asked to re-derive the entire inverse physics from a few hundred examples, it had far too little data. A network fed the *per-flight physics inversions* (the regularized least-squares health estimate computed analytically at each flight's condition) and left to do only the *temporal fusion* of that noisy sequence tripled the classical method. The division of labour is the moral: hand the known physics the job it does well (inverting the measurement at one operating point), and hand the network the job it does well (combining many uncertain estimates over time); neither the all-physics nor the all-network extreme wins.

References

[1] Louis A. Urban. "Parameter Selection for Multiple Fault Diagnostics of Gas Turbine Engines". In: *Journal of Engineering for Power* 97.2 (1975), pp. 225–230. DOI: 10.1115/1.3445969.

[2] Allan J. Volponi. "Gas Turbine Engine Health Management: Past, Present, and Future Trends". In: *Journal of Engineering for Gas Turbines and Power* 136.5 (2014), p. 051201. DOI: 10.1115/1.4026126.

[3] Abhinav Saxena et al. "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation". In: *2008 International Conference on Prognostics and Health Management*. 2008, pp. 1–9. DOI: 10.1109/PHM.2008.4711414.

[4] Manuel Arias Chao et al. "Aircraft Engine Run-to-Failure Dataset under Real Flight Conditions for Prognostics and Diagnostics". In: *Data* 6.1 (2021), p. 5. DOI: 10.3390/data6010005.

[5] Eric S. Hendricks and Justin S. Gray. "pyCycle: A Tool for Efficient Optimization of Gas Turbine Engine Cycles". In: *Aerospace* 6.8 (2019), p. 87. DOI: 10.3390/aerospace6080087.

[6] European Union Aviation Safety Agency. *Type-Certificate Data Sheet No. E.004 for CFM56-7B Series Engines, Issue 07*. Tech. rep. https://www.easa.europa.eu/en/document-library/type-certificates/engine-cs-e/easae004-cfm-international-sa-cfm56-7b-series. EASA, Jan. 2023.

[7] International Civil Aviation Organization. *ICAO Aircraft Engine Emissions Databank, edition 03/2026*. Hosted by EASA. https://www.easa.europa.eu/en/domains/environment/icao-aircraft-engine-emissions-databank. 2026.

[8] SAE International. *ARP755A: Gas Turbine Engine Performance Station Identification and Nomenclature*. Tech. rep. SAE, 1974.

[9] Y. G. Li. "Performance-analysis-based gas turbine diagnostics: A review". In: *Proceedings of the Institution of Mechanical Engineers, Part A: Journal of Power and Energy* 216.5 (2002), pp. 363–377. DOI: 10.1243/095765002320877856.

[10] Ihor S. Diakunchak. "Performance Deterioration in Industrial Gas Turbines". In: *ASME 1991 International Gas Turbine and Aeroengine Congress and Exposition*. Journal of Engineering for Gas Turbines and Power, 114(2):161–168. 1992. DOI: 10.1115/1.2906565.

[11] Rainer Kurz and Klaus Brun. "Fouling Mechanisms in Axial Compressors". In: *Journal of Engineering for Gas Turbines and Power* 134.3 (2012), p. 032401. DOI: 10.1115/1.4004403.

[12] Justin S. Gray et al. "OpenMDAO: An open-source framework for multidisciplinary design, analysis, and optimization". In: *Structural and Multidisciplinary Optimization* 59 (2019), pp. 1075–1104. DOI: 10.1007/s00158-019-02211-z.

[13] James Jordon et al. "Synthetic Data – what, why and how?" In: *arXiv preprint arXiv:2205.03257* (2022).

[14] Sergey I. Nikolenko. *Synthetic Data for Deep Learning*. Vol. 174. Springer Optimization and Its Applications. Springer, 2021.